

인공지능 입문

기초 개념부터 딥러닝, 트랜스포머와 RAG까지

인공지능 · 머신러닝 · 딥러닝 · LLM · RAG
입문자를 위한 종합 개념 교재

목 차

인공지능의 기초

6

인공지능이란 무엇인가	7
1.1 인공지능의 정의와 범위	7
1.2 지능이란 무엇인가	7
1.3 인공지능의 역사	8
1.4 인공지능·기계학습·딥러닝의 관계	8
1.5 강인공지능과 약인공지능, 그리고 튜링 테스트	9
1.6 인공지능의 대표적 응용 분야	9
핵심 용어	10
지능형 에이전트와 합리성	12
2.1 에이전트와 환경의 상호작용	12
2.2 합리적 에이전트	12
2.3 PEAS를 통한 과제 환경 기술	13
2.4 환경의 성질	13
2.5 에이전트의 유형	14
핵심 용어	15
탐색을 통한 문제 해결	17
3.1 문제의 형식화	17
3.2 비정보 탐색	17
3.3 정보 탐색과 휴리스틱	18
3.4 허용성과 일관성	19
3.5 탐색 알고리즘의 비교	20
핵심 용어	20
지식 표현과 추론	22
4.1 지식 기반 에이전트	22
4.2 명제 논리	22
4.3 술어 논리 개요	23
4.4 규칙 기반 시스템과 추론 방향	23
4.5 온톨로지와 개념의 조직화	24
4.6 불확실성 하의 추론 도입	25
핵심 용어	26

머신러닝의 기초

27

머신러닝 개관과 학습의 원리	28
---------------------------	----

5.1 머신러닝이란 무엇인가	28
5.2 학습의 세 갈래: 지도·비지도·강화학습	28
5.3 기본 용어: 데이터·특징·레이블·모델	29
5.4 학습은 손실 최소화이다	29
5.5 일반화와 데이터 분할	30
5.6 특징 공학 입문	30
핵심 용어	31
회귀와 경사하강법	33
6.1 선형회귀와 가설함수	33
6.2 평균제곱오차(MSE) 손실	33
6.3 경사하강법의 원리	34
6.4 배치·확률적·미니배치 경사하강법	34
6.5 다항회귀와 비선형 관계	35
6.6 과적합과 정규화(L1·L2)	35
핵심 용어	36
분류 알고리즘	38
7.1 로지스틱 회귀와 시그모이드	38
7.2 교차엔트로피 손실	38
7.3 K-최근접이웃(KNN)	39
7.4 결정트리: 정보이득과 지니	39
7.5 앙상블: 배깅·랜덤포레스트·부스팅	40
7.6 서포트 벡터 머신과 나이브 베이즈	40
핵심 용어	41
비지도학습과 차원 축소	43
8.1 군집화와 K-평균 알고리즘	43
8.2 군집 수 정하기: 엘보우 방법	43
8.3 계층적 군집	44
8.4 차원의 저주	44
8.5 주성분분석(PCA)	45
8.6 비지도학습의 활용: 이상탐지와 전처리	45
핵심 용어	46
모델 평가와 일반화	47
9.1 편향-분산 트레이드오프	47
9.2 과소적합·과대적합과 교차검증	47
9.3 혼동행렬과 정밀도·재현율·F1	48
9.4 ROC 곡선과 AUC	48

9.5 회귀 평가 지표: MAE·RMSE·R ²	49
9.6 하이퍼파라미터 튜닝과 데이터 누수	49
핵심 용어	50
딥러닝	52
신경망의 기초와 역전파	53
10.1 퍼셉트론에서 다층 신경망으로	53
10.2 활성화 함수	53
10.3 순전파와 출력층	54
10.4 역전파와 연쇄법칙	55
10.5 경사하강법과 학습	55
핵심 용어	57
심층 신경망 학습 기법	58
11.1 기울기 소실과 폭주, 가중치 초기화	58
11.2 배치 정규화와 드롭아웃	58
11.3 옵티마이저: 모멘텀에서 Adam까지	59
11.4 학습률 스케줄링과 규제	59
11.5 데이터 증강	60
핵심 용어	61
합성곱 신경망(CNN)과 컴퓨터 비전	63
12.1 합성곱 연산과 특징맵	63
12.2 스트라이드, 패딩, 출력 크기	63
12.3 풀링과 전형적 CNN 구조	64
12.4 대표 아키텍처의 발전	64
12.5 전이학습과 대표 비전 과제	65
핵심 용어	66
순환 신경망(RNN)과 시퀀스 모델링	67
13.1 시퀀스 데이터와 RNN 구조	67
13.2 시간 전개와 장기 의존성 문제	67
13.3 LSTM과 GRU의 게이트	68
13.4 시퀀스-투-시퀀스와 인코더-디코더	68
13.5 어텐션의 등장과 트랜스포머의 다리	69
핵심 용어	70
현대 인공지능: 트랜스포머와 LLM·RAG	72
트랜스포머와 대규모 언어모델(LLM)	73

14.1 셀프 어텐션의 직관	73
14.2 스케일드 닷프로덕트 어텐션	73
14.3 멀티헤드 어텐션과 위치 인코딩	74
14.4 인코더·디코더 구조와 토큰화·임베딩	75
14.5 사전학습: 다음 토큰 예측과 마스킹	75
14.6 인컨텍스트 학습과 프롬프팅	76
14.7 스케일링 법칙과 대표 모델 계열	76
핵심 용어	77
임베딩과 벡터 검색	79
15.1 표현 학습으로서의 임베딩	79
15.2 단어 임베딩에서 문장 임베딩으로	79
15.3 의미 유사도와 코사인 유사도	80
15.4 벡터 데이터베이스와 색인	81
15.5 근사 최근접 이웃(ANN): HNSW와 IVF	81
15.6 임베딩 모델·청킹이 검색 품질에 미치는 영향	82
핵심 용어	82
검색 증강 생성(RAG)과 생성형 AI의 활용·윤리	84
16.1 RAG의 동기: 최신성과 환각 완화	84
16.2 RAG 파이프라인 개관	84
16.3 청킹 전략	85
16.4 검색 방식: 밀집·희소·하이브리드와 리랭킹	86
16.5 프롬프트 엔지니어링과 환각 완화	86
16.6 RAG의 평가: 검색과 생성	87
16.7 생성형 AI의 한계·편향·윤리와 실무 팁	87
핵심 용어	88

제1부
인공지능의 기초

제1장

인공지능이란 무엇인가

이 장에서는 인공지능이라는 용어가 무엇을 가리키는지, 그리고 그 개념이 왜 이렇게 폭넓게 사용되는지를 살펴본다. 먼저 인공지능의 정의를 여러 관점에서 정리하고, 지능이라는 개념 자체가 얼마나 다층적인지를 짚어 본다. 이어서 1956년 다트머스 회의에서 오늘날의 딥러닝 열풍에 이르기까지의 역사를 개관하고, 인공지능·기계학습·딥러닝이 서로 어떻게 포함 관계를 이루는지 구분한다. 마지막으로 강인공지능과 약인공지능의 차이, 튜링 테스트의 의의, 그리고 인공지능이 실제로 활용되는 대표적 응용 분야를 소개한다.

1.1 인공지능의 정의와 범위

인공지능(Artificial Intelligence, AI)은 인간이 지능적이라고 여기는 과제를 기계가 수행하도록 만드는 학문이자 기술의 총칭이다. 그러나 이 정의는 하나의 고정된 문장으로 못 박기 어렵다. 왜냐하면 지능이라는 말 자체가 시대와 문화에 따라 다르게 이해되어 왔기 때문이다. 어떤 이는 인간처럼 생각하는 기계를 인공지능이라 부르고, 다른 이는 결과적으로 합리적으로 행동하는 기계를 인공지능이라 부른다. 이처럼 정의의 초점이 사고 과정에 있는지 아니면 행동의 결과에 있는지에 따라 인공지능을 바라보는 시각이 달라진다.

고전적으로 인공지능의 정의는 두 개의 축으로 구분된다. 첫째 축은 인간을 기준으로 삼느냐, 아니면 이상적인 합리성을 기준으로 삼느냐이다. 둘째 축은 내부의 사고를 본떠 만드느냐, 아니면 겉으로 드러나는 행동을 본떠 만드느냐이다. 이 두 축을 교차하면 인간처럼 사고하기, 인간처럼 행동하기, 합리적으로 사고하기, 합리적으로 행동하기라는 네 가지 접근이 나온다. 현대 인공지능 교과서는 대체로 합리적으로 행동하는 에이전트를 설계하는 관점을 중심으로 서술하는데, 이는 성능을 객관적으로 측정하고 공학적으로 검증하기가 용이하기 때문이다.

인공지능의 범위는 매우 넓어서 탐색과 최적화, 지식 표현과 추론, 기계학습, 자연어 처리, 컴퓨터 비전, 로봇공학, 계획 수립 등 다양한 하위 분야를 포괄한다. 이들은 서로 독립적으로 발전하기도 하지만, 실제 시스템에서는 여러 기술이 함께 결합되어 동작한다. 예컨대 자율주행 자동차는 카메라 영상을 이해하는 비전 기술, 경로를 찾는 탐색·계획 기술, 그리고 주행 정책을 학습하는 기계학습이 하나로 통합된 결과물이다. 따라서 인공지능을 하나의 알고리즘이 아니라 문제를 지능적으로 푸는 방법들의 생태계로 이해하는 것이 바람직하다.

예시

(‘스팸 메일 필터는 인공지능의 가장 친숙한 사례 가운데 하나이다. 이 시스템은 수많은 메일의 단어 분포와 발신 패턴을 학습하여, 새로 도착한 메일이 스팸일 확률을 계산한다. 사람이 일일이 규칙을 적어 주지 않아도 데이터로부터 판단 기준을 스스로 형성한다는 점에서, 좁은 의미의 지능적 행동을 잘 보여 준다.’)

1.2 지능이란 무엇인가

인공지능을 이해하려면 먼저 지능이 무엇인지 물어야 한다. 심리학에서 지능은 대체로 새로운 상황에 적응하고, 경험으로부터 배우며, 추상적으로 사고하고, 문제를 해결하는 능력으로

설명된다. 그러나 이 능력들은 서로 얽혀 있어 하나의 숫자로 환원하기 어렵다. 인간의 지능조차 언어 지능, 논리·수학 지능, 공간 지능, 사회적 지능 등 여러 측면으로 나누어 볼 수 있다는 다중지능 관점이 제안되기도 하였다. 이러한 다층성은 기계에서 지능을 구현할 때에도 그대로 도전 과제가 된다.

지능을 바라보는 한 가지 유용한 방식은 그것을 목표 달성 능력으로 보는 것이다. 즉 주어진 환경에서 자신의 목표를 얼마나 효과적으로 이루어 내는가로 지능을 가늠하는 것이다. 이 관점은 인공지능 연구에서 특히 매력적인데, 목표와 성능 척도를 명확히 정의하면 시스템의 지능적 수준을 정량적으로 비교할 수 있기 때문이다. 다만 인간의 지능에는 목표 달성 능력만으로 설명되지 않는 창의성, 상식, 정서적 공감 같은 요소도 포함되어 있어, 이러한 부분은 여전히 활발한 연구 대상으로 남아 있다.

지능과 관련하여 흔히 혼동되는 개념으로 계산 능력과 지식이 있다. 빠른 계산이나 방대한 지식의 저장은 지능의 필요조건일 수는 있어도 충분조건은 아니다. 진정한 지능적 행동은 저장된 지식을 적절한 상황에 맞게 꺼내어 활용하고, 불완전한 정보 아래에서도 합리적인 판단을 내리는 데서 드러난다. 이 때문에 인공지능 연구는 단순히 더 많은 데이터를 쌓는 것을 넘어, 지식을 표현하고 추론하며 학습하는 원리를 밝히는 데 집중해 왔다.

1.3 인공지능의 역사

인공지능이라는 용어는 1956년 미국 다트머스 대학에서 열린 여름 워크숍에서 공식적으로 제안되었다. 존 매카시, 마빈 민스키, 클로드 새넌 등이 모인 이 회의는 학문 분야로서의 인공지능이 출발한 상징적 사건으로 여겨진다. 이보다 앞선 1950년에는 앨런 튜링이 기계가 생각할 수 있는가를 물으며 이후 튜링 테스트로 알려진 모방 게임을 제안하였다. 이 시기의 낙관 속에서 정리 증명, 게임 풀이, 간단한 언어 처리 같은 초기 성과가 잇따라 등장하였다.

그러나 1970년대에 이르러 계산 자원의 한계와 문제 규모의 폭발적 증가로 인해 초기의 기대가 꺾이면서, 연구 자금이 크게 줄어드는 이른바 인공지능 겨울이 찾아왔다. 1980년대에는 특정 분야의 전문가 지식을 규칙으로 정리한 전문가 시스템이 상업적으로 주목받으며 다시 부흥기를 맞았으나, 규칙을 일일이 손으로 구축하고 유지하는 비용이 커지면서 또 한 번의 침체를 겪었다. 이러한 부침은 인공지능이 과장된 기대와 냉정한 현실 사이를 오가며 성장해 왔음을 보여 준다.

1990년대 이후에는 통계적 기계학습이 인공지능의 중심으로 떠올랐다. 손으로 규칙을 짜는 대신 데이터로부터 패턴을 학습하는 접근이 자연어 처리와 음성 인식에서 성과를 내기 시작하였다. 2010년을 전후하여 대규모 데이터, 그래픽 처리 장치의 발전, 그리고 심층 신경망 학습 기법이 결합하면서 딥러닝 붐이 일어났다. 2012년 대규모 이미지 인식 대회에서 심층 신경망이 압도적 성능을 보인 사건은 이 흐름의 전환점으로 자주 언급되며, 이후 인공지능은 산업 전반으로 빠르게 확산되었다.

1.4 인공지능·기계학습·딥러닝의 관계

인공지능, 기계학습, 딥러닝은 흔히 뒤섞여 쓰이지만 서로 포함 관계를 이루는 서로 다른 개념이다. 가장 넓은 개념은 인공지능으로, 지능적 행동을 구현하는 모든 방법을 아우른다. 그 안에 데이터로부터 스스로 규칙을 학습하는 기계학습이 자리하고, 다시 그 안에 여러 층으로 이루어진 신경망을 사용하는 딥러닝이 포함된다. 즉 딥러닝은 기계학습의 한 갈래이며,

기계학습은 인공지능을 실현하는 여러 방법 가운데 하나이다.

기계학습은 학습 방식에 따라 크게 지도학습, 비지도학습, 강화학습으로 나뉜다. 지도학습은 입력과 정답이 짝지어진 데이터로부터 입력과 출력의 관계를 배우고, 비지도학습은 정답 없이 데이터에 숨은 구조나 군집을 찾아낸다. 강화학습은 환경과 상호작용하며 받는 보상을 통해 장기적으로 이득이 되는 행동 전략을 학습한다. 이 세 방식은 풀고자 하는 문제의 성격에 따라 선택되며, 실제로는 서로 결합되어 사용되기도 한다.

딥러닝이 주목받은 이유는 사람이 직접 특징을 설계하지 않아도 원자료로부터 유용한 표현을 층층이 자동으로 학습한다는 점에 있다. 전통적 기계학습에서는 이미지의 경계선이나 문장의 단어 빈도 같은 특징을 사람이 손으로 뽑아 주어야 했지만, 심층 신경망은 이러한 특징 추출 과정 자체를 학습에 포함한다. 다만 딥러닝은 대량의 데이터와 계산 자원을 요구하고 내부 동작을 해석하기 어렵다는 한계도 지니므로, 문제 상황에 따라 더 단순한 기계학습 기법이 여전히 적절한 선택이 될 수 있다.

수식

포함 관계: 딥러닝 $\subset$ 기계학습 $\subset$ 인공지능

여기서 기호 $\subset$ 는 '~의 부분집합'을 뜻한다. 즉 모든 딥러닝 기법은 기계학습에 속하고, 모든 기계학습 기법은 인공지능에 속하지만, 그 역은 성립하지 않는다.

1.5 강인공지능과 약인공지능, 그리고 튜링 테스트

인공지능은 그 목표에 따라 약인공지능과 강인공지능으로 구분된다. 약인공지능은 특정한 과제를 잘 수행하도록 만들어진 시스템으로, 오늘날 우리가 사용하는 거의 모든 인공지능이 여기에 해당한다. 번역기, 얼굴 인식, 추천 시스템은 각자의 좁은 영역에서 뛰어나지만 그 영역을 벗어난 일반적 사고는 하지 못한다. 반면 강인공지능은 인간과 같은 수준의 폭넓은 지능, 즉 다양한 문제를 스스로 이해하고 해결하는 범용 지능을 목표로 하며, 아직 실현되지 않은 연구상의 이상에 가깝다.

강인공지능 논의와 관련하여 기계가 실제로 이해하고 의식을 가질 수 있는가라는 철학적 물음이 제기된다. 이와 관련해 존 설이 제안한 중국어 방 논변은, 겉으로는 언어를 이해하는 것처럼 보이는 시스템이 실제로는 기호를 규칙에 따라 조작할 뿐 의미를 이해하지 못할 수 있다고 지적한다. 이러한 논쟁은 아직 합의에 이르지 못했지만, 지능적 행동과 진정한 이해를 구분하려는 문제의식을 분명히 보여 준다.

지능을 판정하는 고전적 기준으로는 1950년 앨런 튜링이 제안한 튜링 테스트가 있다. 이 시험에서 심사자는 보이지 않는 상대와 문자로 대화하며, 상대가 사람인지 기계인지 구별한다. 만약 심사자가 기계를 사람과 신뢰할 만하게 구별하지 못한다면 그 기계는 시험을 통과한 것으로 간주된다. 튜링 테스트는 내부 구조가 아니라 겉으로 드러나는 행동으로 지능을 판정한다는 점에서 행동주의적이며, 오늘날에는 지능의 유일한 척도라기보다 역사적으로 중요한 사고 실험으로 평가된다.

1.6 인공지능의 대표적 응용 분야

인공지능은 이미 우리 생활 곳곳에 스며들어 있다. 자연어 처리 기술은 기계 번역, 음성 비서, 문서 요약, 대화형 챗봇으로 구현되어 사람과 컴퓨터 사이의 언어 장벽을 낮추고 있다. 컴퓨터 비전은 얼굴 인식, 의료 영상 판독, 자율주행의 시각 인지에 활용되어 이미지를 이해하는 능력을 제공한다. 이러한 기술들은 개별적으로도 쓰이지만, 여러 감각 정보를 통합하는 복합 시스템의 구성 요소로 결합될 때 더욱 강력해진다.

산업과 과학의 영역에서도 인공지능의 역할은 빠르게 커지고 있다. 제조업에서는 설비 고장을 미리 예측하고 품질을 자동으로 검사하며, 금융에서는 이상 거래를 탐지하고 위험을 평가한다. 의료와 신약 개발에서는 방대한 데이터에서 유의미한 패턴을 찾아 진단을 보조하고 후보 물질을 탐색한다. 이처럼 인공지능은 사람의 판단을 대체하기보다 보조하고 확장하는 도구로 자리 잡아 가고 있으며, 그 활용 범위는 계속 넓어지고 있다.

예시

추천 시스템은 인공지능이 일상에 스며든 대표적 사례이다. 동영상 플랫폼은 사용자가 시청한 영상과 비슷한 취향을 가진 다른 사용자들의 행동을 함께 분석하여, 다음에 볼 만한 콘텐츠를 예측해 제시한다. 이 과정에서 명시적인 선호 입력이 없어도 클릭과 시청 시간 같은 암묵적 신호로부터 사용자의 관심사를 추정한다.

이 장의 요약

- 인공지능은 지능적 과제를 기계로 수행하려는 학문이자 기술이며, 사고냐 행동이냐, 인간 기준이냐 합리성 기준이냐에 따라 여러 정의가 존재한다.
- 현대 인공지능은 합리적으로 행동하는 에이전트를 설계하는 관점을 중심으로 발전해 왔다.
- 지능은 적응·학습·추상적 사고·문제 해결을 아우르는 다층적 개념으로, 하나의 척도로 환원하기 어렵다.
- 인공지능의 역사는 1956년 다트머스 회의에서 시작해 인공지능 겨울과 전문가 시스템 시기를 거쳐 2010년대 딥러닝 봄으로 이어졌다.
- 인공지능·기계학습·딥러닝은 포함 관계를 이루며, 딥러닝은 기계학습의 한 갈래, 기계학습은 인공지능의 한 방법이다.
- 약인공지능은 특정 과제에 특화된 오늘날의 인공지능이고, 강인공지능은 아직 실현되지 않은 범용 지능을 가리킨다.
- 튜링 테스트는 행동으로 지능을 판정하는 고전적 사고 실험으로, 지능 판별 논의의 출발점이 되었다.
- 인공지능은 자연어 처리, 컴퓨터 비전, 추천, 의료, 금융 등 폭넓은 분야에서 실용적으로 활용되고 있다.

핵심 용어

인공지능(Artificial Intelligence, AI)	인간이 지능적이라고 여기는 과제를 기계가 수행하도록 만드는 학문이자 기술의 총칭이다.
기계학습(Machine Learning, ML)	명시적 규칙을 직접 작성하는 대신 데이터로부터 패턴과 규칙을 학습하는 인공지능의 하위 분야이다.

딥러닝(Deep Learning, DL)	여러 층으로 구성된 심층 신경망을 사용하여 원자료로부터 표현을 자동으로 학습하는 기계학습 기법이다.
지도학습(Supervised Learning)	입력과 정답이 짝지어진 데이터로부터 입력과 출력의 관계를 학습하는 방식이다.
비지도학습(Unsupervised Learning)	정답 없이 데이터에 숨은 구조나 군집을 발견하는 학습 방식이다.
강화학습(Reinforcement Learning)	환경과 상호작용하며 받는 보상을 통해 장기적 이득을 극대화하는 행동 전략을 학습하는 방식이다.
강인공지능(Strong AI)	다양한 문제를 스스로 이해하고 해결하는 인간 수준의 범용 지능을 가리키며 아직 실현되지 않았다.
약인공지능(Weak AI, Narrow AI)	특정한 과제를 잘 수행하도록 설계된 인공지능으로 오늘날의 거의 모든 시스템이 여기에 속한다.
튜링 테스트(Turing Test)	심사자가 문자 대화만으로 상대가 사람인지 기계인지 구별하지 못하면 지능적이라고 보는 판정 기준이다.
다트머스 회의(Dartmouth Workshop)	1956년 인공지능이라는 용어가 공식 제안되어 학문 분야로서 출발한 상징적 회의이다.
인공지능 겨울(AI Winter)	기대에 못 미친 성과와 자금 축소로 인공지능 연구가 침체되었던 시기를 가리키는 표현이다.
전문가 시스템(Expert System)	특정 분야 전문가의 지식을 규칙 형태로 정리하여 추론에 활용하는 규칙 기반 시스템이다.
중국어 방 논변(Chinese Room Argument)	기호를 규칙대로 조작하는 것만으로는 진정한 이해가 성립하지 않을 수 있음을 지적한 존 설의 사고 실험이다.

지능형 에이전트와 합리성

이 장에서는 인공지능 시스템을 설계하고 분석하는 통일된 틀로서 지능형 에이전트 개념을 소개한다. 먼저 에이전트가 환경과 어떻게 상호작용하는지를 감지와 행동의 순환으로 설명하고, 무엇이 좋은 에이전트인가를 판단하는 기준으로 합리성을 정의한다. 이어서 에이전트를 명세하는 표준 틀인 PEAS와, 과제 환경의 성질을 분류하는 여러 축을 살펴본다. 마지막으로 단순 반사 에이전트에서 학습 에이전트에 이르는 다섯 가지 에이전트 구조를 단계적으로 비교하여, 환경의 복잡성에 따라 어떤 설계가 적절한지를 이해한다.

2.1 에이전트와 환경의 상호작용

에이전트란 센서를 통해 환경을 감지하고 작동기를 통해 환경에 행동을 가하는 존재를 말한다. 사람은 눈과 귀 같은 센서와 손발 같은 작동기를 가진 에이전트이고, 소프트웨어 에이전트는 파일이나 네트워크 패킷을 입력으로 받아 화면 출력이나 명령을 통해 행동한다. 인공지능에서 에이전트라는 개념이 중요한 까닭은, 매우 다양한 인공지능 시스템을 하나의 공통된 관점에서 설계하고 분석할 수 있게 해 주기 때문이다.

에이전트의 동작은 감지와 행동의 반복적 순환으로 이해할 수 있다. 매 시점에 에이전트는 환경으로부터 지각을 받아들이고, 지금까지 받은 지각의 이력을 바탕으로 하나의 행동을 선택한다. 이때 지각의 이력 전체를 행동으로 대응시키는 함수를 에이전트 함수라 부르며, 이는 에이전트가 무엇을 해야 하는가를 수학적으로 규정한다. 실제 프로그램은 이 이상적인 함수를 근사적으로 구현한 에이전트 프로그램으로 나타난다.

에이전트 함수와 에이전트 프로그램을 구분하는 것은 개념적으로 중요하다. 에이전트 함수는 가능한 모든 지각 이력에 대한 이상적 대응 관계를 나타내는 추상적 명세이고, 에이전트 프로그램은 제한된 계산 자원 안에서 그 함수를 실제로 계산하는 구체적 알고리즘이다. 인공지능 설계의 목표는 주어진 자원 제약 아래에서 가능한 한 좋은 에이전트 함수를 구현하는 프로그램을 만드는 것이라 할 수 있다.

2.2 합리적 에이전트

좋은 에이전트란 무엇인가라는 질문에 인공지능은 합리성이라는 기준으로 답한다. 합리적 에이전트란 주어진 지각의 이력과 자신이 가진 지식을 바탕으로, 성능 척도의 기댓값을 최대화할 것으로 기대되는 행동을 선택하는 에이전트이다. 여기서 핵심은 결과가 반드시 좋아야 한다는 것이 아니라, 알고 있는 정보에 비추어 최선의 선택을 해야 한다는 데 있다. 정보가 불완전한 상황에서는 최선을 다해도 나쁜 결과가 나올 수 있으므로, 합리성은 결과가 아니라 판단 과정의 정당성으로 정의된다.

합리성은 네 가지 요소에 의존한다. 첫째는 성공을 평가하는 성능 척도이고, 둘째는 에이전트가 환경에 대해 미리 알고 있는 지식이며, 셋째는 에이전트가 수행할 수 있는 행동의 집합이고, 넷째는 지금까지의 지각 이력이다. 이 네 요소가 정해지면 어떤 행동이 합리적인지가 원칙적으로 결정된다. 따라서 합리성은 절대적 개념이 아니라 이들 조건에 상대적인 개념임을 유념해야 한다.

합리성은 전지전능함과 다르다는 점도 중요하다. 전지적 에이전트는 행동의 실제 결과를 미리 알지만, 현실의 에이전트는 그렇지 못하다. 또한 합리적 에이전트는 필요한 경우 정보를 얻기 위한 탐색 행동을 하거나, 환경으로부터 배우며 자신의 지식을 갱신해야 한다. 환경을 미리 완전히 알 수 없을 때 스스로 학습하여 부족한 지식을 보완하는 능력을 자율성이라 하며, 진정으로 합리적인 에이전트는 어느 정도의 자율성을 갖추어야 한다.

예시

진공청소 로봇을 생각해 보자. 이 에이전트의 성능 척도가 일정 시간 동안 청소한 칸의 수라면, 로봇은 먼지가 있는 칸을 감지해 흡입하고 그렇지 않으면 인접한 칸으로 이동하는 행동을 반복함으로써 성능을 높일 수 있다. 같은 로봇이라도 성능 척도에 불필요한 이동에 대한 벌점이 포함된다면, 합리적 행동은 달라진다.

2.3 PEAS를 통한 과제 환경 기술

에이전트를 설계하려면 먼저 그 에이전트가 놓인 과제 환경을 분명히 규정해야 한다. 이를 위해 널리 쓰이는 틀이 PEAS이며, 이는 성능 척도, 환경, 작동기, 센서의 네 요소를 가리킨다. 성능 척도는 에이전트의 성공을 무엇으로 평가할지를 정하고, 환경은 에이전트가 상호작용하는 세계를 규정한다. 작동기는 에이전트가 취할 수 있는 행동의 수단이고, 센서는 에이전트가 환경을 감지하는 수단이다.

PEAS 틀의 가치는 설계에 앞서 문제를 구체적으로 명세하게 만든다는 데 있다. 예컨대 자율주행 택시의 성능 척도로는 안전, 속도, 준법, 승객의 편안함 등을 들 수 있고, 환경으로는 도로, 다른 차량, 보행자, 날씨 등이 있다. 작동기로는 조향, 가속, 제동 장치가, 센서로는 카메라, 레이더, 라이다, 위성 항법 장치 등이 포함된다. 이렇게 요소들을 명확히 나열하면 무엇을 설계하고 무엇을 측정해야 할지가 분명해진다.

PEAS를 신중하게 정의하지 않으면 엉뚱한 목표를 최적화하는 에이전트가 만들어질 수 있다. 성능 척도를 잘못 설정하면 표면적 지표는 좋아 보여도 본래 의도한 바에서 벗어난 행동이 나타날 수 있다. 따라서 성능 척도는 에이전트가 실제로 이루기를 바라는 결과에 최대한 부합하도록, 그리고 조작하기 어렵도록 세심하게 설계해야 한다.

수식

$$PEAS = \{ P: \text{성능 척도(Performance measure)}, \\ E: \text{환경(Environment)}, \\ A: \text{작동기(Actuators)}, \\ S: \text{센서(Sensors)} \}$$

에이전트를 설계하기 전에 이 네 요소를 먼저 규정한다.

2.4 환경의 성질

과제 환경은 여러 성질에 따라 분류되며, 이 성질들이 적절한 에이전트 설계를 좌우한다. 먼저 완전 관측 가능 환경에서는 센서가 매 시점 환경의 모든 관련 상태를 알려 주지만, 부분 관측 가능 환경에서는 정보가 불완전하여 에이전트가 내부에 상태를 추정해 유지해야 한다. 또한 단일

에이전트 환경과 달리 다중 에이전트 환경에서는 다른 에이전트의 행동을 고려해야 하며, 그 관계가 경쟁적인지 협력적인지가 전략에 영향을 준다.

환경은 결정론적인지 확률적인지에 따라서도 나뉜다. 결정론적 환경에서는 현재 상태와 행동이 다음 상태를 완전히 결정하지만, 확률적 환경에서는 결과에 불확실성이 존재한다. 시간의 흐름과 관련해서는, 에이전트가 숙고하는 동안 환경이 변하지 않는 정적 환경과, 계속 변하는 동적 환경이 구분된다. 동적 환경에서는 결정을 미루는 것 자체가 하나의 선택이 되므로 반응 속도가 중요해진다.

그 밖에 에피소드형과 순차형의 구분이 있다. 에피소드형 환경에서는 각 결정이 독립적이어서 이전 행동이 이후에 영향을 주지 않지만, 순차형 환경에서는 현재의 행동이 미래의 상황에 장기적으로 영향을 미친다. 상태와 시간, 지각을 연속적인 값으로 다루는지 이산적인 값으로 다루는지에 따라 연속 환경과 이산 환경도 나뉜다. 일반적으로 부분 관측·확률적·동적·순차적·연속 환경일수록 문제가 어려워지며, 현실 세계의 많은 문제가 바로 이 어려운 쪽에 속한다.

2.5 에이전트의 유형

에이전트는 내부 구조의 복잡성에 따라 여러 유형으로 나뉜다. 가장 단순한 것은 단순 반사 에이전트로, 현재의 지각만을 보고 미리 정해진 조건-행동 규칙에 따라 행동한다. 구현이 간단하지만 현재 지각만으로 충분한 완전 관측 환경에서만 잘 작동하며, 과거 정보가 필요한 상황에서는 한계를 드러낸다. 모델 기반 반사 에이전트는 이 한계를 넘어서기 위해 세계가 어떻게 작동하는지에 대한 내부 모델을 유지하고, 이를 이용해 눈에 보이지 않는 상태를 추정한다.

목표 기반 에이전트는 한 걸음 더 나아가 달성하고자 하는 목표를 명시적으로 가진다. 이 에이전트는 여러 가능한 행동이 미래에 어떤 상태로 이어질지를 고려하여, 목표에 도달하는 행동을 선택한다. 이때 탐색과 계획 수립 기법이 활용된다. 효용 기반 에이전트는 목표의 달성 여부를 넘어서, 여러 상태가 에이전트에게 얼마나 바람직한지를 나타내는 효용 함수를 사용한다. 이를 통해 서로 상충하는 목표들 사이의 균형을 잡거나, 불확실한 상황에서 기대 효용을 최대화하는 선택을 할 수 있다.

마지막으로 학습 에이전트는 경험을 통해 스스로 성능을 개선한다. 학습 에이전트는 행동을 결정하는 수행 요소, 성능을 평가하고 개선점을 알려 주는 비평 요소, 개선을 실행하는 학습 요소, 그리고 새로운 경험을 얻도록 탐색을 유도하는 문제 생성 요소로 구성된다. 이러한 구조 덕분에 학습 에이전트는 미리 알지 못한 환경에서도 시간이 지남에 따라 점점 더 합리적으로 행동할 수 있으며, 앞서 설명한 네 유형 어디에도 학습 능력을 더할 수 있다.

예시

온도 조절기는 단순 반사 에이전트의 전형이다. 현재 온도가 설정값보다 낮으면 난방을 켜고 높으면 끄는 조건-행동 규칙만으로 작동한다. 반면 며칠간의 사용 습관을 학습해 미리 온도를 조절하는 스마트 온도 조절기는 학습 에이전트의 요소를 갖춘 사례로 볼 수 있다.

이 장의 요약

- 에이전트는 센서로 환경을 감지하고 작동기로 행동하는 존재이며, 그 동작은 지각 이력을 행동으로 대응시키는 에이전트 함수로 규정된다.
- 합리적 에이전트는 지각 이력과 지식을 바탕으로 성능 척도의 기댓값을 최대화하는 행동을 선택하며, 합리성은 결과가 아니라 판단 과정의 정당성이다.
- 합리성은 성능 척도, 사전 지식, 가능한 행동, 지각 이력의 네 요소에 상대적으로 정의된다.
- PEAS는 성능 척도·환경·작동기·센서로 과제 환경을 명세하는 표준 틀이다.
- 환경은 관측 가능성, 결정론 여부, 정적·동적, 에피소드·순차, 이산·연속, 단일·다중 에이전트 등의 축으로 분류된다.
- 단순 반사 에이전트는 현재 지각만으로, 모델 기반 에이전트는 내부 모델로 숨은 상태를 추정하여 행동한다.
- 목표 기반 에이전트는 목표 도달을, 효용 기반 에이전트는 효용의 최대화를 추구한다.
- 학습 에이전트는 수행·비평·학습·문제 생성 요소를 통해 경험으로부터 성능을 스스로 개선한다.

핵심 용어

에이전트(Agent)	센서로 환경을 감지하고 작동기로 환경에 행동을 가하는 존재이다.
에이전트 함수(Agent Function)	가능한 모든 지각 이력을 행동으로 대응시키는 추상적 명세이다.
에이전트 프로그램(Agent Program)	제한된 자원 안에서 에이전트 함수를 실제로 계산하는 구체적인 알고리즘이다.
합리적 에이전트(Rational Agent)	지각 이력과 지식을 바탕으로 성능 척도의 기댓값을 최대화하는 행동을 선택하는 에이전트이다.
성능 척도(Performance Measure)	에이전트의 성공을 객관적으로 평가하기 위한 기준이다.
PEAS	성능 척도·환경·작동기·센서로 과제 환경을 규정하는 명세 틀이다.
완전 관측 가능 환경(Fully Observable Environment)	센서가 매 시점 환경의 모든 관련 상태를 알려 주는 환경이다.
부분 관측 가능 환경(Partially Observable Environment)	정보가 불완전하여 에이전트가 내부에 상태를 추정·유지해야 하는 환경이다.
결정론적 환경(Deterministic Environment)	현재 상태와 행동이 다음 상태를 완전히 결정하는 환경이다.
확률적 환경(Stochastic Environment)	행동의 결과에 불확실성이 존재하는 환경이다.
단순 반사 에이전트(Simple Reflex Agent)	현재 지각만을 조건-행동 규칙에 대응시켜 행동하는 가장 단순한 에이전트이다.
모델 기반 에이전트(Model-based Agent)	세계의 작동 방식에 대한 내부 모델로 보이지 않는 상태를 추정하는 에이전트이다.
목표 기반 에이전트(Goal-based Agent)	명시적 목표를 두고 그에 도달하는 행동을 탐색·계획으로 선택하는 에이전트이다.

효용 기반 에이전트(Utility-based Agent)	상태의 바람직함을 나타내는 효용 함수로 기대 효용을 최대화하는 에이전트이다.
학습 에이전트(Learning Agent)	수행·비평·학습·문제 생성 요소를 통해 경험으로 성능을 스스로 개선하는 에이전트이다.

제3장

탐색을 통한 문제 해결

이 장에서는 목표를 이루기 위해 여러 행동의 순서를 찾아내는 탐색 기법을 다룬다. 먼저 현실의 문제를 상태, 행동, 목표, 비용으로 형식화하는 방법을 배우고, 이를 상태 공간 그래프로 표현한다. 이어서 목표에 대한 추가 정보를 사용하지 않는 비정보 탐색으로 너비 우선, 깊이 우선, 균일 비용 탐색을 살펴본다. 그다음 목표까지의 거리를 추정하는 휴리스틱을 활용하는 정보 탐색으로 그리디 탐색과 A* 탐색을 소개하고, 휴리스틱이 갖추어야 할 허용성과 일관성의 개념을 정리한다. 마지막으로 여러 탐색 알고리즘을 완전성·최적성·복잡도의 관점에서 비교한다.

3.1 문제의 형식화

탐색으로 문제를 풀려면 먼저 문제를 정형화된 형태로 기술해야 한다. 하나의 탐색 문제는 다섯 가지 구성 요소로 정의된다. 초기 상태는 에이전트가 출발하는 상황이고, 행동은 각 상태에서 취할 수 있는 선택지의 집합이다. 전이 모델은 어떤 상태에서 어떤 행동을 했을 때 도달하는 다음 상태를 규정하고, 목표 검사는 주어진 상태가 목표인지 판정한다. 마지막으로 경로 비용은 상태들을 잇는 경로에 드는 비용을 수치로 부여한다.

이렇게 정의된 문제는 상태 공간이라는 개념으로 표현된다. 상태 공간은 초기 상태로부터 가능한 행동을 반복해 도달할 수 있는 모든 상태와 그들 사이의 연결로 이루어진 그래프이다. 이 그래프에서 각 노드는 하나의 상태를, 각 간선은 하나의 행동과 그 비용을 나타낸다. 따라서 문제를 푼다는 것은 초기 상태 노드에서 목표 상태 노드에 이르는 경로를 찾는 일이 된다.

탐색 알고리즘은 이 그래프를 체계적으로 펼쳐 가며 해를 찾는다. 이때 아직 확장하지 않은 노드들을 담아 두는 자료 구조를 프론티어 또는 개방 목록이라 하고, 이미 확장한 노드를 기록하는 자료 구조를 탐색된 집합 또는 폐쇄 목록이라 한다. 노드를 확장한다는 것은 그 노드에서 가능한 행동을 적용하여 자식 노드들을 생성하는 것을 뜻한다. 어떤 노드를 다음에 확장할지를 정하는 규칙이 곧 탐색 전략이며, 이 전략의 선택이 알고리즘의 성능을 좌우한다.

예시

8-퍼즐은 문제 형식화의 고전적 예이다. 상태는 3×3 격자 위에 놓인 여덟 개 타일과 빈 칸의 배치이고, 행동은 빈 칸을 상하좌우로 미는 것이다. 초기 상태는 뒤섞인 배치, 목표는 정렬된 배치이며, 각 이동의 비용을 1로 두면 최소 이동 횟수를 찾는 문제가 된다.

3.2 비정보 탐색

비정보 탐색은 목표가 어느 방향에 있는지에 대한 추가 정보 없이, 오직 문제 정의만을 이용해 상태 공간을 훑는 방법이다. 대표적인 것이 너비 우선 탐색으로, 이는 초기 상태에서 가까운 노드부터 층별로 확장한다. 프론티어를 먼저 들어온 것을 먼저 꺼내는 큐로 관리하며, 모든 간선의 비용이 같을 때 최소 단계 수의 해를 반드시 찾아 준다. 다만 얕은 깊이에서도 노드 수가 기하급수적으로 늘어나 많은 메모리를 요구하는 단점이 있다.

깊이 우선 탐색은 반대로 한 방향으로 가능한 한 깊이 내려간 뒤, 막다른 곳에 이르면 되돌아와 다른 경로를 시도한다. 프론티어를 나중에 들어온 것을 먼저 꺼내는 스택으로 관리하며, 저장해야 할 노드가 현재 경로의 깊이에 비례하므로 메모리 사용이 적다. 그러나 깊이가 무한하거나 순환이 있는 공간에서는 목표를 놓치거나 무한히 내려갈 수 있고, 찾은 해가 최단 경로라는 보장도 없다. 이 두 단점을 절충한 기법으로 깊이 제한을 점차 늘려 가는 반복 심화 탐색이 있다.

간선마다 비용이 다를 때에는 균일 비용 탐색이 사용된다. 이 방법은 초기 상태로부터의 누적 경로 비용이 가장 작은 노드를 우선하여 확장하며, 프론티어를 비용을 기준으로 하는 우선순위 큐로 관리한다. 균일 비용 탐색은 비용이 음수가 아닌 한 항상 최소 비용의 해를 찾아내는 최적성을 가진다. 모든 간선 비용이 동일한 특수한 경우에는 너비 우선 탐색과 사실상 같아지므로, 균일 비용 탐색은 너비 우선 탐색을 비용 있는 경우로 일반화한 것으로 볼 수 있다.

3.3 정보 탐색과 휴리스틱

정보 탐색은 목표가 어느 쪽에 있을지를 가늠하는 추가 정보를 활용하여 탐색을 목표 방향으로 유도한다. 이 추가 정보는 휴리스틱 함수로 주어지는데, 휴리스틱 함수 $h(n)$ 은 노드 n 에서 목표까지 남은 비용을 어림잡아 추정한 값이다. 좋은 휴리스틱은 불필요한 방향으로의 탐색을 줄여 훨씬 적은 노드만 확장하고도 해를 찾게 해 준다. 휴리스틱은 문제의 구조에 대한 통찰을 숫자로 담아 내는 장치라 할 수 있다.

가장 단순한 정보 탐색은 그리디 최선 우선 탐색으로, 오직 휴리스틱 값 $h(n)$ 이 가장 작은 노드, 즉 목표에 가장 가까워 보이는 노드를 우선 확장한다. 이 방법은 종종 빠르게 목표에 접근하지만, 지금까지 들인 비용을 무시하기 때문에 최적의 해를 놓칠 수 있고 때로는 잘못된 방향에 갇히기도 한다. 즉 그리디 탐색은 눈앞의 유망함만을 좇는 근시안적 전략이다.

이 약점을 보완하는 것이 A^* 탐색이다. A^* 는 시작점에서 노드 n 까지 실제로 들인 비용 $g(n)$ 과 n 에서 목표까지의 추정 비용 $h(n)$ 을 더한 값 $f(n)$ 이 가장 작은 노드를 확장한다. 이렇게 하면 이미 들인 비용과 앞으로 들 비용을 함께 고려하므로, 그리디 탐색의 근시안과 균일 비용 탐색의 비효율을 동시에 극복한다. A^* 는 적절한 조건을 갖춘 휴리스틱과 함께 사용될 때 최적의 해를 보장하며, 실제로 경로 찾기와 계획 수립에서 가장 널리 쓰이는 탐색 알고리즘이다.

수식

A^* 의 평가 함수: $f(n) = g(n) + h(n)$

· $g(n)$: 시작 상태에서 노드 n 까지 실제로 든 경로 비용

· $h(n)$: 노드 n 에서 목표까지의 추정 비용(휴리스틱)

· $f(n)$: n 을 지나는 해의 총비용에 대한 추정치

A^* 는 프론티어에서 $f(n)$ 이 가장 작은 노드를 먼저 확장한다.

의사코드

```
function A_STAR(problem, h):
    start = problem.initial_state
    frontier = PriorityQueue()      # f(n) 기준 최소 우선순위 큐
    frontier.push(start, priority = h(start))
    g = { start: 0 }                # 시작점에서의 실제 비용
    came_from = { start: None }     # 경로 복원용
    explored = set()

    while not frontier.is_empty():
        n = frontier.pop()          # f 값이 가장 작은 노드
        if problem.is_goal(n):
            return reconstruct_path(came_from, n)
        explored.add(n)
        for action in problem.actions(n):
            m = problem.result(n, action)
            tentative_g = g[n] + problem.step_cost(n, action, m)
            if m not in g or tentative_g < g[m]:
                g[m] = tentative_g
                came_from[m] = n
                f = tentative_g + h(m)
                frontier.push(m, priority = f)
    return failure                  # 해가 존재하지 않음
```

3.4 허용성과 일관성

A* 탐색이 최적의 해를 보장하려면 휴리스틱 함수가 특정 성질을 만족해야 한다. 첫째 성질은 허용성으로, 휴리스틱이 실제 목표까지의 비용을 결코 과대평가하지 않는 성질이다. 즉 임의의 노드 n 에서 $h(n)$ 은 n 에서 목표까지의 실제 최소 비용을 넘지 않아야 한다. 허용적 휴리스틱은 낙관적이어서 남은 비용을 실제보다 작거나 같게 어림잡으며, 이 성질이 있으면 트리 탐색에서 A*의 최적성이 보장된다.

둘째 성질은 일관성 또는 단조성으로, 허용성보다 강한 조건이다. 일관성은 임의의 노드 n 과 그 이웃 노드 n' 에 대해, $h(n)$ 이 n 에서 n' 로 가는 비용에 $h(n')$ 을 더한 값보다 크지 않다는 삼각 부등식 형태로 표현된다. 일관성이 성립하면 목표로 다가갈수록 f 값이 감소하지 않고 일정하게 유지되거나 증가하므로, 한 번 확장한 노드를 다시 확장할 필요가 없어진다. 이 덕분에 그래프 탐색에서도 A*가 효율적으로 최적해를 찾는다.

일관적인 휴리스틱은 반드시 허용적이지만, 허용적인 휴리스틱이 언제나 일관적인 것은 아니다. 실무에서 사용하는 대부분의 자연스러운 휴리스틱은 일관성을 함께 만족한다. 예를 들어 격자 위에서 장애물을 무시하고 직선 거리나 격자 이동 거리를 계산한 값은 실제 경로 비용을 넘지 않으면서 삼각 부등식도 만족하는 대표적 휴리스틱이다. 좋은 휴리스틱을 설계하는 한 가지 방법은 원래 문제의 제약을 일부 완화한 이완 문제의 최적 비용을 사용하는 것이며, 이렇게 얻은 값은 자동으로 허용성을 갖는다.

수식

허용성: 모든 노드 n 에 대해 $0 \leq h(n) \leq h^*(n)$

($h^*(n)$ 은 n 에서 목표까지의 실제 최소 비용)

일관성: 이웃한 n, n' 에 대해 $h(n) \leq c(n, n') + h(n')$

($c(n, n')$ 은 n 에서 n' 로 가는 간선 비용)

3.5 탐색 알고리즘의 비교

탐색 알고리즘은 네 가지 기준으로 평가된다. 완전성은 해가 존재할 때 반드시 그것을 찾아 내는가를, 최적성은 찾은 해가 항상 최소 비용인가를 묻는다. 시간 복잡도는 해를 찾기까지 확장하는 노드의 수를, 공간 복잡도는 동시에 저장해야 하는 노드의 수를 나타낸다. 이 네 기준을 함께 보면 각 알고리즘의 장단점이 뚜렷이 드러난다.

이 기준에 비추어 보면 너비 우선 탐색은 완전하고 간선 비용이 같을 때 최적이지만 메모리 소모가 크고, 깊이 우선 탐색은 메모리는 적게 쓰지만 완전성과 최적성을 일반적으로 보장하지 못한다. 균일 비용 탐색은 완전하고 최적이지만 비용이 작은 노드를 광범위하게 확장해 느릴 수 있다. A^* 는 허용적 휴리스틱과 함께 완전하고 최적이면서도, 좋은 휴리스틱을 쓰면 확장 노드 수를 크게 줄여 실용적 효율을 낸다. 반면 그리디 탐색은 대개 빠르지만 최적성을 보장하지 못한다.

결국 어떤 알고리즘을 선택할지는 문제의 성질과 가용 자원에 달려 있다. 상태 공간이 작고 비용이 균일하다면 단순한 비정보 탐색으로 충분할 수 있고, 공간이 방대하여 무작정 펼치기 어렵다면 좋은 휴리스틱을 갖춘 A^* 가 적합하다. 메모리가 극도로 제한된 환경에서는 반복 심화 탐색이나 그 정보 탐색 변형이 대안이 된다. 탐색 알고리즘의 비교는 곧 완전성과 최적성이라는 이상과 시간·공간이라는 현실적 제약 사이의 균형을 찾는 과정이라 할 수 있다.

이 장의 요약

- 탐색 문제는 초기 상태, 행동, 전이 모델, 목표 검사, 경로 비용의 다섯 요소로 형식화되며 상태 공간 그래프로 표현된다.
- 비정보 탐색은 문제 정의만 사용하며, 너비 우선·깊이 우선·균일 비용 탐색이 대표적이다.
- 너비 우선 탐색은 최소 단계 해를 보장하나 메모리 소모가 크고, 깊이 우선 탐색은 메모리는 적지만 완전성·최적성이 보장되지 않는다.
- 균일 비용 탐색은 누적 비용이 최소인 노드를 확장하여 음이 아닌 비용에서 최적해를 찾는다.
- 정보 탐색은 휴리스틱 $h(n)$ 으로 탐색을 목표 방향으로 유도하며, 그리디 탐색은 $h(n)$ 만, A^* 는 $f(n)=g(n)+h(n)$ 을 사용한다.
- 허용성은 휴리스틱이 실제 비용을 과대평가하지 않는 성질이고, 일관성은 그보다 강한 삼각 부등식 조건이다.
- 허용적 휴리스틱과 함께 A^* 는 완전하고 최적이며, 좋은 휴리스틱은 확장 노드 수를 크게 줄인다.
- 탐색 알고리즘은 완전성·최적성·시간 복잡도·공간 복잡도의 네 기준으로 비교·선택된다.

핵심 용어

상태 공간(State Space)	초기 상태에서 도달 가능한 모든 상태와 그 연결로 이루어진 그래프이다.
초기 상태(Initial State)	에이전트가 탐색을 시작하는 출발 상황이다.
전이 모델(Transition Model)	어떤 상태에서 특정 행동을 했을 때 도달하는 다음 상태를 규정하는 함수이다.
경로 비용(Path Cost)	상태들을 잇는 경로 전체에 드는 비용을 수치로 나타낸 값이다.
프론티어(Frontier)	아직 확장하지 않은 노드들을 담아 두는 자료 구조로 개방 목록이라고도 한다.
너비 우선 탐색(Breadth-First Search, BFS)	초기 상태에서 가까운 노드부터 층별로 확장하는 비정보 탐색이다.
깊이 우선 탐색(Depth-First Search, DFS)	한 방향으로 깊이 내려간 뒤 막다른 곳에서 되돌아오는 비정보 탐색이다.
균일 비용 탐색(Uniform-Cost Search)	누적 경로 비용이 최소인 노드를 우선 확장하여 최적해를 찾는 탐색이다.
휴리스틱 함수(Heuristic Function, $h(n)$)	노드에서 목표까지 남은 비용을 어림잡아 추정하는 함수이다.
그리디 최선 우선 탐색(Greedy Best-First Search)	휴리스틱 값이 가장 작은 노드를 우선 확장하는 근시안적 정보 탐색이다.
A* 탐색(A* Search)	$f(n)=g(n)+h(n)$ 이 최소인 노드를 확장하여 최적해를 찾는 정보 탐색이다.
허용성(Admissibility)	휴리스틱이 실제 목표 비용을 결코 과대평가하지 않는 성질이다.
일관성(Consistency)	$h(n)$ 이 간선 비용과 이웃의 휴리스틱 합을 넘지 않는, 허용성보다 강한 삼각 부등식 조건이다.
이완 문제(Relaxed Problem)	원 문제의 제약을 일부 완화한 문제로, 그 최적 비용은 허용적 휴리스틱이 된다.

제4장

지식 표현과 추론

이 장에서는 에이전트가 세계에 대한 지식을 명시적으로 저장하고 그로부터 새로운 사실을 이끌어 내는 방법을 다룬다. 먼저 지식을 지식 베이스에 담아 추론하는 지식 기반 에이전트의 구조를 살펴보고, 가장 단순한 논리 체계인 명제 논리를 소개한다. 이어서 명제 논리의 표현력을 넘어서는 술어 논리의 개요를 설명하고, 규칙의 형태로 지식을 다루는 규칙 기반 시스템의 전향 추론과 후향 추론을 비교한다. 끝으로 개념과 관계를 체계적으로 정리하는 온톨로지를 소개하고, 확실한 논리만으로는 다루기 어려운 불확실성을 왜 확률로 다루어야 하는지를 논의한다.

4.1 지식 기반 에이전트

지식 기반 에이전트는 세계에 대한 사실들의 집합인 지식 베이스를 중심으로 동작한다. 지식 베이스는 에이전트가 참이라고 여기는 문장들로 이루어지며, 에이전트는 새로 얻은 지각을 문장으로 바꾸어 지식 베이스에 추가하고, 저장된 지식으로부터 무엇을 해야 할지를 추론한다. 이처럼 지식을 코드 속에 암묵적으로 숨기지 않고 별도의 선언적 형태로 분리해 두는 방식을 선언적 접근이라 하며, 이는 지식을 수정하고 확장하기 쉽게 만든다.

지식 베이스를 다루는 두 가지 기본 연산은 텔과 애스크이다. 텔은 새로운 문장을 지식 베이스에 알려 주어 저장하는 연산이고, 애스크는 지식 베이스에 질의하여 무엇이 도출되는지를 묻는 연산이다. 에이전트는 매 시점 지각을 텔로 추가한 뒤, 어떤 행동을 해야 하는지를 애스크로 물어 그 답에 따라 행동한다. 이 두 연산의 배후에는 저장된 문장들로부터 새로운 문장을 논리적으로 이끌어 내는 추론 과정이 자리한다.

추론에서 핵심이 되는 개념은 함의이다. 어떤 문장들의 집합이 참일 때 다른 문장이 반드시 참이 되면, 앞의 집합이 뒤의 문장을 함의한다고 말한다. 추론 절차가 지식 베이스가 함의하는 문장만을 도출하면 그 절차는 건전하다고 하고, 함의되는 모든 문장을 도출할 수 있으면 완전하다고 한다. 건전하고 완전한 추론은 지식 기반 에이전트가 신뢰할 수 있는 결론에 이르도록 보장하는 이론적 토대이다.

4.2 명제 논리

명제 논리는 가장 단순하면서도 기초가 되는 논리 체계이다. 명제란 참 또는 거짓 가운데 하나의 값을 가지는 선언적 문장을 말하며, 명제 기호로 표현된다. 예컨대 비가 온다를 하나의 기호로 두면, 그 기호는 상황에 따라 참이거나 거짓이 된다. 명제 논리는 이러한 원자 명제들을 논리 연결사로 결합하여 더 복잡한 문장을 구성한다.

명제 논리의 주요 연결사에는 부정, 논리곱, 논리합, 함축, 상호 함축이 있다. 부정은 명제의 참·거짓을 뒤집고, 논리곱은 두 명제가 모두 참일 때만 참이며, 논리합은 적어도 하나가 참이면 참이 된다. 함축은 앞의 명제가 참인데 뒤의 명제가 거짓인 경우에만 거짓이 되고, 상호 함축은 두 명제의 참·거짓이 일치할 때 참이 된다. 이러한 연결사의 의미는 진리표를 통해 명확하게 규정된다.

명제 논리에서 추론은 진리표를 이용해 모든 경우를 따지거나, 정해진 추론 규칙을 반복 적용하여 수행할 수 있다. 널리 쓰이는 추론 규칙으로는, 함축과 그 전제가 참이면 결론을 이끌어 내는 전건 긍정 규칙이 있다. 그러나 명제 논리는 표현력에 한계가 있다. 개별 명제가 통째로 하나의 기호이므로, 대상들 사이의 관계나 모든 대상에 대한 일반적 진술을 간결하게 나타내지 못한다. 이 한계가 더 강력한 논리 체계를 필요로 하는 이유가 된다.

수식

주요 연결사:

$\neg P$ 부정(NOT) P 가 거짓일 때 참

$P \wedge Q$ 논리곱(AND) 둘 다 참일 때만 참

$P \vee Q$ 논리합(OR) 적어도 하나가 참이면 참

$P \rightarrow Q$ 함축(IMPLIES) P 가 참이고 Q 가 거짓일 때만 거짓

$P \leftrightarrow Q$ 상호 함축 P 와 Q 의 진리값이 같을 때 참

전건 긍정(Modus Ponens): P 와 $(P \rightarrow Q)$ 로부터 Q 를 도출한다.

4.3 술어 논리 개요

술어 논리는 1차 논리라고도 불리며, 명제 논리의 표현력을 크게 확장한다. 술어 논리는 세계를 대상, 관계, 함수로 나누어 바라본다. 대상은 사람이나 숫자처럼 이야기의 소재가 되는 개체이고, 술어는 대상들이 지닌 성질이나 대상들 사이의 관계를 나타내며, 함수는 대상을 다른 대상으로 대응시킨다. 이러한 구조 덕분에 명제 논리에서는 통째로 다루어야 했던 문장의 내부 구조를 세밀하게 표현할 수 있다.

술어 논리가 특히 강력한 것은 한정사를 제공하기 때문이다. 전칭 한정사는 어떤 성질이 모든 대상에 대해 성립함을 나타내고, 존재 한정사는 그 성질을 만족하는 대상이 적어도 하나 존재함을 나타낸다. 예를 들어 모든 사람은 죽는다와 같은 일반적 진술은 전칭 한정사로 간결히 표현되며, 이는 명제 논리로는 각 사람마다 별개의 명제를 나열해야 했던 것을 하나의 문장으로 압축한다. 한정사는 술어 논리가 인간의 자연스러운 추론을 담아 내는 핵심 장치이다.

술어 논리는 수학, 자연어의 의미 표현, 지식 베이스 구축 등 다양한 분야에서 지식을 형식화하는 표준 도구로 쓰인다. 표현력이 커진 대신 추론의 계산 비용도 늘어나, 일반적인 술어 논리의 함의 판정은 명제 논리보다 훨씬 어렵다. 그럼에도 술어 논리는 대상과 관계를 자연스럽게 다룰 수 있어, 지식 표현의 이론적 기반이자 이후 규칙 기반 시스템과 온톨로지의 밑바탕이 된다.

예시

가족 관계를 술어 논리로 나타내 보자. 부모(철수, 영희)라는 술어는 철수가 영희의 부모임을 뜻한다. 여기에 모든 x, y 에 대해 부모(x, y)이면 조상(x, y)이라는 규칙과, 조상 관계가 전이적이라는 규칙을 더하면, 개별 사실만으로는 드러나지 않던 할아버지·후손 같은 관계를 추론으로 이끌어 낼 수 있다.

4.4 규칙 기반 시스템과 추론 방향

규칙 기반 시스템은 만약 조건이면 결론이라는 형태의 규칙들로 지식을 표현한다. 이러한 규칙의 모음과 현재 알려진 사실들의 집합, 그리고 규칙을 적용하는 추론 엔진이 결합하여 하나의 시스템을 이룬다. 규칙 기반 표현은 사람이 이해하고 검토하기 쉬워서, 의료 진단이나 고장 진단 같은 전문가 시스템에서 오랫동안 활용되어 왔다. 규칙의 조건이 사실과 맞아떨어지면 그 규칙이 발화되어 새로운 사실이 더해진다.

규칙을 적용하는 방향에는 크게 두 가지가 있다. 전향 추론은 알려진 사실에서 출발하여 조건이 충족되는 규칙을 차례로 발화하고, 그 결론을 새로운 사실로 추가하는 과정을 목표가 나올 때까지 반복한다. 이 방식은 자료가 주어졌을 때 그로부터 도출되는 모든 결론을 이끌어내는 데이터 주도적 방식으로, 상황을 감시하거나 가능한 결론을 폭넓게 뽑아낼 때 유용하다.

후향 추론은 반대로 증명하고자 하는 목표에서 출발한다. 목표를 결론으로 갖는 규칙을 찾아 그 규칙의 조건들을 다시 증명해야 할 하위 목표로 삼고, 이 과정을 이미 알려진 사실에 도달할 때까지 되풀이한다. 이 방식은 특정 질문에 답을 구할 때 관련된 규칙만 선택적으로 살피므로 목표 주도적이며 효율적이다. 전향 추론과 후향 추론은 각각 데이터가 풍부한 상황과 목표가 분명한 상황에 어울리며, 실제 시스템은 문제의 성격에 따라 둘을 선택하거나 결합해 사용한다.

의사코드

```
// 전향 추론(Forward Chaining) 개요
facts = 알려진 사실들의 집합
rules = 규칙들의 집합           // 각 규칙: 조건들 -> 결론
repeat:
    new = 비어 있음
    for each rule (조건들 -> 결론) in rules:
        if 규칙의 모든 조건이 facts 에 있고 결론이 아직 facts 에 없으면:
            new 에 결론을 추가
    if new 가 비어 있으면:
        return facts           // 더 이상 도출할 것이 없음
    facts = facts ∪ new
    if 목표가 facts 에 있으면:
        return 목표 증명됨
```

4.5 온톨로지와 개념의 조직화

지식이 방대해지면 개별 사실을 나열하는 것만으로는 다루기 어려워지므로, 개념들을 체계적으로 조직하는 틀이 필요하다. 온톨로지는 특정 영역에 존재하는 개념들과 그들 사이의 관계를 명시적으로 정의한 지식의 뼈대이다. 온톨로지는 개념을 상위와 하위로 잇는 분류 체계, 개념이 가지는 속성, 그리고 개념 간의 다양한 관계를 규정함으로써 지식을 구조화한다. 이를 통해 서로 다른 시스템이나 사람이 같은 용어를 같은 의미로 사용하도록 돕는다.

온톨로지의 대표적 구성 요소는 분류 계층이다. 예를 들어 개는 포유류의 하위 개념이고 포유류는 동물의 하위 개념이라는 계층을 두면, 개가 동물이라는 사실을 일일이 적지 않아도 상속을 통해 자동으로 이끌어 낼 수 있다. 이러한 상속은 지식을 간결하게 유지하면서도 풍부한 추론을 가능하게 한다. 온톨로지는 이처럼 개념 사이의 일반화와 특수화 관계를 활용하여 지식의 중복을 줄이고 일관성을 높인다.

온톨로지는 시맨틱 웹, 의료 정보, 전자 상거래의 상품 분류 등 여러 분야에서 지식을 공유하고 통합하는 기반으로 쓰인다. 잘 설계된 온톨로지는 서로 다른 출처의 데이터를 하나의 일관된 개념 틀 위에서 연결해 주어, 기계가 데이터의 의미를 해석하고 추론하도록 돕는다. 다만 현실의 개념들은 경계가 모호하고 예외가 많아, 완전하고 정확한 온톨로지를 구축하는 일은 상당한 노력을 요구하는 지속적 과제로 남는다.

4.6 불확실성 하의 추론 도입

지금까지 살펴본 논리는 명제가 확실히 참이거나 거짓이라는 가정 위에서 있다. 그러나 현실의 많은 지식은 이러한 확실성을 갖지 못한다. 센서는 잡음을 포함하고, 규칙에는 예외가 있으며, 미래의 사건은 예측 불가능하다. 예를 들어 새는 난다는 규칙은 대체로 옳지만 펭귄이나 타조 같은 예외가 있어, 이를 확실한 논리 규칙으로 두면 잘못된 결론에 이를 수 있다. 이처럼 일반적으로 성립하지만 예외를 가지는 지식은 참·거짓의 이분법만으로 다루기 어렵다.

확실한 논리로 불확실성을 다루려는 시도는 여러 난관에 부딪힌다. 모든 예외를 규칙에 일일이 적어 넣으려 하면 규칙이 감당할 수 없이 복잡해지고, 그럼에도 미처 고려하지 못한 예외는 언제나 남는다. 또한 여러 불확실한 증거가 서로 얼마나 강하게 결론을 뒷받침하는지를 논리의 참·거짓만으로는 정량적으로 견주기 어렵다. 결국 불확실성 아래에서 합리적으로 판단하려면 믿음의 정도를 수치로 나타내는 도구가 필요하다.

이 도구가 바로 확률이다. 확률은 명제가 참일 것이라는 믿음의 정도를 0과 1 사이의 수로 나타내어, 불확실한 증거들을 일관된 방식으로 결합하고 갱신하게 해 준다. 새로운 증거가 관측되면 그에 따라 믿음을 갱신하는 원리를 제공하는 것이 베이즈 정리이며, 이는 확률적 추론의 핵심을 이룬다. 이렇게 확률에 기반한 추론은 불확실성을 회피하지 않고 정면으로 다루며, 현대 인공지능에서 기계학습과 자연스럽게 결합하여 폭넓게 활용된다. 확률적 추론의 구체적인 방법은 이후 관련 장에서 더 깊이 다루게 된다.

이 장의 요약

- 지식 기반 에이전트는 참이라고 여기는 문장들의 집합인 지식 베이스를 중심으로 텔과 에스클 연산을 통해 동작한다.
- 함의는 추론의 핵심 개념이며, 건전한 추론은 함의되는 것만을, 완전한 추론은 함의되는 모든 것을 도출한다.
- 명제 논리는 참·거짓 값을 갖는 명제를 부정·논리곱·논리합·함축 등으로 결합하지만, 대상과 관계를 표현하는 데 한계가 있다.
- 술어 논리는 대상·관계·함수와 전칭·존재 한정사를 도입하여 일반적 진술을 간결하게 표현한다.
- 규칙 기반 시스템은 조건-결론 규칙으로 지식을 표현하며, 전향 추론은 데이터 주도적, 후향 추론은 목표 주도적이다.
- 온톨로지는 영역의 개념과 관계를 분류 계층과 상속으로 조직하여 지식을 구조화하고 공유·통합을 돕는다.
- 센서 잡음, 규칙의 예외, 예측 불가능성 때문에 현실의 지식은 확실한 논리만으로 다루기 어렵다.
- 확률은 믿음의 정도를 수치로 나타내어 불확실한 증거를 결합·갱신하게 하며, 베이즈 정리가 그 핵심 원리이다.

핵심 용어

지식 베이스(Knowledge Base, KB)	에이전트가 참이라고 여기는 문장들의 집합으로 추론의 바탕이 된다.
텔과 애스크(Tell/Ask)	지식 베이스에 문장을 추가하는 연산과 도출되는 결론을 질의하는 연산이다.
함의(Entailment)	어떤 문장 집합이 참일 때 다른 문장이 반드시 참이 되는 논리적 관계이다.
건전성(Soundness)	추론 절차가 함의되는 문장만을 도출하는 성질이다.
완전성(Completeness)	추론 절차가 함의되는 모든 문장을 도출할 수 있는 성질이다.
명제 논리(Propositional Logic)	참·거짓 값을 갖는 명제를 논리 연결사로 결합하는 가장 단순한 논리 체계이다.
논리 연결사(Logical Connective)	부정·논리곱·논리합·함축·상호 함축 등 명제를 결합하는 기호이다.
전건 긍정(Modus Ponens)	P와 $P \rightarrow Q$ 가 참이면 Q를 도출하는 기본 추론 규칙이다.
술어 논리(1차 논리, First-Order Logic)	대상·관계·함수와 한정사를 도입해 명제 논리의 표현력을 확장한 논리이다.
한정사(Quantifier)	모든 대상에 대한 전칭 한정사와 대상의 존재를 나타내는 존재 한정사를 통칭한다.
규칙 기반 시스템(Rule-based System)	조건-결론 형태의 규칙과 사실, 추론 엔진으로 구성된 지식 시스템이다.
전향 추론(Forward Chaining)	알려진 사실에서 출발해 규칙을 발화하며 결론을 넓혀 가는 데이터 주도적 추론이다.
후향 추론(Backward Chaining)	목표에서 출발해 하위 목표로 거슬러 올라가며 증명하는 목표 주도적 추론이다.
온톨로지(Ontology)	영역의 개념과 그 관계를 분류 계층과 상속으로 명시적으로 정의한 지식의 뼈대이다.
베이즈 정리(Bayes' Theorem)	새로운 증거에 따라 명제에 대한 믿음의 정도를 갱신하는 확률적 추론의 핵심 원리이다.

제2부
머신러닝의 기초

제5장

머신러닝 개관과 학습의 원리

머신러닝(machine learning)은 사람이 규칙을 일일이 프로그래밍하는 대신, 데이터로부터 규칙을 스스로 찾아내도록 컴퓨터를 학습시키는 방법이다. 전통적인 프로그래밍이 '규칙과 입력을 넣으면 답이 나오는' 구조라면, 머신러닝은 '입력과 답의 예시를 많이 넣어 규칙을 얻어내는' 구조라고 이해할 수 있다. 이 장에서는 머신러닝의 큰 갈래인 지도학습·비지도학습·강화학습을 구분하고, 데이터·특징·레이블·모델·학습이라는 기본 개념을 정의한 뒤, 학습이 결국 손실을 최소화하는 과정이며 진짜 목표는 일반화(generalization)임을 살펴본다.

5.1 머신러닝이란 무엇인가

머신러닝은 명시적으로 프로그래밍하지 않아도 컴퓨터가 데이터에서 패턴을 학습하여 예측이나 판단을 수행하도록 하는 인공지능의 한 분야이다. 예를 들어 스팸 메일을 걸러내는 규칙을 사람이 직접 작성하려면 수많은 예외를 고려해야 하지만, 머신러닝은 스팸과 정상 메일의 예시를 대량으로 학습하여 분류 규칙을 자동으로 만들어 낸다. 이렇게 얻은 규칙은 사람이 미처 생각하지 못한 미묘한 패턴까지 반영할 수 있다는 점에서 강력하다.

전통적 프로그래밍과 머신러닝의 차이는 무엇을 사람이 정하고 무엇을 기계가 정하는가에 있다. 전통적 방식에서는 규칙(프로그램)을 사람이 작성하고 컴퓨터는 그것을 실행할 뿐이다. 반면 머신러닝에서는 사람이 데이터와 학습 절차를 제공하면 컴퓨터가 데이터에 맞는 규칙, 즉 모델을 찾아낸다. 따라서 문제의 규칙이 복잡하거나 사람이 명확히 서술하기 어려울수록 머신러닝의 이점이 커진다.

머신러닝이 만능은 아니다. 좋은 성능을 얻으려면 문제에 적합한 양질의 데이터가 충분해야 하고, 데이터에 담긴 편향이나 오류가 그대로 모델에 전이될 수 있음을 주의해야 한다. 또한 학습된 모델은 학습에 사용한 데이터의 분포를 벗어난 상황에서는 신뢰하기 어렵다. 그러므로 머신러닝을 활용할 때는 데이터의 품질, 문제의 성격, 결과의 검증 방법을 함께 고려하는 태도가 중요하다.

예시

손글씨 숫자 인식을 생각해 보자. '숫자 8은 이렇게 생겼다'를 규칙으로 적기는 매우 어렵지만, 0부터 9까지의 손글씨 이미지 수만 장과 그 정답 레이블을 학습시키면 모델은 스스로 각 숫자의 형태적 특징을 익혀 처음 보는 손글씨도 높은 정확도로 분류한다.

5.2 학습의 세 갈래: 지도·비지도·강화학습

머신러닝은 학습에 사용하는 데이터의 형태와 피드백 방식에 따라 크게 세 갈래로 나뉜다. 지도학습(supervised learning)은 입력과 정답(레이블)이 짝지어진 데이터를 사용해 입력으로부터 정답을 예측하는 함수를 학습한다. 집의 면적으로 가격을 예측하는 회귀나, 메일을 스팸/정상으로 나누는 분류가 대표적이다. 정답이라는 '교사 신호'가 있기 때문에 학습 목표가 명확하다는 특징이 있다.

비지도학습(unsupervised learning)은 정답 레이블 없이 데이터 자체의 구조와 패턴을 발견한다. 고객을 구매 성향에 따라 몇 개의 집단으로 묶는 군집화, 고차원 데이터를 소수의 축으로 요약하는 차원 축소가 여기에 속한다. 정답이 없으므로 결과의 좋고 나쁨을 평가하기가 지도학습보다 까다롭고, 발견된 패턴의 의미를 사람이 해석해야 하는 경우가 많다.

강화학습(reinforcement learning)은 에이전트가 환경과 상호작용하며 행동에 대한 보상(reward)을 받아, 누적 보상을 최대화하는 정책을 학습한다. 바둑이나 게임 플레이, 로봇 제어처럼 순차적 의사결정이 중요한 문제에 적합하다. 정답이 즉시 주어지지 않고 여러 행동을 거친 뒤에야 보상이 나타나므로, 탐험(exploration)과 활용(exploitation)의 균형을 맞추는 것이 핵심 과제가 된다.

세 갈래는 서로 배타적이지 않으며 실제 문제에서는 결합되기도 한다. 예컨대 레이블이 일부만 있는 상황에서 지도학습과 비지도학습을 함께 쓰는 준지도학습, 데이터 자체에서 정답 신호를 만들어 학습하는 자기지도학습 등이 활발히 연구된다. 이 책의 제2부에서는 입문 단계에서 가장 기본이 되는 지도학습과 비지도학습을 중심으로 다룬다.

5.3 기본 용어: 데이터·특징·레이블·모델

머신러닝에서 데이터는 보통 표 형태로 정리된다. 각 행은 하나의 관측치, 즉 샘플(sample) 또는 인스턴스(instance)이며, 각 열은 그 샘플을 설명하는 특징(feature)이다. 예를 들어 집값 예측 데이터에서 한 행은 특정 주택 하나이고, 면적·방 개수·건축 연도 등이 특징이 된다. 여러 특징을 모은 벡터를 특징 벡터라 부르며 보통 x 로 표기한다.

지도학습에서는 각 샘플에 우리가 예측하고자 하는 목표값이 함께 주어지는데, 이를 레이블(label) 또는 타겟(target)이라 하고 보통 y 로 표기한다. 레이블이 집값처럼 연속적인 수이면 회귀 문제, 스팸/정상처럼 몇 개의 범주 중 하나이면 분류 문제이다. 입력 x 와 정답 y 의 쌍 (x, y)들이 모여 학습 데이터셋을 이룬다.

모델(model)은 입력 x 를 받아 예측값 $\hat{y}$ 를 내놓는 함수 $\hat{y} = f(x; \theta)$ 이다. 여기서 θ 는 모델이 데이터에 맞춰 조정하는 파라미터(parameter)로, 선형회귀의 기울기와 절편이나 신경망의 가중치가 그 예이다. 학습(training)이란 주어진 데이터에 대해 예측이 정답에 가까워지도록 파라미터 θ 를 조정하는 과정을 말한다. 학습이 끝난 모델은 새로운 입력에 대해 예측을 수행하는데 쓰인다.

예시

붓꽃(iris) 데이터셋에서는 꽃받침 길이·너비, 꽃잎 길이·너비의 네 가지가 특징(x)이고, 세 가지 품종(setosa-versicolor-virginica) 중 하나가 레이블(y)이다. 모델은 네 개의 수치를 입력받아 품종을 예측한다.

5.4 학습은 손실 최소화이다

모델의 예측이 얼마나 틀렸는지를 하나의 수로 나타내는 함수를 손실함수(loss function) 또는 비용함수(cost function)라 한다. 손실이 클수록 예측이 정답에서 멀리 떨어져 있다는 뜻이며, 학습의 목표는 이 손실을 가능한 한 작게 만드는 파라미터를 찾는 것이다. 즉 학습은 본질적으로 손실함수를 최소화하는 최적화 문제로 정식화된다.

회귀 문제에서 널리 쓰이는 손실은 평균제곱오차(MSE)이다. 이는 각 샘플의 예측 오차를 제곱하여 평균낸 값으로, 오차가 클수록 더 크게 벌점을 준다. 분류 문제에서는 예측 확률과 실제 정답의 차이를 재는 교차엔트로피(cross-entropy) 손실이 흔히 사용된다. 문제의 성격에 맞는 손실함수를 선택하는 것은 학습이 무엇을 '잘하도록' 최적화될지를 결정하는 중요한 설계 요소이다.

손실을 최소화하는 파라미터를 찾는 대표적 방법이 6장에서 자세히 다룰 경사하강법(gradient descent)이다. 손실함수를 파라미터에 대한 지형(surface)으로 상상하면, 경사하강법은 기울기가 가장 가파르게 내려가는 방향으로 조금씩 이동하며 골짜기(최솟값)를 찾아가는 절차이다. 다만 학습 데이터의 손실만 무작정 줄이는 것이 항상 좋은 것은 아니라는 점을 다음 절에서 살펴본다.

수식

$$MSE(\theta) = (1/n) \cdot \sum (y - \hat{y})^2, \quad \hat{y} = f(x; \theta)$$

5.5 일반화와 데이터 분할

머신러닝의 진짜 목표는 학습에 사용한 데이터를 잘 맞히는 것이 아니라, 학습 때 보지 못한 새로운 데이터에 대해서도 잘 예측하는 능력, 곧 일반화이다. 학습 데이터에만 지나치게 맞춰져 세부적인 잡음까지 외워 버리면, 정작 새로운 데이터에서는 성능이 떨어진다. 이러한 현상을 과적합(overfitting)이라 하며, 반대로 모델이 너무 단순해 데이터의 패턴조차 담지 못하는 경우를 과소적합(underfitting)이라 한다.

일반화 성능을 정직하게 가늠하려면 데이터를 용도에 따라 나누어야 한다. 흔히 훈련 세트(training set)로 파라미터를 학습하고, 검증 세트(validation set)로 하이퍼파라미터를 조정하거나 모델을 선택하며, 마지막으로 테스트 세트(test set)로 최종 성능을 한 번만 평가한다. 테스트 세트는 학습 과정 어디에도 관여해서는 안 되며, 그래야 미지의 데이터에 대한 성능을 편향 없이 추정할 수 있다.

데이터가 부족할 때는 단순 분할 대신 교차검증(cross-validation)을 사용해 데이터를 더 효율적으로 활용한다. 이는 9장에서 자세히 다룬다. 핵심은, 모델을 평가한 데이터로 다시 모델을 고르거나 학습하면 성능이 부풀려지는 데이터 누수(data leakage)가 발생하므로, 평가용 데이터를 학습 과정과 철저히 분리해야 한다는 원칙이다.

예시

1,000개의 샘플을 훈련 60%, 검증 20%, 테스트 20%로 나눈다면 훈련 600개, 검증 200개, 테스트 200개가 된다. 여러 모델을 훈련·검증으로 비교해 최선의 모델을 고른 뒤, 마지막에 테스트 200개로 단 한 번 성능을 보고한다.

5.6 특징 공학 입문

모델의 성능은 알고리즘 못지않게 입력으로 주는 특징의 품질에 크게 좌우된다. 특징 공학(feature engineering)이란 원시 데이터를 모델이 학습하기 좋은 형태의 특징으로 가공하는 작업을 말한다. 예를 들어 날짜에서 요일이나 계절을 뽑아내거나, 키와 몸무게로부터 체질량지수를 계산하는 것처럼, 문제에 대한 도메인 지식을 특징에 반영하면 단순한 모델로도

좋은 성능을 얻을 수 있다.

특징 가공의 기본 절차에는 결측값 처리, 범주형 변수의 인코딩, 수치 특징의 스케일링이 포함된다. 범주형 값은 원-핫 인코딩 등으로 수치화하고, 크기 단위가 제각각인 수치 특징은 표준화(평균 0, 표준편차 1)나 정규화(0~1 범위)로 크기를 맞춘다. 거리 기반 알고리즘이나 경사하강법 기반 모델에서는 스케일을 맞추는 것이 학습의 안정성과 속도에 특히 중요하다.

최근에는 딥러닝이 원시 데이터로부터 유용한 특징을 스스로 학습하는 표현 학습(representation learning)으로 특징 공학의 부담을 줄여 주지만, 정형 데이터를 다루는 많은 현실 문제에서는 여전히 좋은 특징을 설계하는 일이 성패를 가른다. 특징을 만들 때도 테스트 데이터의 정보가 새어 들어가지 않도록, 스케일링의 기준값 등은 반드시 훈련 데이터로만 계산해야 한다.

이 장의 요약

- 머신러닝은 규칙을 직접 프로그래밍하지 않고 데이터에서 규칙(모델)을 학습하는 방법이다.
- 학습은 지도학습(정답 있음), 비지도학습(정답 없음), 강화학습(보상 기반)으로 나뉜다.
- 데이터는 샘플(행)과 특징(열)로 구성되고, 지도학습에는 레이블(정답)이 함께 주어진다.
- 모델 $= f(x; \theta)$ 는 파라미터 θ 를 데이터에 맞춰 조정하며, 이 과정이 학습이다.
- 학습은 손실함수(MSE, 교차엔트로피 등)를 최소화하는 최적화 문제로 정식화된다.
- 진짜 목표는 미지의 데이터에 대한 일반화이며, 과적합과 과소적합을 경계해야 한다.
- 훈련·검증·테스트로 데이터를 분할하고 테스트 세트는 최종 평가에만 사용한다.
- 특징 공학은 원시 데이터를 좋은 특징으로 가공하는 작업으로 모델 성능에 큰 영향을 준다.

핵심 용어

머신러닝(Machine Learning, ML)	데이터로부터 규칙(모델)을 학습하여 예측·판단을 수행하는 인공지능 분야.
지도학습(Supervised Learning)	입력과 정답 레이블의 쌍으로 예측 함수를 학습하는 방식으로 회귀와 분류가 대표적이다.
비지도학습(Unsupervised Learning)	레이블 없이 데이터의 구조·패턴을 찾는 방식으로 군집화와 차원 축소가 속한다.
강화학습(Reinforcement Learning)	환경과 상호작용하며 보상을 최대화하는 정책을 학습하는 방식이다.
특징(Feature)	샘플을 설명하는 입력 변수로, 데이터 표의 각 열에 해당한다.
레이블(Label)	지도학습에서 예측 대상이 되는 정답값으로 타깃이라고도 한다.
샘플(Sample)	하나의 관측치로 데이터 표의 한 행에 해당하며 인스턴스라고도 한다.
모델(Model)	입력 x 를 예측값 $\hat{y}$ 로 변환하는 함수 $f(x; \theta)$ 를 말한다.
파라미터(Parameter)	모델이 데이터에 맞춰 학습으로 조정하는 내부 값(가중치·절편 등)이다.
손실함수(Loss Function)	예측이 정답에서 얼마나 벗어났는지를 수치화한 함수로 학습은 이를 최소화한다.

일반화(Generalization)	학습에 쓰지 않은 새로운 데이터에 대해서도 잘 예측하는 능력이다.
과적합(Overfitting)	훈련 데이터의 잡음까지 외워 새 데이터에서 성능이 떨어지는 현상이다.
훈련/검증/테스트 세트	학습·모델 선택·최종 평가에 각각 사용하는 데이터 분할이다.
특징 공학(Feature Engineering)	원시 데이터를 모델이 학습하기 좋은 특징으로 가공하는 작업이다.

제6장

회귀와 경사하강법

회귀(regression)는 연속적인 수치 값을 예측하는 지도학습 문제이다. 집의 면적으로 가격을, 광고비로 매출을 예측하는 것이 대표적인 예이다. 이 장에서는 가장 기본이 되는 선형회귀 모델과 그 가설함수를 정의하고, 예측의 오차를 재는 평균제곱오차(MSE) 손실을 살펴본다. 이어 손실을 최소화하는 대표적 방법인 경사하강법을 배치·확률적·미니배치 방식으로 구분하여 알고리즘과 함께 소개하고, 곡선을 표현하는 다항회귀와 과적합을 억제하는 정규화(릿지·라쏘)까지 다룬다.

6.1 선형회귀와 가설함수

선형회귀는 입력 특징들의 가중합으로 출력을 예측하는 모델이다. 특징이 하나일 때 가설함수는 $y = w \cdot x + b$ 형태의 직선이며, 여기서 w 는 기울기(가중치), b 는 절편(bias)이다. 특징이 여러 개이면 각 특징에 가중치를 곱해 모두 더하고 절편을 더한다. 즉 모델은 데이터를 가장 잘 설명하는 직선(또는 초평면)의 기울기와 위치를 찾는 문제로 귀결된다.

선형회귀가 '선형'인 이유는 파라미터 w 에 대해 예측이 선형이기 때문이다. 이 단순함 덕분에 해석이 쉽다. 예를 들어 면적의 가중치가 500이라면 면적이 1 단위 늘 때 예측 가격이 500만큼 오른다고 직접 해석할 수 있다. 이런 해석 가능성은 선형회귀가 여전히 널리 쓰이는 중요한 이유 중 하나이다.

여러 특징을 다룰 때는 표기를 간결히 하기 위해 벡터를 사용한다. 특징 벡터 x 와 가중치 벡터 w 의 내적에 절편을 더한 $y = w \cdot x + b$ 로 쓴다. 절편 b 를 가중치 벡터에 포함시키기 위해 항상 1인 특징을 하나 추가하는 기법도 흔히 쓰인다. 이제 남은 과제는 데이터에 가장 잘 맞는 w 와 b 를 어떻게 정할 것인가이다.

수식

단일: $y = w \cdot x + b$ | 다중: $y = w \cdot x + b = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$

6.2 평균제곱오차(MSE) 손실

선형회귀에서 가장 널리 쓰이는 손실은 평균제곱오차(Mean Squared Error, MSE)이다. 각 샘플에서 실제값 y 와 예측값 $\hat{y}$ 의 차이(잔차)를 제곱하여 모든 샘플에 대해 평균낸 값이다. 오차를 제곱하기 때문에 부호가 상쇄되지 않고, 큰 오차에 더 큰 벌점을 준다는 특징이 있다. 손실이 0에 가까울수록 예측이 실제값에 잘 들어맞는다.

MSE를 손실로 쓰면 손실함수가 파라미터에 대해 아래로 볼록한(convex) 이차식이 되어, 최솟값이 하나뿐인 매끄러운 그릇 모양이 된다. 덕분에 경사하강법이 국소 최솟값에 빠질 걱정 없이 전역 최솟값으로 수렴한다. 또한 미분이 간단해 기울기를 손으로도 구할 수 있다는 실용적 장점이 있다.

선형회귀와 MSE의 조합은 정규방정식(normal equation)이라는 닫힌 형태의 해로도 풀 수 있다. 그러나 특징 수가 매우 많거나 데이터가 방대해 역행렬 계산이 부담될 때는, 반복적으로

해에 접근하는 경사하강법이 더 실용적이다. 다음 절에서 그 원리를 살펴본다.

수식

$$\text{MSE}(w, b) = (1/n) \cdot \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \hat{y}_i = w x_i + b$$

예시

실제값이 [3, 5, 7], 예측값이 [2.5, 5, 8]이면 잔차는 [0.5, 0, -1], 제곱은 [0.25, 0, 1], 합은 1.25, 샘플 수 3으로 나눈 MSE는 약 0.417이다.

6.3 경사하강법의 원리

경사하강법(gradient descent)은 손실함수의 기울기(gradient)를 이용해 손실이 줄어드는 방향으로 파라미터를 반복적으로 조금씩 갱신하는 최적화 알고리즘이다. 기울기는 각 파라미터를 조금 늘렸을 때 손실이 얼마나 변하는지를 나타내는 벡터로, 손실이 가장 가파르게 증가하는 방향을 가리킨다. 따라서 그 반대 방향으로 이동하면 손실이 감소한다.

갱신식은 $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$ 이다. 여기서 θ 는 파라미터, $\nabla L(\theta)$ 는 손실의 기울기, η (에타)는 학습률(learning rate)이라 부르는 이동 보폭이다. 이 과정을 손실이 충분히 작아지거나 더 이상 줄지 않을 때까지 반복한다. 손실 지형을 안개 낀 산에서 발밑의 가장 가파른 내리막을 따라 한 걸음씩 골짜기로 내려가는 것에 비유할 수 있다.

학습률 η 의 선택이 매우 중요하다. η 가 너무 작으면 수렴이 느려 학습에 오랜 시간이 걸리고, 너무 크면 최솟값을 지나쳐 손실이 오히려 커지거나 발산할 수 있다. 실무에서는 여러 학습률을 시도하거나, 학습이 진행되면서 학습률을 점차 줄이는 학습률 스케줄링을 사용한다. 특징의 스케일을 미리 맞추면 경사하강법이 더 안정적이고 빠르게 수렴한다.

수식

$$\theta \leftarrow \theta - \eta \cdot \nabla L(\theta) \quad | \quad \text{선형회귀 MSE 기울기: } \partial L / \partial w = (2/n) \cdot \sum x_i (y_i - \hat{y}_i), \quad \partial L / \partial b = (2/n) \cdot \sum (y_i - \hat{y}_i)$$

의사코드

```
# 선형회귀 배치 경사하강법 (의사코드)
w, b = 0, 0          # 파라미터 초기화
eta = 0.01           # 학습률
for epoch in range(num_epochs): # 전체 반복
    y_pred = w * X + b      # 예측 (모든 샘플)
    error = y_pred - y      # 잔차 (y - y_pred)
    grad_w = (2/n) * sum(X * error) # ∂L/∂w
    grad_b = (2/n) * sum(error)     # ∂L/∂b
    w = w - eta * grad_w           # 가중치 갱신
    b = b - eta * grad_b           # 절편 갱신
    # 손실이 충분히 작아지면 조기 종료 가능
return w, b
```

6.4 배치·확률적·미니배치 경사하강법

경사하강법은 한 번의 갱신에 얼마나 많은 데이터를 사용하는가에 따라 세 가지로 나뉜다. 배치 경사하강법(batch gradient descent)은 매 갱신마다 전체 훈련 데이터로 기울기를 계산한다. 기울기 추정이 정확해 수렴이 안정적이지만, 데이터가 많으면 한 번의 갱신에도 계산량이 커서 느리다는 단점이 있다.

확률적 경사하강법(Stochastic Gradient Descent, SGD)은 매 갱신마다 무작위로 고른 샘플 하나만으로 기울기를 계산한다. 갱신이 매우 빠르고 온라인 학습에 적합하지만, 한 샘플에 기반하므로 기울기에 잡음이 많아 손실이 요동치며 감소한다. 이 잡음은 때때로 얇은 국소 최솟값을 빠져나오는 데 도움을 주기도 한다.

미니배치 경사하강법(mini-batch gradient descent)은 두 방식의 절충안으로, 매 갱신마다 32개나 64개 같은 소규모 묶음(미니배치)으로 기울기를 계산한다. 배치의 안정성과 SGD의 속도를 함께 취하며, 행렬 연산으로 GPU를 효율적으로 활용할 수 있어 오늘날 딥러닝 학습의 사실상 표준으로 쓰인다. 배치 크기는 조정 가능한 하이퍼파라미터이다.

예시

훈련 데이터가 10,000개일 때, 배치는 10,000개 전부로 1회 갱신하고, SGD는 1개씩 10,000회 갱신하며, 배치 크기 100인 미니배치는 100개씩 100회 갱신하여 1 에폭(epoch, 전체 데이터 1회 순회)을 마친다.

6.5 다항회귀와 비선형 관계

현실의 많은 관계는 직선으로 표현하기 어렵다. 다항회귀(polynomial regression)는 입력 특징의 거듭제곱 항(x^2 , x^3 등)을 새로운 특징으로 추가하여 곡선 형태의 관계를 학습한다. 예컨대 $= w_1x + w_2x^2 + b$ 는 포물선을 표현할 수 있다. 이때도 모델은 여전히 파라미터 w 에 대해 선형이므로, 앞서 배운 선형회귀와 경사하강법을 그대로 적용할 수 있다.

다항의 차수(degree)는 모델의 복잡도를 조절하는 하이퍼파라미터이다. 차수가 낮으면 데이터의 곡률을 담지 못해 과소적합이 일어나고, 차수가 지나치게 높으면 훈련 데이터의 잡음까지 따라가 구불구불한 곡선이 되면서 과적합이 발생한다. 따라서 적절한 차수를 검증 데이터로 선택해야 한다.

다항회귀는 모델의 표현력을 높이는 손쉬운 방법이지만, 차수를 높일수록 특징 수가 빠르게 늘고 과적합 위험이 커진다는 대가가 따른다. 이런 위험을 줄이기 위해 다음 절에서 다룰 정규화 기법을 함께 사용하는 경우가 많다. 표현력과 일반화 사이의 균형을 잡는 것이 회귀 모델링의 핵심이다.

수식

$$\text{다항회귀(차수 } m): \quad = b + w_1x + w_2x^2 + \dots + w_mx^m$$

6.6 과적합과 정규화(L1·L2)

과적합을 억제하는 대표적 방법이 정규화(regularization)이다. 정규화는 손실함수에 가중치의 크기에 대한 벌점 항을 더하여, 모델이 특정 가중치를 지나치게 크게 키우지 못하도록 제약한다. 가중치가 작을수록 모델이 단순하고 매끄러워져 잡음에 덜 민감해지므로 일반화 성능이

향상되는 효과가 있다. 벌점의 세기는 하이퍼파라미터 λ (람다)로 조절한다.

L2 정규화는 가중치 제곱합을 벌점으로 더하며, 이를 적용한 선형회귀를 릿지 회귀(Ridge regression)라 한다. 릿지는 가중치를 전반적으로 0에 가깝게 줄이되 완전히 0으로 만들지는 않아, 상관된 특징들에 가중치를 고르게 분배하는 경향이 있다. 반면 L1 정규화는 가중치 절댓값의 합을 벌점으로 더하며 이를 라쏘 회귀(Lasso regression)라 한다.

라쏘는 일부 가중치를 정확히 0으로 만드는 성질이 있어, 중요하지 않은 특징을 자동으로 골라내는 특징 선택(feature selection) 효과를 낸다. 따라서 특징이 매우 많고 그중 일부만 중요하다고 여겨질 때 유용하다. 두 방식을 결합한 엘라스틱넷(Elastic Net)도 널리 쓰인다. λ 가 너무 크면 벌점이 과해 과소적합이 되므로, λ 역시 검증 데이터로 적절히 선택해야 한다.

수식

$$L2(\text{릿지}): L = \text{MSE} + \lambda \sum w^2 \quad L1(\text{라쏘}): L = \text{MSE} + \lambda \sum |w|$$

예시

특징이 50개인데 실제로는 5개만 예측에 유효한 상황에서, 라쏘(L1)는 나머지 45개 가중치를 0으로 만들어 희소한 모델을 만들고, 릿지(L2)는 50개 가중치를 모두 작게 유지한다.

이 장의 요약

- 선형회귀는 특징의 가중합 $= w \cdot x + b$ 로 연속값을 예측하는 모델이다.
- 평균제곱오차 $\text{MSE} = (1/n) \cdot \sum (y - \hat{y})^2$ 가 회귀의 대표적 손실함수이다.
- 경사하강법은 $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$ 로 손실이 줄어드는 방향으로 파라미터를 갱신한다.
- 학습률 η 가 너무 작으면 느리고 너무 크면 발산하므로 적절히 선택해야 한다.
- 배치·확률적(SGD)·미니배치는 한 갱신에 쓰는 데이터 양에 따른 구분이다.
- 다항회귀는 거듭제곱 항을 추가해 곡선 관계를 표현하며 차수로 복잡도를 조절한다.
- 정규화는 손실에 가중치 벌점을 더해 과적합을 억제한다.
- L2(릿지)는 가중치를 고르게 줄이고, L1(라쏘)는 일부를 0으로 만들어 특징을 선택한다.

핵심 용어

선형회귀(Linear Regression)	특징의 가중합으로 연속값을 예측하는 지도학습 회귀 모델이다.
가설함수(Hypothesis Function)	입력을 예측값으로 대응시키는 모델 함수 $= w \cdot x + b$ 이다.
가중치(Weight)	각 특징이 예측에 기여하는 정도를 나타내는 파라미터이다.
절편(Bias)	모든 특징이 0일 때의 기준 예측값을 정하는 파라미터이다.
평균제곱오차(MSE)	잔차 제곱의 평균으로 정의되는 회귀 손실함수이다.
경사하강법(Gradient Descent)	손실의 기울기 반대 방향으로 파라미터를 갱신하는 최적화 알고리즘이다.
학습률(Learning Rate, η)	한 번의 갱신에서 파라미터가 이동하는 보폭을 정하는 하이퍼파라미터이다.

확률적 경사하강법(SGD)	매 갱신마다 샘플 하나로 기울기를 계산하는 방식으로 빠르지만 잡음이 많다.
미니배치(Mini-batch)	소규모 샘플 묶음으로 기울기를 계산하는 경사하강법의 절충 방식이다.
에폭(Epoch)	전체 훈련 데이터를 한 번 모두 사용해 학습을 순회하는 단위이다.
다항회귀(Polynomial Regression)	거듭제곱 항을 특징으로 추가해 곡선 관계를 표현하는 회귀이다.
정규화(Regularization)	가중치 크기에 벌점을 부과해 과적합을 억제하는 기법이다.
릿지 회귀(Ridge, L2)	가중치 제곱합을 벌점으로 더해 가중치를 전반적으로 줄이는 회귀이다.
라쏘 회귀(Lasso, L1)	가중치 절댓값 합을 벌점으로 더해 일부 가중치를 0으로 만드는 회귀이다.

제7장 분류 알고리즘

분류(classification)는 입력을 미리 정해진 몇 개의 범주(클래스) 중 하나로 예측하는 지도학습 문제이다. 메일을 스팸/정상으로 나누거나, 이미지에서 개와 고양이를 구별하는 것이 그 예이다. 이 장에서는 확률 기반의 로지스틱 회귀와 시그모이드·교차엔트로피부터 시작하여, 이웃을 참고하는 K-최근접이웃, 규칙을 나무 구조로 학습하는 결정트리와 정보이득·지니, 여러 모델을 결합하는 앙상블, 마진을 최대화하는 서포트 벡터 머신, 확률의 곱으로 판단하는 나이브 베이즈까지 대표적인 분류 알고리즘을 두루 살펴본다.

7.1 로지스틱 회귀와 시그모이드

이름에 '회귀'가 들어 있지만 로지스틱 회귀(logistic regression)는 이진 분류를 위한 대표적 알고리즘이다. 선형회귀처럼 특징의 가중합 $z = w x + b$ 를 계산한 뒤, 이 값을 시그모이드(sigmoid) 함수에 통과시켜 0과 1 사이의 확률로 변환한다. 출력값은 '입력이 양성 클래스에 속할 확률'로 해석되며, 보통 0.5를 기준으로 클래스를 결정한다.

시그모이드 함수 $\sigma(z) = 1/(1 + e^{-z})$ 는 어떤 실수든 0과 1 사이로 매끄럽게 눌러 주는 S자 곡선이다. z 가 크면 1에, 작으면 0에 가까워지고 z 가 0이면 0.5를 반환한다. 이 성질 덕분에 선형 결합의 결과를 확률로 자연스럽게 바꿀 수 있고, 함수가 미분 가능하여 경사하강법으로 학습할 수 있다.

로지스틱 회귀는 결정 경계가 직선(또는 초평면)인 선형 분류기이다. 해석이 쉽고 확률을 출력하며 학습이 빠르다는 장점 때문에, 복잡한 모델의 성능을 비교하는 기준선(baseline)으로도 자주 쓰인다. 여러 클래스를 다룰 때는 소프트맥스(softmax) 함수로 확장한 소프트맥스 회귀를 사용한다.

수식

$$z = w x + b, \sigma(z) = 1 / (1 + e^{-(z)}), P(y=1 | x) = \sigma(z)$$

7.2 교차엔트로피 손실

로지스틱 회귀의 학습에는 교차엔트로피(cross-entropy) 손실, 다른 이름으로 로그 손실(log loss)을 사용한다. 이 손실은 모델이 예측한 확률이 실제 정답과 얼마나 잘 맞는지를 측정한다. 정답이 1인데 모델이 1에 가까운 확률을 주면 손실이 작고, 정답과 반대되는 확률을 자신 있게 주면 손실이 매우 커진다.

이진 분류의 교차엔트로피는 각 샘플에 대해 $[-y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$ 형태이다. 정답 y 가 1이면 $\log(\hat{y})$ 만 남아 예측 확률 $\hat{y}$ 가 1에 가까울수록 손실이 0에 수렴하고, y 가 0이면 $\log(1 - \hat{y})$ 만 남아 $\hat{y}$ 가 0에 가까울수록 손실이 작아진다. 확률이 틀린 쪽으로 갈수록 로그 때문에 손실이 급격히 커지는 것이 핵심이다.

MSE 대신 교차엔트로피를 쓰는 이유는, 시그모이드와 결합했을 때 손실함수가 볼록해지고 기울기가 잘 전달되어 학습이 안정적이기 때문이다. 선형회귀의 MSE와 마찬가지로 이 손실도

경사하강법으로 최소화하며, 기울기가 $(-y) \cdot x$ 라는 간결한 형태를 가진다는 점이 실용적 장점이다.

수식

$$L = (1/n) \cdot \sum [y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

예시

정답 $y=1$ 인 샘플에서 모델이 $\hat{y}=0.9$ 를 예측하면 손실은 $-\log(0.9) \approx 0.105$ 로 작지만, $\hat{y}=0.1$ 로 틀리게 예측하면 $-\log(0.1) \approx 2.303$ 으로 크게 벌점을 받는다.

7.3 K-최근접이웃(KNN)

K-최근접이웃(K-Nearest Neighbors, KNN)은 별도의 학습 과정 없이 예측 시점에 판단하는 게으른 학습(lazy learning) 알고리즘이다. 새로운 입력이 주어지면 훈련 데이터 중 가장 가까운 K개의 이웃을 찾고, 그 이웃들의 다수결 클래스로 분류를 결정한다. 회귀에 쓸 때는 이웃들의 평균값을 예측으로 사용한다. '가깝다'는 보통 유클리드 거리로 정의한다.

K 값은 결과에 큰 영향을 준다. K가 너무 작으면(예: $K=1$) 잡음 하나에도 민감해 과적합되고, K가 너무 크면 서로 다른 클래스가 뒤섞여 경계가 흐려지며 과소적합이 된다. 보통 검증 데이터로 적절한 K를 고르며, 동점을 피하기 위해 이진 분류에서는 홀수 K를 선호한다.

KNN은 원리가 직관적이고 구현이 간단하지만, 예측할 때마다 모든 훈련 데이터와의 거리를 계산해야 하므로 데이터가 많으면 느리고 메모리를 많이 쓴다. 또한 거리 기반이므로 특징의 스케일에 민감하여 반드시 스케일링이 필요하고, 특징 수가 매우 많으면 다음 장에서 다룰 차원의 저주로 성능이 떨어진다.

예시

$K=3$ 으로 설정했을 때 새 샘플의 가장 가까운 이웃 3개의 클래스가 각각 A, A, B 라면 다수결에 따라 A로 분류한다.

7.4 결정트리: 정보이득과 지니

결정트리(decision tree)는 데이터를 특징에 대한 질문으로 반복해서 나누어 가는 나무 구조의 모델이다. 각 내부 노드는 '특징 A가 값 t보다 큰가?' 같은 조건을 나타내고, 가지를 따라 내려가 도달한 잎 노드에서 예측 클래스가 결정된다. 규칙이 사람이 읽을 수 있는 형태로 드러나 해석이 매우 쉽다는 점이 큰 장점이다.

트리를 만들 때는 각 분기에서 데이터를 가장 잘 나누는 질문을 고른다. '잘 나눈다'는 것은 나눈 뒤 각 그룹이 한 클래스로 순수해진다는 뜻으로, 순수도를 재는 지표로 엔트로피와 지니 불순도(Gini impurity)를 쓴다. 분기 전후의 불순도 감소량을 정보이득(information gain)이라 하며, 정보이득이 가장 큰 질문을 선택해 나무를 키운다.

결정트리는 특징 스케일링이 필요 없고 수치·범주형 특징을 함께 다룰 수 있어 편리하지만, 나무를 끝까지 키우면 훈련 데이터에 과적합되기 쉽다. 이를 막기 위해 나무의 최대 깊이를 제한하거나, 다 자란 나무의 가지를 쳐내는 가지치기(pruning)를 적용한다. 단일 트리의

불안정성을 극복하기 위해 다음 절의 앙상블이 등장한다.

수식

지니 불순도: $Gini = 1 - \sum p^2$ 엔트로피: $H = - \sum p \cdot \log_2 p$ 정보이득: $IG = H(\text{부모}) - \sum (\text{자식 비율}) \cdot H(\text{자식})$

예시

어떤 노드에 두 클래스가 각각 50%씩 섞여 있으면 지니는 $1 - (0.5^2 + 0.5^2) = 0.5$ 로 가장 불순하고, 한 클래스만 100%이면 지니는 $1 - 1^2 = 0$ 으로 완전히 순수하다.

7.5 앙상블: 배깅·랜덤포레스트·부스팅

앙상블(ensemble)은 여러 개의 모델을 결합해 단일 모델보다 나은 성능과 안정성을 얻는 기법이다. 여러 사람의 의견을 모으면 개인의 실수가 상쇄되듯, 여러 모델의 예측을 합치면 오차가 줄어드는 효과가 있다. 대표적인 방식으로 배깅과 부스팅이 있다.

배깅(bagging, bootstrap aggregating)은 훈련 데이터에서 중복을 허용해 여러 개의 표본을 뽑고, 각 표본으로 독립적인 모델을 학습시킨 뒤 예측을 평균(회귀)하거나 다수결(분류)로 합친다. 결정트리에 배깅을 적용하되 각 분기에서 특징도 무작위로 일부만 고려하도록 한 것이 랜덤포레스트(random forest)이다. 랜덤포레스트는 트리 간 상관을 낮춰 과적합을 크게 줄이며 강력하고 안정적인 성능을 보인다.

부스팅(boosting)은 모델을 순차적으로 학습시키되, 앞선 모델이 틀린 샘플에 더 집중하도록 가중치를 조정하여 약한 모델들을 점진적으로 강한 모델로 키운다. 에이다부스트, 그리고 잔차를 순차적으로 학습하는 그래디언트 부스팅(XGBoost, LightGBM 등)이 대표적이며, 정형 데이터 분류·회귀에서 최상위 성능을 자주 낸다. 다만 순차 학습이라 병렬화가 어렵고 과적합에 주의가 필요하다.

예시

100그룹의 결정트리로 이루어진 랜덤포레스트에서 새 샘플에 대해 63그룹이 '양성', 37그룹이 '음성'으로 예측하면 다수결로 '양성'이 최종 예측이 된다.

7.6 서포트 벡터 머신과 나이브 베이즈

서포트 벡터 머신(Support Vector Machine, SVM)은 두 클래스를 나누는 여러 경계 중, 양쪽 클래스에서 가장 가까운 점들과의 거리(마진, margin)를 최대로 하는 경계를 찾는 분류기이다. 마진을 최대화하면 새로운 데이터에 대한 일반화가 좋아진다는 직관에 기반한다. 이때 경계를 결정하는 최전선의 데이터 점들을 서포트 벡터라 부른다.

데이터가 직선으로 나뉘지 않을 때 SVM은 커널(kernel) 기법을 사용한다. 커널은 데이터를 더 높은 차원의 공간으로 옮긴 것처럼 계산하여, 원래 공간에서는 비선형인 경계를 그릴 수 있게 한다. 대표적으로 RBF(가우시안) 커널이 널리 쓰이며, 이 덕분에 SVM은 복잡한 경계를 가진 문제에도 강력하게 대응한다. 다만 대규모 데이터에서는 학습이 느릴 수 있다.

나이브 베이즈(Naive Bayes)는 베이즈 정리에 기반해 각 클래스일 확률을 계산하는 확률적 분류기이다. '모든 특징이 서로 독립'이라는 순진한(naive) 가정을 두어 계산을 크게 단순화한다. 이 가정은 현실과 다를 때가 많지만, 실제로는 텍스트 분류나 스팸 필터링처럼 특징이 많은 문제에서 빠르고 놀라울 만큼 잘 작동하여 널리 쓰인다.

수식

SVM 마진 최대화: $\max (2 / w) \text{ s.t. } y (w x + b) \geq 1$ 베이즈 정리: $P(y | x) = P(x | y) \cdot P(y) / P(x)$

이 장의 요약

- 분류는 입력을 정해진 범주 중 하나로 예측하는 지도학습 문제이다.
- 로지스틱 회귀는 시그모이드 $\sigma(z)=1/(1+e^{-z})$ 로 선형 결합을 확률로 바꾸는 선형 분류기이다.
- 분류 학습에는 교차엔트로피(로그 손실)를 사용하며 틀린 확신에 큰 벌점을 준다.
- KNN은 가장 가까운 K개 이웃의 다수결로 분류하는 게으른 학습 알고리즘이다.
- 결정트리는 정보이득·지니로 데이터를 나누며 해석이 쉽지만 과적합에 주의해야 한다.
- 앙상블은 여러 모델을 결합하며 배깅(랜덤포레스트)과 부스팅이 대표적이다.
- SVM은 마진을 최대화하는 경계를 찾고 커널로 비선형 문제를 해결한다.
- 나이브 베이즈는 특징 독립 가정으로 베이즈 정리를 단순화한 빠른 확률 분류기이다.

핵심 용어

분류(Classification)	입력을 정해진 여러 범주 중 하나로 예측하는 지도학습 문제이다.
로지스틱 회귀(Logistic Regression)	시그모이드로 확률을 출력하는 이진 분류용 선형 모델이다.
시그모이드(Sigmoid)	실수를 0~1 확률로 눌러 주는 S자 함수 $\sigma(z)=1/(1+e^{-z})$ 이다.
교차엔트로피(Cross-Entropy)	예측 확률과 정답의 불일치를 재는 분류 손실로 로그 손실이라고도 한다.
K-최근접이웃(KNN)	가장 가까운 K개 이웃의 다수결로 분류하는 게으른 학습 알고리즘이다.
결정트리(Decision Tree)	특징에 대한 질문으로 데이터를 나누는 나무 구조 모델이다.
정보이득(Information Gain)	분기 전후의 불순도(엔트로피) 감소량으로 최선의 분할을 고르는 기준이다.
지니 불순도(Gini Impurity)	한 노드에 여러 클래스가 섞인 정도를 재는 지표로 0이면 순수하다.
앙상블(Ensemble)	여러 모델을 결합해 성능과 안정성을 높이는 기법이다.
랜덤포레스트(Random Forest)	데이터와 특징을 무작위화한 다수의 결정트리를 배깅으로 결합한 모델이다.
부스팅(Boosting)	약한 모델을 순차적으로 학습해 오차를 줄여 강한 모델로 키우는 앙상블이다.
서포트 벡터 머신(SVM)	클래스 간 마진을 최대화하는 경계를 찾는 분류기이다.

커널(Kernel)	데이터를 고차원으로 사상한 것처럼 계산해 비선형 경계를 가능케 하는 기법이다.
나이브 베이즈(Naive Bayes)	특징 독립 가정과 베이즈 정리에 기반한 빠른 확률적 분류기이다.

제8장

비지도학습과 차원 축소

비지도학습은 정답 레이블 없이 데이터 스스로가 지닌 구조와 패턴을 발견하는 방법이다. 레이블을 붙이는 데 드는 비용이 크거나 애초에 정답이 존재하지 않는 상황에서 특히 유용하다. 이 장에서는 비슷한 데이터끼리 묶는 군집화의 대표 알고리즘인 K-평균과 적절한 군집 수를 정하는 엘보우 방법, 그리고 계층적 군집을 살펴본다. 이어 특징이 많을 때 생기는 차원의 저주를 설명하고, 데이터를 소수의 축으로 요약하는 주성분분석(PCA)의 직관과 절차를 다룬 뒤, 이상탐지와 전처리 같은 비지도학습의 활용을 소개한다.

8.1 군집화와 K-평균 알고리즘

군집화(clustering)는 비슷한 특성을 가진 데이터를 같은 그룹(군집)으로 묶는 비지도학습 작업이다. 정답 레이블이 없으므로, 같은 군집 안의 데이터는 서로 가깝고 다른 군집의 데이터와는 멀도록 그룹을 형성하는 것이 목표이다. 고객 세분화, 문서 주제 분류, 이미지 색상 압축 등 다양한 분야에서 활용된다.

가장 널리 쓰이는 K-평균(K-means) 알고리즘은 미리 정한 군집 수 K개의 중심(centroid)을 기준으로 데이터를 나눈다. 먼저 K개의 중심을 무작위로 놓은 뒤, 각 데이터를 가장 가까운 중심에 배정하고(할당 단계), 각 군집에 속한 데이터의 평균 위치로 중심을 옮긴다(갱신 단계). 이 두 단계를 중심이 더 이상 변하지 않을 때까지 반복한다.

K-평균은 빠르고 구현이 간단하지만 몇 가지 한계가 있다. 초기 중심 위치에 따라 결과가 달라질 수 있어 여러 번 실행하거나 K-means++ 같은 영리한 초기화를 사용한다. 또한 군집이 둥근 형태이고 크기가 비슷하다고 가정하므로 길쭉하거나 밀도가 다른 군집에는 약하고, 거리 기반이라 특징 스케일링이 필수이며 K를 미리 정해야 한다는 부담이 있다.

수식

목적함수(관성): $J = \sum \sum_{x \in C} \|x - \mu\|^2$ (μ 는 군집 C의 중심)

의사코드

```
# K-평균 알고리즘 (의사코드)
중심 K개를 무작위로 초기화
repeat:
    # 할당 단계
    for 각 데이터 x:
        x를 가장 가까운 중심의 군집에 배정
    # 갱신 단계
    for 각 군집 C:
        중심 = C에 속한 데이터의 평균
until 중심이 더 이상 변하지 않음
return 군집 배정 결과
```

8.2 군집 수 정하기: 엘보우 방법

K-평균에서 군집 수 K를 몇으로 할지는 결과를 좌우하는 중요한 선택이다. 정답이 없으므로 데이터만 보고 적절한 K를 가늠하는 방법이 필요하다. 가장 널리 쓰이는 것이 엘보우(elbow) 방법으로, K를 1부터 점차 늘려 가며 각 K에서의 군집 응집도, 즉 군집 내 거리 제곱합(관성, inertia)을 계산해 그래프로 그린다.

K가 커질수록 관성은 항상 줄어들지만, 어느 지점을 지나면 감소 폭이 급격히 완만해진다. 이 꺾이는 지점이 마치 팔꿈치(elbow)처럼 보이며, 그 K가 데이터 구조를 설명하기에 적절한 군집 수라고 판단한다. 그 이후로 K를 더 늘려도 얻는 이득이 작기 때문이다.

엘보우가 항상 뚜렷하게 나타나는 것은 아니어서, 보완적으로 실루엣 계수(silhouette score) 같은 지표를 함께 사용한다. 실루엣 계수는 각 데이터가 자기 군집에는 잘 묻혀 있고 이웃 군집과는 잘 분리되어 있는 정도를 -1에서 1 사이의 값으로 나타낸다. 결국 K 선택에는 지표와 더불어 문제의 맥락과 도메인 지식을 함께 고려하는 것이 바람직하다.

예시

K=1,2,3,4,5,6에 대한 관성이 각각 800, 340, 150, 120, 100, 88로 나올 때, K=3까지는 급감하다 이후 완만해지므로 팔꿈치 지점인 K=3을 군집 수로 택한다.

8.3 계층적 군집

계층적 군집(hierarchical clustering)은 군집 수를 미리 정하지 않고, 데이터 사이의 유사도에 따라 군집을 계층적으로 형성한다. 대표적인 병합적(agglomerative) 방식은 처음에 모든 데이터를 각각 하나의 군집으로 두고, 가장 가까운 두 군집을 반복적으로 합쳐 나가 마지막에 하나의 큰 군집이 될 때까지 진행한다.

이 과정을 나무 형태로 시각화한 것이 덴드로그램(dendrogram)이다. 덴드로그램을 특정 높이에서 자르면 원하는 개수의 군집을 얻을 수 있어, K를 사후에 유연하게 정할 수 있다는 장점이 있다. 군집 간 거리를 정의하는 연결 기준(최단·최장·평균·와드 연결 등)에 따라 결과가 달라지므로 문제에 맞게 선택한다.

계층적 군집은 군집의 개수를 미리 몰라도 되고 군집 구조를 한눈에 보여 주지만, 모든 데이터 쌍의 거리를 계산해야 하므로 데이터가 많으면 계산 비용이 매우 커진다. 따라서 소규모에서 중간 규모의 데이터나, 데이터의 위계 구조를 탐색적으로 이해하고 싶을 때 특히 유용하다.

8.4 차원의 저주

차원의 저주(curse of dimensionality)는 특징(차원)의 수가 늘어날수록 데이터 분석이 급격히 어려워지는 현상을 통칭한다. 차원이 높아지면 공간의 부피가 기하급수적으로 커져, 같은 수의 데이터로는 공간이 텅 비게 된다. 즉 데이터가 희소해져 신뢰할 만한 패턴을 찾으려면 필요한 데이터 양이 폭증한다.

고차원에서는 거리의 개념도 무더진다. 차원이 매우 높으면 가장 가까운 점과 가장 먼 점 사이의 거리 차이가 상대적으로 줄어들어, KNN이나 K-평균처럼 거리에 기반한 알고리즘의 성능이 떨어진다. 또한 특징이 많으면 모델이 잡음까지 학습하기 쉬워 과적합 위험도 커진다.

이러한 문제를 완화하기 위해 실제로 의미 있는 소수의 특징만 남기거나, 여러 특징을 결합해 더 적은 수의 새로운 축으로 데이터를 요약하는 차원 축소(dimensionality reduction)를 사용한다. 다행히 현실의 고차원 데이터는 실제로는 훨씬 낮은 차원의 구조 위에 놓여 있는 경우가 많아, 차원 축소가 효과적으로 작동한다. 대표적 방법이 다음 절의 주성분분석이다.

8.5 주성분분석(PCA)

주성분분석(Principal Component Analysis, PCA)은 데이터의 분산을 가장 잘 보존하는 새로운 축을 찾아 고차원 데이터를 저차원으로 요약하는 대표적 차원 축소 기법이다. 핵심 직관은 데이터가 가장 넓게 퍼져 있는(분산이 큰) 방향일수록 그 데이터의 정보를 많이 담고 있다는 것이다. PCA는 그런 방향을 순서대로 찾아낸다.

이 방향들을 주성분(principal component)이라 하며, 첫 번째 주성분은 분산이 가장 큰 방향, 두 번째 주성분은 첫 번째와 직교하면서 남은 분산이 가장 큰 방향이다. 데이터를 앞쪽 몇 개의 주성분 축에 투영하면, 원래 정보를 최대한 유지하면서 차원을 줄일 수 있다. 각 주성분이 전체 분산의 몇 퍼센트를 설명하는지를 보고 몇 개의 주성분을 남길지 결정한다.

PCA의 절차는 대략 다음과 같다. 먼저 각 특징을 평균이 0이 되도록 중심화하고 (스케일이 다르면 표준화도 함께 한다), 특징 간 공분산행렬을 구한 뒤, 그 고유벡터와 고유값을 계산한다. 고유값이 큰 순서대로 고유벡터가 주성분이 되며, 상위 몇 개를 골라 데이터를 그 축에 투영한다. PCA는 시각화, 잡음 제거, 다른 알고리즘의 전처리로 두루 쓰이지만, 새 축이 원래 특징의 선형 결합이라 해석이 어려워질 수 있다는 점은 유의해야 한다.

수식

누적 설명 분산비 = $(\sum_{i=1}^k \lambda_i) / (\sum_{j=1}^d \lambda_j)$ (λ 는 고유값, k 는 남긴 주성분 수)

예시

784차원(28×28 픽셀) 손글씨 이미지를 PCA로 50차원으로 줄이면서 전체 분산의 약 90%를 보존하면, 정보 손실을 최소화하면서도 이후 분류기의 학습 속도를 크게 높일 수 있다.

8.6 비지도학습의 활용: 이상탐지와 전처리

비지도학습은 이상탐지(anomaly detection)에 널리 활용된다. 정상 데이터가 이루는 패턴이나 밀집 영역에서 크게 벗어난 데이터를 이상치로 판단하는 방식이다. 군집의 어떤 중심에서도 멀리 떨어져 있거나, 밀도가 매우 낮은 영역에 놓인 점을 이상으로 본다. 신용카드 부정 거래 탐지, 설비 고장 예측, 네트워크 침입 탐지 등에서 정상 사례만으로 학습해 이상을 잡아내는 데 유용하다.

차원 축소와 군집화는 지도학습의 전처리 단계로도 요긴하다. PCA로 특징 수를 줄이면 차원의 저주를 완화하고 학습 속도를 높이며 잡음을 제거하는 효과가 있다. 군집 결과를 새로운 특징으로 추가하거나, 대량의 미레이블 데이터를 군집화해 대표 샘플에만 레이블을 붙이는 식으로 레이블링 비용을 아낄 수도 있다.

다만 비지도학습은 정답이 없기에 결과를 객관적으로 평가하기가 어렵고, 발견된 패턴이 실제로 의미가 있는지는 사람이 도메인 지식으로 해석하고 검증해야 한다. 또한 전처리에 쓰는

변환의 기준값(PCA의 축, 스케일링 평균 등)은 반드시 훈련 데이터로만 계산하고 테스트 데이터에 동일하게 적용해, 데이터 누수가 생기지 않도록 주의해야 한다.

이 장의 요약

- 비지도학습은 레이블 없이 데이터의 구조와 패턴을 발견하는 방법이다.
- 군집화는 비슷한 데이터를 같은 그룹으로 묶으며 K-평균이 대표적 알고리즘이다.
- K-평균은 할당과 중심 갱신을 반복하며 관성 J를 줄이지만 K와 초기값·스케일에 민감하다.
- 엘보우 방법은 관성 감소가 완만해지는 팔꿈치 지점으로 군집 수 K를 정한다.
- 계층적 군집은 K를 미리 정하지 않고 덴드로그램으로 군집 구조를 보여 준다.
- 차원의 저주는 특징 수가 늘수록 데이터가 희소해지고 거리가 무뎠지는 현상이다.
- PCA는 분산을 가장 잘 보존하는 주성분 축으로 데이터를 저차원으로 요약한다.
- 비지도학습은 이상탐지와 지도학습 전처리에 활용되나 결과 해석과 검증이 필요하다.

핵심 용어

비지도학습(Unsupervised Learning)	레이블 없이 데이터의 구조·패턴을 찾는 머신러닝 방식이다.
군집화(Clustering)	비슷한 데이터를 같은 그룹으로 묶는 비지도학습 작업이다.
K-평균(K-means)	K개 중심에 데이터를 배정하고 중심을 갱신하기를 반복하는 군집 알고리즘이다.
중심(Centroid)	한 군집에 속한 데이터들의 평균 위치로 군집을 대표하는 점이다.
관성(Inertia)	각 데이터와 소속 군집 중심 간 거리 제곱의 합으로 군집 응집도를 재는 값이다.
엘보우 방법(Elbow Method)	관성 감소가 완만해지는 지점으로 적절한 군집 수 K를 정하는 방법이다.
실루엣 계수(Silhouette Score)	데이터가 자기 군집에 뭉치고 이웃 군집과 분리된 정도를 재는 지표이다.
계층적 군집(Hierarchical Clustering)	군집을 순차적으로 병합·분할해 계층 구조로 형성하는 군집화이다.
덴드로그램(Dendrogram)	계층적 군집의 병합 과정을 나무 형태로 시각화한 그림이다.
차원의 저주(Curse of Dimensionality)	특징 수가 늘수록 데이터가 희소해지고 분석이 어려워지는 현상이다.
차원 축소(Dimensionality Reduction)	정보를 최대한 보존하며 특징 수를 줄이는 기법이다.
주성분분석(PCA)	분산을 가장 잘 보존하는 축(주성분)으로 데이터를 저차원으로 요약하는 기법이다.
주성분(Principal Component)	데이터의 분산이 큰 방향을 나타내는 새로운 직교 축이다.
이상탐지(Anomaly Detection)	정상 패턴에서 크게 벗어난 데이터를 찾아내는 작업이다.

모델 평가와 일반화

좋은 모델을 만들려면 잘 학습시키는 것만큼이나 성능을 올바르게 측정하는 일이 중요하다. 성능을 잘못 재면 실제로는 형편없는 모델을 좋다고 착각하게 된다. 이 장에서는 일반화 오차를 이해하는 틀인 편향-분산 트레이드오프와 과소·과대적합을 살펴보고, 데이터를 아껴 쓰며 성능을 안정적으로 추정하는 교차검증을 배운다. 이어 분류를 평가하는 혼동행렬·정밀도·재현율·F1·ROC/AUC, 회귀를 평가하는 MAE·RMSE·R²를 정리하고, 하이퍼파라미터 튜닝과 혼한 함정인 데이터 누수 방지까지 다룬다.

9.1 편향-분산 트레이드오프

모델의 일반화 오차는 크게 편향(bias), 분산(variance), 그리고 줄일 수 없는 잡음의 합으로 분해된다. 편향은 모델이 지나치게 단순해서 데이터의 실제 관계를 담지 못해 생기는 체계적 오차이고, 분산은 모델이 훈련 데이터의 사소한 변화에 지나치게 민감하게 반응해 생기는 오차이다. 두 오차의 성격은 서로 반대 방향을 향한다.

편향이 높은 모델은 훈련 데이터조차 잘 맞지 못하는 과소적합 상태로, 데이터를 너무 단순하게 요약한 결과이다. 반대로 분산이 높은 모델은 훈련 데이터는 아주 잘 맞지만 새로운 데이터에서는 성능이 크게 떨어지는 과적합 상태로, 데이터의 잡음까지 외워 버린 결과이다. 이상적인 모델은 편향과 분산이 모두 낮은 지점에 있다.

문제는 편향과 분산을 동시에 줄이기 어렵다는 데 있다. 모델을 복잡하게 만들면 편향은 줄지만 분산이 늘고, 단순하게 만들면 분산은 줄지만 편향이 는다. 이 상충 관계가 편향-분산 트레이드오프이며, 모델 복잡도를 적절히 조절해 둘의 합이 최소가 되는 균형점을 찾는 것이 모델링의 핵심 과제이다. 정규화, 앙상블, 더 많은 데이터 확보 등이 이 균형을 맞추는 도구가 된다.

수식

기대 일반화 오차 = 편향² + 분산 + 줄일 수 없는 잡음

9.2 과소적합·과대적합과 교차검증

과소적합(underfitting)은 모델이 너무 단순해 훈련 데이터의 패턴조차 담지 못하는 상태로, 훈련 성능과 검증 성능이 모두 낮게 나타난다. 반대로 과대적합(overfitting)은 모델이 훈련 데이터에 지나치게 맞춰진 상태로, 훈련 성능은 매우 높지만 검증 성능은 크게 뒤쳐진다. 두 성능의 격차를 관찰하면 모델이 어느 쪽 문제에 있는지 진단할 수 있다.

성능을 한 번의 훈련/검증 분할로만 평가하면, 어떻게 나누느냐에 따라 결과가 우연히 좋거나 나쁘게 나올 수 있다. 이를 완화하는 방법이 k-겹 교차검증(k-fold cross-validation)이다. 데이터를 k개의 겹으로 나눈 뒤, 그중 하나를 검증용으로 두고 나머지로 학습하는 과정을 각 겹이 한 번씩 검증용이 되도록 k번 반복하여, k개 성능의 평균을 최종 추정치로 삼는다.

교차검증은 모든 데이터가 학습과 검증에 골고루 쓰이므로 성능 추정이 안정적이고, 데이터가 적을 때 특히 유용하다. 분류에서 클래스 비율을 각 겹에 동일하게 유지하는 층화(stratified) k-겹, 데이터를 하나씩만 검증에 쓰는 LOOCV 등 여러 변형이 있다. 다만 k번 학습하므로 계산 비용이 그만큼 늘어난다는 점은 감수해야 한다.

예시

5-겹 교차검증에서 각 겹의 정확도가 0.82, 0.85, 0.80, 0.83, 0.84로 나오면, 평균 0.828을 모델의 일반화 성능 추정치로 보고한다.

9.3 혼동행렬과 정밀도·재현율·F1

분류 성능은 정확도(accuracy)만으로는 부족할 때가 많다. 특히 클래스가 불균형한 경우, 예를 들어 99%가 정상이고 1%가 질병인 데이터에서 무조건 '정상'이라고만 예측해도 정확도는 99%에 달하지만 정작 질병은 하나도 못 잡는다. 이런 한계를 드러내기 위해 예측 결과를 참/거짓과 양성/음성의 조합으로 정리한 혼동행렬(confusion matrix)을 사용한다.

혼동행렬은 참양성(TP), 거짓양성(FP), 참음성(TN), 거짓음성(FN)의 네 칸으로 이루어진다. 여기서 정밀도(precision)는 양성이라 예측한 것 중 실제 양성 비율($TP/(TP+FP)$)로, 예측의 정확성을 뜻한다. 재현율(recall)은 실제 양성 중 모델이 찾아낸 비율($TP/(TP+FN)$)로, 놓치지 않는 능력을 뜻한다. 두 지표는 대개 한쪽을 높이면 다른 쪽이 낮아지는 상충 관계에 있다.

정밀도와 재현율을 함께 반영하기 위해 둘의 조화평균인 F1 점수를 쓴다. F1은 두 값이 모두 높을 때만 커지므로, 한쪽만 좋은 편향된 모델을 걸러내는 데 유용하다. 어느 지표를 중시할지는 문제에 달렸다. 암 진단처럼 놓치면 치명적인 경우는 재현율을, 스팸 분류처럼 정상 메일을 잘못 거르면 곤란한 경우는 정밀도를 우선한다.

수식

$$\text{정밀도} = TP / (TP + FP) \quad \text{재현율} = TP / (TP + FN) \quad F1 = 2 \cdot (\text{정밀도} \cdot \text{재현율}) / (\text{정밀도} + \text{재현율}) \quad \text{정확도} = (TP + TN) / (TP + TN + FP + FN)$$

예시

실제 양성 100개 중 모델이 80개를 양성으로 맞히고($TP=80, FN=20$), 양성이라 예측한 것이 총 100개($FP=20$)라면, 정밀도= $80/100=0.8$, 재현율= $80/100=0.8$, $F1=0.8$ 이다.

9.4 ROC 곡선과 AUC

로지스틱 회귀 같은 모델은 클래스가 아니라 확률을 출력하며, 이 확률을 어떤 임계값(threshold)에서 양성으로 판정할지에 따라 정밀도와 재현율이 달라진다. 임계값을 0에서 1까지 바꿔 가며 성능이 어떻게 변하는지를 한눈에 보여 주는 것이 ROC(Receiver Operating Characteristic) 곡선이다.

ROC 곡선은 가로축에 거짓양성률($FPR = FP/(FP+TN)$), 세로축에 참양성률(재현율과 동일, TPR)을 놓고 임계값에 따른 점들을 이은 곡선이다. 곡선이 왼쪽 위 모서리에 가까울수록, 즉 거짓양성은 적으면서 참양성은 많이 잡을수록 좋은 모델이다. 무작위로 찍는 모델은 대각선에

해당한다.

이 곡선 아래의 넓이를 AUC(Area Under the Curve)라 하며, 0.5(무작위)에서 1.0(완벽) 사이의 값을 가진다. AUC는 특정 임계값에 얽매이지 않고 모델의 전반적인 판별 능력을 하나의 수로 요약해 주어 모델 비교에 편리하다. 다만 클래스가 극심하게 불균형한 경우에는 정밀도-재현율 곡선(PR curve)이 더 유용할 수 있다.

수식

거짓양성률 $FPR = FP / (FP + TN)$, 참양성률 $TPR = TP / (TP + FN) = \text{재현율}$

9.5 회귀 평가 지표: MAE·RMSE·R²

회귀 모델의 성능은 예측값이 실제값에서 얼마나 벗어났는지로 평가한다. 평균절대오차(MAE)는 오차 절댓값의 평균으로, 예측이 평균적으로 얼마나 빗나가는지를 원래 단위로 직관적으로 보여 준다. 오차를 제공하지 않으므로 이상치(outlier)에 상대적으로 덜 민감하다는 특징이 있다.

평균제곱근오차(RMSE)는 오차를 제곱해 평균낸 뒤 제곱근을 취한 값으로, MSE와 같은 정보를 원래 단위로 되돌려 해석하기 쉽게 한 것이다. 오차를 제곱하기 때문에 큰 오차에 더 큰 벌점을 주어, 이상치에 민감하다. 큰 실수를 특히 피해야 하는 문제에서는 RMSE를, 모든 오차를 고르게 보고 싶을 때는 MAE를 선호한다.

결정계수 R²(R-squared)는 모델이 데이터의 분산을 얼마나 설명하는지를 0~1 사이의 비율로 나타낸다(음수도 가능). 1에 가까울수록 잘 설명하는 것이고, 0이면 단순히 평균으로 예측하는 것과 다를 바 없으며, 음수이면 평균보다도 못한 것이다. R²는 단위에 무관해 서로 다른 문제 사이의 성능을 비교하기에 편리하다.

수식

$$MAE = (1/n) \cdot \sum |y_i - \hat{y}_i| \quad | \quad RMSE = \sqrt{(1/n) \cdot \sum (y_i - \hat{y}_i)^2} \quad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

예시

실제값이 [10, 20, 30], 예측이 [12, 18, 33]이면 오차는 [2, 2, -3]이다. $MAE = (2+2+3)/3 = 2.33$, $RMSE = \sqrt{((2+4+9)/3)} = \sqrt{17/3} \approx 2.38$ 이다.

9.6 하이퍼파라미터 튜닝과 데이터 누수

하이퍼파라미터(hyperparameter)는 학습률, 정규화 강도 λ , KNN의 K, 트리의 깊이처럼 학습으로 정해지지 않고 사람이 미리 설정하는 값이다. 좋은 하이퍼파라미터를 찾는 과정을 튜닝이라 한다. 그리드 서치(grid search)는 후보 값들의 모든 조합을 빠짐없이 시도해 최선을 찾고, 랜덤 서치(random search)는 조합을 무작위로 표집해 탐색한다.

그리드 서치는 철저하지만 하이퍼파라미터가 많아지면 조합 수가 폭발적으로 늘어 비용이 커진다. 랜덤 서치는 같은 예산으로 더 다양한 값을 시도할 수 있어 종종 더 효율적이며, 중요한 하이퍼파라미터에 탐색을 집중하는 효과가 있다. 최근에는 이전 결과를 활용해 유망한 영역을 탐색하는 베이지안 최적화도 널리 쓰인다. 튜닝은 반드시 테스트 세트가 아닌 검증 세트나

교차검증으로 수행해야 한다.

성능 평가에서 가장 흔하고 치명적인 함정이 데이터 누수(data leakage)이다. 이는 평가 시점에 알 수 없어야 할 정보가 학습 과정에 새어 들어가, 성능이 실제보다 부풀려지는 현상이다. 예컨대 전체 데이터로 스케일링 기준을 계산한 뒤 나누거나, 테스트 데이터를 특징 선택·튜닝에 사용하면 누수가 발생한다. 이를 막으려면 모든 전처리의 기준을 훈련 데이터로만 산출하고 테스트 데이터에 동일하게 적용하며, 미래 정보가 특징에 섞이지 않도록 데이터 분할을 가장 먼저 수행해야 한다.

예시

학습률 {0.01, 0.1}과 트리 깊이 {3, 5, 7}을 그리드 서치하면 $2 \times 3 = 6$ 개 조합을 모두 교차검증으로 평가해 가장 좋은 조합을 고른다.

이 장의 요약

- 일반화 오차는 편향²+분산+잡음으로 분해되며 둘을 동시에 줄이기 어렵다(트레이드오프).
- 과소적합은 훈련·검증 성능이 모두 낮고, 과대적합은 훈련은 높지만 검증이 낮다.
- k-겹 교차검증은 데이터를 k개로 나눠 번갈아 검증하여 성능을 안정적으로 추정한다.
- 혼동행렬로 TP·FP·TN·FN을 정리하며 불균형 데이터에서는 정확도만으로 부족하다.
- 정밀도 = $TP / (TP + FP)$, 재현율 = $TP / (TP + FN)$, F1은 둘의 조화평균이다.
- ROC 곡선과 AUC는 임계값에 무관하게 모델의 판별 능력을 요약한다.
- 회귀는 MAE·RMSE·R²로 평가하며 RMSE는 이상치에, MAE는 덜 민감하다.
- 하이퍼파라미터는 그리드·랜덤 서치로 튜닝하고 데이터 누수를 반드시 경계해야 한다.

핵심 용어

편향-분산 트레이드오프(Bias-Variance Tradeoff)	모델 복잡도에 따라 편향과 분산이 상충하는 관계이다.
편향(Bias)	모델이 지나치게 단순해 실제 관계를 담지 못해 생기는 체계적 오차이다.
분산(Variance)	모델이 훈련 데이터의 변화에 과민하게 반응해 생기는 오차이다.
과소적합(Underfitting)	모델이 너무 단순해 훈련·검증 성능이 모두 낮은 상태이다.
과대적합(Overfitting)	훈련에는 잘 맞지만 새 데이터에서 성능이 떨어지는 상태이다.
k-겹 교차검증(k-fold Cross-Validation)	데이터를 k개로 나눠 번갈아 검증해 성능을 안정적으로 추정하는 방법이다.
혼동행렬(Confusion Matrix)	TP·FP·TN·FN으로 분류 예측 결과를 정리한 표이다.
정밀도(Precision)	양성이라 예측한 것 중 실제 양성의 비율 $TP / (TP + FP)$ 이다.
재현율(Recall)	실제 양성 중 모델이 찾아낸 비율 $TP / (TP + FN)$ 이다.
F1 점수(F1 Score)	정밀도와 재현율의 조화평균으로 둘의 균형을 나타내는 지표이다.

ROC/AUC	임꺽값에 따른 참·거짓양성률 곡선과 그 아래 넓이로 판별 능력을 요약한다.
RMSE(평균제곱근오차)	오차 제곱 평균의 제곱근으로 큰 오차에 민감한 회귀 지표이다.
결정계수(R^2)	모델이 데이터 분산을 설명하는 비율로 1에 가까울수록 좋다.
하이퍼파라미터(Hyperparameter)	학습으로 정해지지 않고 사람이 미리 설정하는 값이다.
데이터 누수(Data Leakage)	평가에 쓰일 정보가 학습에 새어 들어가 성능이 부풀려지는 현상이다.

제3부
딥러닝

신경망의 기초와 역전파

딥러닝은 여러 층으로 쌓인 인공신경망을 대량의 데이터로 학습시켜 복잡한 패턴을 표현하는 기계학습의 한 갈래이다. 이 장에서는 신경망의 가장 작은 단위인 퍼셉트론에서 출발하여, 여러 층을 쌓은 다층 퍼셉트론이 어떻게 비선형 문제를 풀 수 있는지 살펴본다. 이어서 입력에서 출력으로 신호가 전달되는 순전파, 예측과 정답의 차이를 재는 손실함수, 그리고 그 오차를 거꾸로 흘려보내며 가중치를 갱신하는 역전파 알고리즘을 수식과 의사코드로 차근차근 설명한다. 이 장을 마치면 신경망이 '학습한다'는 말의 구체적 의미를 이해하게 될 것이다.

10.1 퍼셉트론에서 다층 신경망으로

퍼셉트론(perceptron)은 1957년 프랭크 로젠블랫이 제안한 초기 인공뉴런 모형으로, 여러 입력값에 각각 가중치를 곱해 더한 뒤 임계값과 비교하여 0 또는 1을 출력한다. 즉 입력 $x_1, x_2, \dots, x_n$ 에 대해 가중합 $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ 를 계산하고, 이 값이 0보다 크면 1을, 그렇지 않으면 0을 내보낸다. 여기서 w 는 각 입력의 중요도를 나타내는 가중치이고 b 는 편향(bias)으로 뉴런이 활성화되는 문턱을 조절한다. 이 단순한 구조만으로도 AND, OR 같은 선형 분리가 가능한 논리 연산을 학습할 수 있어 큰 기대를 모았다.

그러나 단층 퍼셉트론에는 치명적 한계가 있었다. 1969년 민스키와 페퍼트는 XOR(배타적 논리합) 문제를 하나의 퍼셉트론으로는 결코 풀 수 없음을 수학적으로 보였다. XOR은 두 입력이 서로 다를 때만 1을 출력하는데, 이는 하나의 직선으로 두 부류를 나눌 수 없는 '선형 분리 불가능' 문제이기 때문이다. 이 지적은 신경망 연구를 한동안 침체시킨 이른바 첫 번째 인공지능 겨울의 배경이 되었다.

해결의 열쇠는 뉴런을 여러 층으로 쌓는 데 있었다. 입력층과 출력층 사이에 하나 이상의 은닉층(hidden layer)을 두어 만든 구조를 다층 퍼셉트론(Multi-Layer Perceptron, MLP)이라 부른다. 은닉층은 입력을 새로운 특징 공간으로 변환하여, 원래 공간에서는 선형 분리가 불가능하던 문제도 변환된 공간에서는 선형 분리가 가능하도록 만든다. 실제로 은닉층 하나만 추가해도 XOR 문제를 완벽히 풀 수 있으며, 이론적으로 충분히 큰 은닉층을 가진 MLP는 임의의 연속함수를 원하는 정밀도로 근사할 수 있다(보편 근사 정리).

수식

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \sum w x + b$$

예시

XOR 예시: 입력 (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0. 단층 퍼셉트론은 이 네 점을 하나의 직선으로 나눌 수 없다. 그러나 은닉 뉴런 두 개를 두어 각각 OR과 NAND를 계산하고 그 결과를 AND로 결합하면 XOR을 구현할 수 있다.

10.2 활성화 함수

가중합 z 를 그대로 다음 층에 전달하면, 아무리 층을 많이 쌓아도 전체는 하나의 선형 변환에 지나지 않는다. 선형 함수의 합성은 여전히 선형이기 때문이다. 따라서 각 뉴런의 출력에 비선형 활성화 함수(activation function)를 적용해야 신경망이 비선형 관계를 표현할 수 있다. 활성화 함수는 뉴런이 '얼마나 활성화될지'를 결정하는 문지기 역할을 한다.

대표적인 활성화 함수로 시그모이드(sigmoid)는 $\sigma(z) = 1 / (1 + e^{-z})$ 로 정의되며 출력을 0과 1 사이로 눌러 확률처럼 해석할 수 있게 한다. 하이퍼볼릭 탄젠트(tanh)는 출력 범위가 -1에서 1이며 0을 중심으로 대칭이어서 시그모이드보다 학습이 안정적인 경향이 있다. 다만 두 함수 모두 입력의 절댓값이 커지면 기울기가 0에 가까워지는 포화(saturation) 현상이 생겨, 층이 깊어질수록 기울기가 사라지는 문제를 일으킨다.

이 문제를 완화한 것이 ReLU(Rectified Linear Unit)로 $\text{ReLU}(z) = \max(0, z)$ 라는 매우 단순한 형태다. z 가 양수이면 그대로 통과시키고 음수이면 0으로 만든다. 계산이 빠르고 양수 구간에서 기울기가 항상 1이라 기울기 소실이 덜해, 오늘날 은닉층의 기본 활성화 함수로 널리 쓰인다. 다만 음수 입력에서 기울기가 0이 되어 뉴런이 영구히 죽는 '죽은 ReLU' 문제가 있어, 음수 구간에 작은 기울기를 주는 Leaky ReLU 등의 변형도 사용된다.

수식

$$\sigma(z) = 1 / (1 + e^{-z}), \tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z}), \text{ReLU}(z) = \max(0, z)$$

10.3 순전파와 출력층

순전파(forward propagation)는 입력 데이터가 신경망의 층을 차례로 통과하며 최종 출력을 만들어내는 과정이다. 각 층에서는 먼저 앞 층의 출력 a 에 가중치 행렬 W 를 곱하고 편향 b 를 더해 가중합 z 를 구한 뒤, 활성화 함수 f 를 적용해 그 층의 출력 a 를 얻는다. 이를 층 번호 l 에 대해 쓰면 $z^l = W^l a^{l-1} + b^l$, $a^l = f(z^l)$ 이며, 첫 층의 입력 a 는 데이터 x 자체다. 이렇게 층에서 층으로 계산을 이어가면 마지막 출력층에서 예측값이 나온다.

출력층의 형태는 풀려는 문제에 따라 달라진다. 회귀 문제에서는 활성화 없이 실숫값을 그대로 내보내고, 이진 분류에서는 시그모이드로 하나의 확률을 출력한다. 다중 클래스 분류에서는 소프트맥스(softmax) 함수를 사용한다. 소프트맥스는 각 클래스 점수 z 를 지수함수로 변환한 뒤 전체 합으로 나누어, 모든 출력이 0과 1 사이이며 그 합이 정확히 1이 되는 확률 분포를 만든다.

소프트맥스와 짝을 이루는 손실함수는 교차 엔트로피(cross-entropy)다. 정답 클래스에 해당하는 예측 확률이 1에 가까울수록 손실이 작아지고, 0에 가까우면 손실이 크게 벌어진다. 회귀에서는 예측과 정답의 차이를 제공해 평균 제곱 오차(MSE)를 흔히 쓴다. 손실함수는 신경망이 '얼마나 틀렸는가'를 하나의 숫자로 요약하며, 학습의 목표는 이 값을 최소화하는 가중치를 찾는 것이다.

수식

$$\text{softmax}(z) = e^{z_j} / \sum_k e^{z_k}, \text{교차엔트로피 } L = - \sum y \log(\hat{y})$$

의사코드

```
# 순전파 의사코드 (L개 층)
a = x # 입력, a
for l in range(1, L+1):
    z = W[l] @ a + b[l] # 가중합
    a = f(z) # 활성화 (은닉층: ReLU 등)
y_hat = softmax(z) # 출력층: 확률 분포
loss = cross_entropy(y_hat, y_true)
```

10.4 역전파와 연쇄법칙

신경망 학습의 핵심은 손실 L 을 각 가중치로 편미분한 기울기 $\partial L / \partial w$ 를 구해, 손실이 줄어드는 방향으로 가중치를 조금씩 옮기는 것이다. 문제는 수백만 개의 가중치 각각에 대해 기울기를 일일이 계산하면 엄청난 비용이 든다는 점이다. 역전파(backpropagation)는 미적분의 연쇄법칙(chain rule)을 영리하게 활용해 이 기울기들을 출력층에서 입력층 방향으로 단 한 번의 역방향 순회로 효율적으로 계산하는 알고리즘이다.

연쇄법칙이란 합성함수의 미분을 각 단계 미분의 곱으로 분해하는 규칙이다. 손실은 출력 a 에, 출력은 가중합 z 에, z 는 가중치 w 에 차례로 의존하므로 $\partial L / \partial w = (\partial L / \partial a)(\partial a / \partial z)(\partial z / \partial w)$ 로 쪼갤 수 있다. 역전파는 각 층에서 '오차 신호' $\delta = \partial L / \partial z$ 를 정의하고, 출력층의 δ 를 먼저 구한 뒤 이를 가중치 행렬을 통해 앞 층으로 거꾸로 전파한다. 구체적으로 은닉층의 오차는 $\delta = (W^{(1)} \delta^{(1)}) \odot f'(z)$ 로 계산되며, $\odot$ 는 원소별 곱을 뜻한다.

각 층의 오차 δ 를 알면 그 층의 가중치와 편향에 대한 기울기는 즉시 얻어진다. 가중치 기울기는 $\partial L / \partial W = \delta (a^{(1)})^T$ 이고 편향 기울기는 $\partial L / \partial b = \delta$ 이다. 즉 순전파 때 저장해 둔 각 층의 활성화값 a 와 역방향으로 전파된 오차 δ 만 있으면 모든 파라미터의 기울기를 곱셈 몇 번으로 구할 수 있다. 이 효율성 덕분에 층이 깊은 신경망도 현실적인 시간 안에 학습할 수 있게 되었다.

수식

$$\delta = \partial L / \partial z \quad (\text{출력층}), \delta = (W^{(1)} \delta^{(1)}) \odot f'(z), \partial L / \partial W = \delta (a^{(1)})^T, \partial L / \partial b = \delta$$

의사코드

```
# 역전파 의사코드
# 순전파에서 각 층의 z[l], a[l]를 저장해 둬
delta = dL_dz_output(y_hat, y_true) # 출력층 오차 δ
for l in range(L, 0, -1):
    grad_W[l] = delta @ a[l-1].T # ∂L/∂W
    grad_b[l] = delta # ∂L/∂b
    if l > 1:
        delta = (W[l].T @ delta) * f_prime(z[l-1]) # 앞 층으로 전파
```

10.5 경사하강법과 학습

역전파로 기울기를 얻고 나면, 경사하강법(gradient descent)으로 가중치를 갱신한다. 기본 규칙은 각 가중치를 기울기의 반대 방향으로 학습률 η 만큼 이동시키는 것으로, $w \leftarrow w - \eta \cdot$

$\partial L / \partial w$ 로 표현된다. 학습률은 한 걸음의 크기를 정하는 중요한 하이퍼파라미터로, 너무 크면 최소값을 지나쳐 발산하고 너무 작으면 학습이 지나치게 느려진다.

전체 데이터를 한 번에 사용해 기울기를 계산하는 배치 경사하강법은 정확하지만 데이터가 크면 한 걸음마다 비용이 크다. 반대로 한 개 샘플만 쓰는 확률적 경사하강법(SGD)은 빠르지만 갱신이 요동친다. 실제로는 그 절충안인 미니배치 경사하강법을 가장 널리 사용하는데, 수십에서 수백 개의 샘플을 묶어 기울기를 추정함으로써 계산 효율과 안정성을 함께 얻는다.

학습은 여러 에폭(epoch)에 걸쳐 반복된다. 한 에폭은 훈련 데이터 전체를 한 번 훑는 것을 뜻하며, 각 에폭에서 데이터를 미니배치로 나누어 순전파·역전파·가중치 갱신을 되풀이한다. 손실이 충분히 낮아지고 검증 데이터에서의 성능이 더 이상 나아지지 않으면 학습을 멈춘다. 이 전체 순환, 즉 순전파로 예측하고 손실을 재고 역전파로 기울기를 구해 가중치를 조정하는 반복이 바로 신경망이 '학습한다'는 말의 구체적 내용이다.

수식

$$w \leftarrow w - \eta \cdot \partial L / \partial w, b \leftarrow b - \eta \cdot \partial L / \partial b$$

의사코드

```
# 학습 루프 의사코드 (미니배치 SGD)
for epoch in range(num_epochs):
    for batch in iterate_minibatches(data, batch_size):
        y_hat = forward(batch.x)      # 순전파
        loss = loss_fn(y_hat, batch.y)
        grads = backward(loss)        # 역전파
        for p in parameters:
            p -= lr * grads[p]        # 경사하강 갱신
```

이 장의 요약

- 퍼셉트론은 가중합에 임계값을 적용해 0/1을 출력하는 최초의 인공뉴런이지만, XOR 같은 선형 분리 불가능 문제는 풀 수 없다.
- 은닉층을 추가한 다층 퍼셉트론(MLP)은 입력을 새 특징 공간으로 변환해 비선형 문제를 풀며, 충분한 크기면 임의의 연속함수를 근사한다(보편 근사 정리).
- 활성화 함수는 신경망에 비선형성을 부여한다. 시그모이드·tanh는 포화 문제가 있고, ReLU(max(0,z))는 계산이 빠르고 기울기 소실이 덜해 은닉층 표준으로 쓰인다.
- 순전파는 $z=Wa+b$, $a=f(z)$ 를 층마다 반복해 출력을 계산하며, 출력층은 문제에 따라 항등(회귀)·시그모이드(이진)·소프트맥스(다중 분류)를 쓴다.
- 손실함수는 예측과 정답의 차이를 한 숫자로 요약한다. 분류에는 교차 엔트로피, 회귀에는 평균제곱오차(MSE)가 대표적이다.
- 역전파는 연쇄법칙으로 오차 신호 δ 를 출력층에서 입력층으로 전파해, 모든 가중치의 기울기를 한 번의 역방향 순회로 효율적으로 계산한다.
- 경사하강법은 $w \leftarrow w - \eta \cdot \partial L / \partial w$ 규칙으로 손실을 줄이는 방향으로 가중치를 갱신하며, 학습률 η 가 걸음 크기를 결정한다.
- 실무에서는 미니배치 경사하강을 여러 에폭 반복하며, 순전파→손실→역전파→갱신의 순환이 신경망 학습의 본질이다.

핵심 용어

퍼셉트론(Perceptron)	입력의 가중합에 임계값을 적용해 0 또는 1을 출력하는 초기 인공뉴런 모형.
다층 퍼셉트론(Multi-Layer Perceptron, MLP)	입력층과 출력층 사이에 하나 이상의 은닉층을 둔 완전 연결 신경망.
은닉층(Hidden Layer)	입력과 출력 사이에서 특징을 변환하는 중간 층으로, 비선형 표현력을 제공한다.
활성화 함수(Activation Function)	뉴런 출력에 비선형성을 부여하는 함수로 시그모이드·tanh·ReLU 등이 있다.
ReLU(Rectified Linear Unit)	$\max(0, z)$ 형태의 활성화 함수로 계산이 빠르고 기울기 소실이 덜하다.
소프트맥스(Softmax)	여러 클래스 점수를 합이 1인 확률 분포로 변환하는 다중 분류 출력 함수.
순전파(Forward Propagation)	입력이 층을 차례로 통과하며 출력을 계산하는 과정.
손실함수(Loss Function)	예측과 정답의 차이를 하나의 값으로 요약하는 함수로 교차 엔트로피·MSE 등.
교차 엔트로피(Cross-Entropy)	예측 확률 분포와 정답 분포의 차이를 재는 분류용 손실함수.
역전파(Backpropagation)	연쇄법칙으로 오차를 역방향 전파해 모든 가중치의 기울기를 효율적으로 구하는 알고리즘.
연쇄법칙(Chain Rule)	합성함수의 미분을 각 단계 미분의 곱으로 분해하는 미적분 규칙.
경사하강법(Gradient Descent)	기울기의 반대 방향으로 파라미터를 갱신해 손실을 최소화하는 최적화 기법.
학습률(Learning Rate)	경사하강에서 한 번의 갱신 크기를 정하는 하이퍼파라미터.
에폭(EPOCH)	훈련 데이터 전체를 한 번 모두 사용해 학습을 진행하는 단위.

심층 신경망 학습 기법

층을 깊게 쌓으면 신경망의 표현력은 커지지만, 실제로 깊은 망을 안정적으로 학습시키는 일은 오랫동안 쉽지 않았다. 기울기가 사라지거나 폭발하고, 학습이 느리며, 훈련 데이터에는 잘 맞지만 새 데이터에는 약한 과대적합이 자주 발생했기 때문이다. 이 장에서는 이러한 문제들을 극복하기 위해 개발된 핵심 기법들, 즉 가중치 초기화, 배치 정규화, 드롭아웃, 현대적 옵티마이저, 학습률 스케줄링, 규제와 데이터 증강을 다룬다. 이 도구들은 오늘날 거의 모든 딥러닝 모델 학습에 표준적으로 쓰인다.

11.1 기울기 소실과 폭주, 가중치 초기화

역전파는 오차를 층마다 곱셈으로 전파하기 때문에, 층이 깊어지면 기울기가 연쇄적으로 곱해지며 지수적으로 작아지거나 커질 수 있다. 각 층의 기울기가 1보다 작은 값끼리 계속 곱해지면 입력층에 가까운 층의 기울기는 거의 0이 되는 기울기 소실(vanishing gradient)이 일어나 앞쪽 층이 사실상 학습되지 않는다. 반대로 1보다 큰 값이 곱해지면 기울기가 폭발적으로 커지는 기울기 폭주(exploding gradient)가 생겨 학습이 발산한다.

이 문제는 활성화 함수 선택뿐 아니라 가중치를 처음에 어떻게 설정하느냐에도 크게 좌우된다. 가중치를 모두 0으로 두면 모든 뉴런이 같은 값을 계산해 학습이 되지 않고, 지나치게 크거나 작은 무작위 값으로 두면 신호나 기울기의 분산이 층을 지나며 커지거나 줄어든다. 좋은 초기화의 목표는 순전파의 활성화값과 역전파의 기울기 분산이 층을 통과해도 대체로 유지되도록 하는 것이다.

Xavier(글로트) 초기화는 시그모이드·tanh 계열에 맞춘 방법으로, 입력 뉴런 수와 출력 뉴런 수를 함께 고려해 가중치의 분산을 정한다. He(허) 초기화는 ReLU 계열에 맞춰 설계된 방법으로, ReLU가 음수 절반을 0으로 만들어 분산을 절반으로 줄이는 점을 보정해 분산을 $2/n$ (입력 수)로 잡는다. 활성화 함수에 맞는 초기화를 쓰는 것만으로도 깊은 망의 학습 안정성이 크게 향상된다.

수식

Xavier: $\text{Var}(w) = 1 / n_{\text{in}}$ (또는 $2/(n_{\text{in}}+n_{\text{out}})$), He: $\text{Var}(w) = 2 / n_{\text{in}}$

11.2 배치 정규화와 드롭아웃

배치 정규화(Batch Normalization)는 각 층에 들어가는 입력 분포가 학습 도중 계속 변하는 문제를 완화하기 위해, 미니배치 단위로 각 특징의 평균을 0, 분산을 1로 정규화한 뒤 학습 가능한 스케일 γ 와 이동 β 로 다시 조정한다. 이렇게 하면 층마다 입력 분포가 안정되어 더 큰 학습률을 쓸 수 있고 수렴이 빨라지며, 초기화에 대한 민감도도 줄어든다. 또한 미니배치마다 통계가 조금씩 달라 약한 규제 효과도 부수적으로 얻는다.

배치 정규화는 훈련과 추론에서 동작이 다르다는 점에 주의해야 한다. 훈련 중에는 현재 미니배치의 평균과 분산을 쓰지만, 추론 시에는 배치가 없거나 하나뿐일 수 있으므로 훈련 동안 누적해 둔 이동 평균 통계를 사용한다. 이 차이를 잊으면 추론 성능이 크게 떨어질 수 있다.

드롭아웃(Dropout)은 과대적합을 막는 강력한 규제 기법이다. 훈련의 매 반복마다 은닉 뉴런을 확률 p 로 무작위로 꺼서 0으로 만들어, 특정 뉴런들의 우연한 공모에 의존하지 않는 견고한 표현을 학습하게 한다. 이는 마치 매번 조금씩 다른 여러 부분 신경망을 앙상블하는 효과를 낸다. 추론 시에는 뉴런을 끄지 않고 모두 사용하되, 훈련 때 꺼진 비율을 보정하기 위해 출력을 조정한다(흔히 훈련 중 남은 뉴런 값을 $1/(1-p)$ 로 키우는 역드롭아웃 방식을 쓴다).

수식

$$\text{BN: } x = (x - \mu_{\text{batch}}) / \sqrt{(\sigma^2_{\text{batch}} + \epsilon)}, y = \gamma \cdot x + \beta$$

11.3 옵티마이저: 모멘텀에서 Adam까지

기본 SGD는 손실 표면이 좁고 긴 골짜기 모양일 때 지그재그로 진동하며 느리게 수렴한다. 모멘텀(Momentum)은 물리의 관성 개념을 빌려, 과거 기울기들을 지속적으로 누적한 속도 벡터 v 를 유지하고 그 방향으로 이동한다. 일관된 방향의 기울기는 가속되고 진동하는 성분은 상쇄되어, 골짜기를 따라 더 빠르고 매끄럽게 내려간다.

RMSProp은 각 파라미터마다 학습률을 자동으로 조절하는 적응적 방법이다. 최근 기울기 제공의 이동 평균을 유지하고, 그 제곱근으로 학습률을 나눈다. 그 결과 기울기가 큰 방향은 걸음을 줄이고 기울기가 작은 방향은 걸음을 키워, 파라미터마다 적절한 크기로 갱신한다. 이는 특히 기울기 크기가 방향마다 크게 다를 때 유리하다.

Adam(Adaptive Moment Estimation)은 모멘텀과 RMSProp의 장점을 결합한 오늘날 가장 널리 쓰이는 옵티마이저다. 기울기의 1차 모멘트(평균) m 와 2차 모멘트(제곱 평균) v 를 각각 지수 이동 평균으로 추정하고, 학습 초기에 이 값들이 0으로 편향되는 것을 보정한 뒤, 갱신량을 $m / (\sqrt{v} + \epsilon)$ 로 정한다. 방향은 모멘텀으로 안정시키고 크기는 파라미터별로 적응시키므로, 하이퍼파라미터 조정이 적어도 대체로 잘 작동한다.

수식

$$\text{모멘텀: } v = \beta v_{-1} + \partial L / \partial w, w \leftarrow w - \eta \cdot v \quad | \quad \text{Adam: } m = \beta_1 m_{-1} + (1 - \beta_1)g, v = \beta_2 v_{-1} + (1 - \beta_2)g^2, w \leftarrow w - \eta \cdot m / (\sqrt{v} + \epsilon)$$

의사코드

```
# Adam 갱신 의사코드 (파라미터 w, 기울기 g)
m = beta1*m + (1-beta1)*g      # 1차 모멘트
v = beta2*v + (1-beta2)*(g*g)  # 2차 모멘트
m_hat = m / (1 - beta1**t)     # 편향 보정
v_hat = v / (1 - beta2**t)
w = w - lr * m_hat / (sqrt(v_hat) + eps)
```

11.4 학습률 스케줄링과 규제

고정된 학습률보다 학습이 진행됨에 따라 학습률을 조절하면 더 나은 결과를 얻는 경우가 많다. 이를 학습률 스케줄링이라 한다. 초반에는 큰 학습률로 빠르게 내려가고 후반에는 작은 학습률로 세밀하게 최적점에 다가가는 것이 기본 아이디어다. 대표적으로 일정 에폭마다 학습률을 계단식으로 줄이는 단계 감쇠, 코사인 곡선을 따라 부드럽게 줄이는 코사인 감쇠, 학습

초반에 학습률을 서서히 키우는 워밍업 등이 있다.

규제(regularization)는 모델이 훈련 데이터에 지나치게 맞춰지는 과대적합을 억제하는 기법의 총칭이다. 가중치 감쇠(weight decay), 즉 L2 규제는 손실함수에 가중치 제곱합에 비례하는 항 $(\lambda/2) \cdot \sum w^2$ 을 더해, 가중치가 불필요하게 커지는 것을 별한다. 그 결과 모델이 더 단순하고 매끄러운 함수를 선호하게 되어 일반화 성능이 좋아진다.

조기 종료(early stopping)는 가장 단순하면서도 효과적인 규제다. 학습 중 검증 데이터의 손실을 지켜보다가 더 이상 개선되지 않고 오르기 시작하면, 즉 과대적합이 시작되면 학습을 멈추고 그때까지 가장 좋았던 모델을 채택한다. 이렇게 하면 훈련 손실은 더 낮출 수 있어도 검증 성능이 최고인 지점에서 학습을 마무리할 수 있다.

수식

$$\text{L2 규제 손실: } L_{\text{total}} = L_{\text{data}} + (\lambda/2) \cdot \sum w^2$$

11.5 데이터 증강

딥러닝 모델은 데이터가 많을수록 잘 일반화되지만, 실제로 라벨링된 데이터를 확보하는 데는 큰 비용이 든다. 데이터 증강(data augmentation)은 기존 데이터에 의미를 바꾸지 않는 변형을 가해 인위적으로 데이터를 늘리는 기법이다. 예컨대 고양이 사진을 좌우로 뒤집거나 조금 회전해도 여전히 고양이이므로, 이런 변형본을 추가로 학습시키면 모델이 다양한 조건에 견고해진다.

이미지에서는 좌우 반전, 임의 자르기, 회전, 이동, 확대·축소, 밝기와 대비 조정, 노이즈 추가 등이 흔히 쓰인다. 최근에는 두 이미지를 섞는 믹스업(mixup)이나 한 이미지의 일부를 잘라 다른 이미지에 붙이는 컷믹스(CutMix) 같은 고급 기법도 널리 사용된다. 텍스트나 음성 등 다른 종류의 데이터에도 각 영역에 맞는 증강 방법이 있다.

데이터 증강은 훈련 데이터의 다양성을 키워 과대적합을 줄이는 강력한 규제 역할을 한다. 다만 변형이 데이터의 본래 의미를 훼손하지 않도록 주의해야 한다. 예를 들어 손글씨 숫자 6과 9는 상하로 뒤집으면 서로 바뀌므로 수직 반전을 함부로 적용하면 라벨이 잘못되어 오히려 학습을 방해한다. 증강은 문제의 성격을 고려해 신중히 선택해야 한다.

예시

MNIST 손글씨 분류에서 좌우 소폭 회전과 임의 이동은 유효한 증강이지만, 상하 반전은 숫자의 정체성을 바꾸므로 부적절하다. 반면 자연 사물 사진 분류에서는 좌우 반전이 대개 안전하고 효과적이다.

이 장의 요약

- 깊은 신경망에서는 기울기가 층을 거치며 지수적으로 작아지거나 커지는 기울기 소실·폭주 문제가 학습을 방해한다.
- 가중치 초기화가 학습 안정성을 좌우하며, 시그모이드·tanh에는 Xavier, ReLU에는 He 초기화가 신호와 기울기의 분산을 보존한다.
- 배치 정규화는 미니배치 단위로 입력을 정규화해 수렴을 가속하고 초기화 민감도를 낮추며, 훈련과 추론에서 통계 사용 방식이 다르다.
- 드롭아웃은 훈련 중 뉴런을 무작위로 꺼서 견고한 표현을 학습하게 하는 앙상블 효과의 규제 기법이다.
- 모멘텀은 관성으로 진동을 줄이고, RMSProp은 파라미터별 학습률을 적응시키며, Adam은 둘을 결합해 가장 널리 쓰인다.
- 학습률 스케줄링(단계·코사인 감쇠, 워밍업)은 학습 단계에 맞춰 걸음 크기를 조절해 수렴 품질을 높인다.
- L2 규제(가중치 감쇠)는 큰 가중치를 벌해 모델을 단순하게 만들고, 조기 종료는 검증 손실이 오르기 전에 학습을 멈춰 과대적합을 막는다.
- 데이터 증강은 의미를 보존하는 변형으로 데이터를 늘려 일반화를 돕지만, 변형이 라벨을 훼손하지 않도록 주의해야 한다.

핵심 용어

기울기 소실(Vanishing Gradient)	역전파 시 기울기가 층을 거치며 0에 가까워져 앞쪽 층이 학습되지 않는 문제.
기울기 폭주(Exploding Gradient)	기울기가 층을 거치며 지수적으로 커져 학습이 발산하는 문제.
Xavier 초기화(Glorot Initialization)	입력·출력 뉴런 수를 고려해 분산을 정하는 시그모이드·tanh용 가중치 초기화.
He 초기화(He Initialization)	ReLU가 분산을 절반으로 줄이는 점을 보정해 분산을 $2/n$ 으로 잡는 초기화.
배치 정규화(Batch Normalization)	미니배치 단위로 입력을 평균 0·분산 1로 정규화한 뒤 $\gamma \cdot \beta$ 로 재조정하는 기법.
드롭아웃(Dropout)	훈련 중 뉴런을 확률 p 로 무작위로 꺼서 과대적합을 억제하는 규제 기법.
모멘텀(Momentum)	과거 기울기를 누적한 속도로 이동해 진동을 줄이고 수렴을 가속하는 방법.
RMSProp	기울기 제곱의 이동 평균으로 파라미터별 학습률을 적응적으로 조절하는 옵티마이저.
Adam(Adaptive Moment Estimation)	1·2차 모멘트 추정을 결합해 방향과 크기를 함께 적응시키는 대표적 옵티마이저.
학습률 스케줄링(Learning Rate Scheduling)	학습 진행에 따라 학습률을 조절해 수렴 품질을 높이는 전략.

가중치 감쇠(Weight Decay, L2 규제)	손실에 가중치 제곱합 항을 더해 큰 가중치를 벌하는 규제 기법.
조기 종료(Early Stopping)	검증 손실이 개선되지 않으면 학습을 멈춰 과대적합을 막는 규제 기법.
데이터 증강(Data Augmentation)	의미를 보존하는 변형으로 훈련 데이터를 인위적으로 늘려 일반화를 돕는 기법.
과대적합(Overfitting)	훈련 데이터에는 잘 맞지만 새 데이터에는 성능이 떨어지는 현상.

제12장

합성곱 신경망(CNN)과 컴퓨터 비전

이미지는 픽셀 수가 매우 많고 인접 픽셀 사이에 강한 공간적 상관관계가 있어, 모든 픽셀을 완전 연결로 잇는 방식은 파라미터가 폭발하고 위치 변화에 취약하다. 합성곱 신경망(Convolutional Neural Network, CNN)은 작은 필터를 이미지 전체에 미끄러뜨리며 지역적 특징을 추출하고 가중치를 공유함으로써 이 문제를 우아하게 해결한다. 이 장에서는 합성곱 연산과 필터, 스트라이드와 패딩, 풀링 같은 핵심 구성 요소를 살펴보고, LeNet에서 ResNet에 이르는 대표 아키텍처의 발전 흐름과 전이학습, 그리고 분류·검출·분할이라는 주요 비전 과제를 개념적으로 정리한다.

12.1 합성곱 연산과 특징맵

합성곱(convolution)은 작은 크기의 가중치 격자인 필터(또는 커널)를 입력 이미지 위에서 한 칸씩 이동시키며, 겹치는 영역의 픽셀과 필터 값을 원소별로 곱해 모두 더하는 연산이다. 이 곱셈-덧셈의 결과가 해당 위치의 출력값이 되고, 필터를 이미지 전체에 적용해 얻은 2차원 출력을 특징맵(feature map)이라 부른다. 하나의 필터는 이미지 전 영역에서 동일한 가중치를 사용하므로, 이를 가중치 공유(weight sharing)라 한다.

가중치 공유는 두 가지 큰 이점을 준다. 첫째, 완전 연결에 비해 파라미터 수가 극적으로 줄어 학습이 효율적이고 과대적합에 강하다. 둘째, 같은 필터가 이미지 어디에서든 같은 패턴을 찾으므로, 물체가 이미지의 어느 위치에 있든 그 특징을 감지하는 위치 불변성(translation invariance)을 얻는다. 이는 물체의 위치가 제각각인 실제 이미지 인식에 특히 유용하다.

각 필터는 특정한 시각적 패턴에 반응하도록 학습된다. 얇은 층의 필터는 수직·수평 에지나 색 변화 같은 단순한 저수준 특징을 잡아내고, 층이 깊어질수록 여러 저수준 특징이 결합되어 질감, 부분 모양, 나아가 눈이나 바퀴 같은 고수준 개념에 반응하게 된다. 실제 합성곱 층은 수십에서 수백 개의 필터를 병렬로 두어, 한 위치에서 여러 종류의 특징을 동시에 추출한다.

예시

3×3 소벨 필터처럼 특정 가중치 배열은 에지를 강조한다. CNN에서는 이런 필터의 값을 사람이 정하지 않고 역전파로 데이터에 맞게 학습한다는 점이 핵심이다.

12.2 스트라이드, 패딩, 출력 크기

스트라이드(stride)는 필터를 한 번에 몇 칸씩 이동시킬지를 정하는 값이다. 스트라이드가 1이면 한 칸씩 촘촘히 이동하고, 2이면 두 칸씩 건너뛰어 출력 특징맵의 크기가 대략 절반으로 줄어든다. 스트라이드를 키우면 계산량과 출력 크기를 줄여 공간 해상도를 낮추는 효과가 있다.

패딩(padding)은 입력 가장자리에 값을 덧대는 것으로, 보통 0을 채우는 제로 패딩을 쓴다. 패딩이 없으면 합성곱을 거칠 때마다 출력이 조금씩 작아지고 가장자리 픽셀이 중앙보다 덜 반영되는 문제가 있다. 필터 크기에 맞춰 적절히 패딩하면 출력 크기를 입력과 같게 유지할 수 있는데, 이를 '세임(same) 패딩'이라 부른다.

이 세 요소가 출력 크기를 어떻게 결정하는지는 하나의 공식으로 정리된다. 입력 한 변의 크기 W , 필터 크기 F , 패딩 P , 스트라이드 S 에 대해 출력 한 변의 크기는 $(W - F + 2P)/S + 1$ 이다. 예를 들어 입력 32, 필터 5, 패딩 0, 스트라이드 1이면 $(32 - 5 + 0)/1 + 1 = 28$ 이 된다. 이 공식은 CNN 구조를 설계할 때 각 층의 출력 크기를 정확히 계산하는 기본 도구이므로 반드시 익혀 두어야 한다.

수식

출력 크기 = $(W - F + 2P) / S + 1$ (W : 입력 크기, F : 필터 크기, P : 패딩, S : 스트라이드)

예시

입력 28×28 , 필터 3×3 , 패딩 1, 스트라이드 1 $\rightarrow (28 - 3 + 2)/1 + 1 = 28$ (세임 패딩). 같은 입력에 스트라이드 2를 주면 $(28 - 3 + 2)/2 + 1 = 14$ 로 절반이 된다.

12.3 풀링과 전형적 CNN 구조

풀링(pooling)은 특징맵의 공간 크기를 줄여 계산량을 낮추고 위치 변화에 대한 견고성을 높이는 다운샘플링 연산이다. 가장 널리 쓰이는 최대 풀링(max pooling)은 예컨대 2×2 영역에서 가장 큰 값 하나만 남긴다. 이렇게 하면 그 영역에서 가장 강하게 반응한 특징만 보존되어, 물체가 약간 이동하거나 변형되어도 출력이 크게 흔들리지 않는다. 평균 풀링은 영역의 평균을 취하는 변형이다.

전형적인 CNN은 합성곱 층과 풀링 층을 번갈아 여러 번 쌓아 점점 더 추상적인 특징을 추출한 뒤, 마지막에 완전 연결 층을 붙여 분류를 수행하는 구조를 가진다. 앞부분의 합성곱-풀링 계층은 이미지에서 유용한 특징을 뽑아내는 특징 추출기 역할을 하고, 뒷부분의 완전 연결 층과 소프트맥스는 그 특징을 바탕으로 클래스 확률을 계산하는 분류기 역할을 한다.

공간 해상도는 층을 지날수록 줄어드는 대신, 필터 개수, 즉 채널 수는 대개 늘어난다. 이는 '어디에 있는가'라는 세밀한 위치 정보를 조금씩 버리는 대신 '무엇이 있는가'라는 의미 정보를 점점 풍부하게 담아 가는 과정으로 이해할 수 있다. 최종적으로 작지만 채널이 많은 특징맵이 물체의 정체를 판단하는 근거가 된다.

의사코드

```
# 전형적 CNN 구조 의사코드
x = Conv(filters=32, size=3) -> ReLU
x = Conv(filters=32, size=3) -> ReLU -> MaxPool(2)
x = Conv(filters=64, size=3) -> ReLU -> MaxPool(2)
x = Flatten(x)
x = Dense(128) -> ReLU -> Dropout
y = Dense(num_classes) -> Softmax
```

12.4 대표 아키텍처의 발전

CNN의 발전사는 아이디어의 연쇄로 읽을 수 있다. 1998년 얀 르쿤의 LeNet-5는 손글씨 숫자 인식을 위해 합성곱-풀링-완전 연결을 결합한 초기 CNN으로, 오늘날 구조의 원형을 제시했다. 2012년 AlexNet은 ImageNet 대회에서 압도적 성적을 거두며 딥러닝 시대를 열었는데, GPU를

이용한 대규모 학습, ReLU 활성화, 드롭아웃 규제를 도입해 깊은 CNN이 실제로 작동함을 보였다.

2014년 VGG는 3×3의 작은 필터만을 규칙적으로 깊게 쌓아도 성능이 좋아짐을 보여, 작은 필터를 여러 층 쌓는 것이 큰 필터 하나보다 파라미터가 적으면서도 표현력이 크다는 통찰을 주었다. 같은 해 GoogLeNet(인셉션)은 여러 크기의 필터를 병렬로 적용해 계산 효율을 높이는 인셉션 모듈을 제안했다. 그러나 층을 무작정 깊게 쌓자 오히려 성능이 나빠지는 열화 문제가 드러났다.

2015년 ResNet은 이 난제를 잔차 연결(residual connection)로 해결했다. 각 블록이 목표 함수 전체가 아니라 입력과의 차이(잔차)만 학습하도록, 입력을 몇 개 층을 건너뛰어 출력에 직접 더해 주는 지름길을 두었다. 이 지름길 덕분에 기울기가 소실 없이 깊은 층까지 흘러, 수백 층에 이르는 초심층 망도 안정적으로 학습할 수 있게 되었다. 잔차 연결은 이후 거의 모든 심층 아키텍처의 표준 요소가 되었다.

수식

잔차 블록: $y = F(x) + x$ (F는 학습되는 잔차 함수, x는 지름길 연결)

12.5 전이학습과 대표 비전 과제

대규모 데이터로 처음부터 CNN을 학습시키려면 막대한 데이터와 계산이 필요하다. 전이학습(transfer learning)은 ImageNet처럼 큰 데이터로 미리 학습된 모델의 특징 추출 능력을 가져다 새 문제에 재사용하는 전략이다. 사전학습 모델의 앞쪽 층은 에지·질감 같은 범용 특징을 이미 잘 뽑아내므로, 뒷부분만 새 과제에 맞게 교체해 다시 학습하는 미세조정(fine-tuning)으로 적은 데이터로도 높은 성능을 얻을 수 있다.

컴퓨터 비전의 대표 과제는 목적에 따라 몇 가지로 나뉜다. 이미지 분류(classification)는 사진 한 장이 어떤 범주에 속하는지 하나의 라벨을 예측하는 가장 기본적인 과제다. 객체 검출(object detection)은 한 이미지 안의 여러 물체 각각에 대해 '무엇'인지와 함께 그 위치를 경계 상자(bounding box)로 찾아낸다. 즉 분류에 위치 추정이 더해진 과제다.

영역 분할(segmentation)은 한 걸음 더 나아가 이미지의 모든 픽셀에 라벨을 붙인다. 같은 종류의 물체를 하나로 묶는 의미 분할(semantic segmentation)과 같은 종류라도 개체별로 구분하는 인스턴스 분할(instance segmentation)로 나뉜다. 분류에서 검출, 분할로 갈수록 요구되는 위치 정밀도가 높아지며, 이 과제들은 자율주행·의료영상·위성사진 분석 등 다양한 실제 응용의 토대가 된다.

예시

소량의 의료영상만 있을 때, ImageNet 사전학습 ResNet의 앞쪽 층을 고정하고 마지막 분류 층만 교체·학습하면 처음부터 학습하는 것보다 빠르고 정확하게 병변을 분류할 수 있다.

이 장의 요약

- 합성곱은 작은 필터를 이미지 위에서 이동시키며 지역 특징을 추출하고, 가중치 공유로 파라미터를 줄이며 위치 불변성을 얻는다.
- 얇은 층은 에지 같은 저수준 특징을, 깊은 층은 부분 모양·물체 같은 고수준 특징을 학습한다.
- 출력 크기는 $(W - F + 2P)/S + 1$ 로 계산되며, 스트라이드는 다운샘플링을, 패딩은 크기 유지와 가장자리 보존을 담당한다.
- 풀링(특히 최대 풀링)은 공간 크기를 줄이고 위치 변화에 대한 견고성을 높인다.
- 전형적 CNN은 합성곱·풀링으로 특징을 추출하고 완전 연결·소프트맥스로 분류하며, 깊어질수록 공간 크기는 줄고 채널은 늘어난다.
- LeNet→AlexNet→VGG→GoogLeNet→ResNet으로 이어지는 발전에서, VGG는 작은 필터 적층을, ResNet은 잔차 연결($y=F(x)+x$)로 초심층 학습을 가능케 했다.
- 전이학습은 대규모 데이터로 사전학습된 모델을 재사용해 적은 데이터로도 높은 성능을 얻는 실무의 표준 전략이다.
- 대표 비전 과제는 이미지 분류, 위치까지 찾는 객체 검출, 픽셀 단위로 라벨링하는 영역 분할로 나뉘며 위치 정밀도 요구가 점점 높아진다.

핵심 용어

합성곱(Convolution)	필터를 이미지 위에서 이동시키며 원소별 곱의 합으로 특징을 추출하는 연산.
필터/커널(Filter/Kernel)	특정 시각 패턴에 반응하도록 학습되는 작은 가중치 격자.
특징맵(Feature Map)	필터를 이미지 전체에 적용해 얻은 2차원 출력.
가중치 공유(Weight Sharing)	하나의 필터를 이미지 전 영역에 동일하게 적용해 파라미터를 줄이는 원리.
위치 불변성(Translation Invariance)	물체의 위치가 달라도 같은 특징을 감지하는 CNN의 성질.
스트라이드(Stride)	필터를 한 번에 이동시키는 칸 수로, 출력 크기와 계산량을 조절한다.
패딩(Padding)	입력 가장자리에 값을 덧대 출력 크기를 유지하거나 가장자리를 보존하는 기법.
풀링(Pooling)	영역의 최댓값이나 평균으로 특징맵을 축소하는 다운샘플링 연산.
잔차 연결(Residual Connection)	입력을 층을 건너뛰어 출력에 더해 기울기 흐름을 돕는 ResNet의 핵심 구조.
ResNet(Residual Network)	잔차 연결로 수백 층 규모의 초심층 CNN 학습을 가능케 한 아키텍처.
전이학습(Transfer Learning)	대규모 데이터로 사전학습된 모델을 새 문제에 재사용하는 전략.
미세조정(Fine-tuning)	사전학습 모델의 일부 층을 새 과제 데이터로 다시 학습하는 전이학습 방식.
객체 검출(Object Detection)	이미지 내 여러 물체의 종류와 위치를 경계 상자로 찾는 과제.
영역 분할(Segmentation)	이미지의 각 픽셀에 라벨을 부여하는 과제로 의미 분할과 인스턴스 분할이 있다.

순환 신경망(RNN)과 시퀀스 모델링

언어, 음성, 주가, 센서 신호처럼 순서가 중요한 데이터를 시퀀스 데이터라 한다. 이런 데이터는 각 시점의 값이 앞뒤 맥락에 의존하므로, 입력을 독립적으로 다루는 일반 신경망으로는 다루기 어렵다. 순환 신경망(Recurrent Neural Network, RNN)은 과거 정보를 담은 은닉 상태를 시점마다 이어받아 순서와 맥락을 기억한다. 이 장에서는 RNN의 구조와 시간 전개, 장기 의존성 문제, 이를 해결한 LSTM과 GRU의 게이트 구조를 살펴보고, 시퀀스-투-시퀀스와 인코더-디코더, 그리고 어텐션과 트랜스포머로 이어지는 다리를 개념적으로 정리한다.

13.1 시퀀스 데이터와 RNN 구조

시퀀스 데이터는 요소들이 특정한 순서로 나열되어 그 순서 자체가 의미를 갖는 데이터다. 문장은 단어의 시퀀스이고, 음성은 시간에 따른 음향 신호의 시퀀스이며, 주가는 날짜별 값의 시퀀스다. 이런 데이터에서는 같은 요소라도 앞뒤 맥락에 따라 의미가 달라지고, 시퀀스의 길이도 예제마다 다르다. 따라서 고정 크기 입력을 전제하는 일반 신경망으로는 자연스럽게 다루기 어렵다.

RNN은 시퀀스를 한 시점씩 순서대로 읽으면서, 지금까지 본 내용을 요약한 은닉 상태(hidden state) h 를 유지한다. 각 시점 t 에서 현재 입력 x 와 직전 은닉 상태 h_{t-1} 를 함께 받아 새로운 은닉 상태를 계산하는데, 그 규칙은 $h_t = \tanh(W_x x + W_h h_{t-1} + b)$ 로 표현된다. 여기서 h_{t-1} 를 통해 과거 정보가 현재로 흘러들어, RNN은 순서와 맥락을 '기억'하게 된다.

RNN의 핵심은 모든 시점이 같은 가중치 W_x, W_h 를 공유한다는 점이다. 덕분에 시퀀스 길이에 상관없이 하나의 작은 신경망을 반복 적용할 수 있어, 임의 길이의 입력을 유연하게 처리한다. 출력은 과제에 따라 매 시점마다 낼 수도 있고(예: 품사 태깅), 시퀀스 전체를 다 읽은 뒤 마지막에 한 번 낼 수도 있다(예: 문장 감성 분류).

수식

$$h_t = \tanh(W_x x + W_h h_{t-1} + b), y_t = W_y h_t + b_y$$

13.2 시간 전개와 장기 의존성 문제

순환 구조는 이해를 돕기 위해 시간 축을 따라 펼쳐서 볼 수 있는데, 이를 시간 전개(unrolling)라 한다. 펼치면 각 시점이 하나의 층처럼 보이는, 시퀀스 길이만큼 깊은 신경망이 된다. 다만 모든 시점의 층이 동일한 가중치를 공유한다는 점이 일반 심층망과 다르다. 이렇게 펼친 망에 역전파를 적용하는 것을 시간에 걸친 역전파(BPTT, Backpropagation Through Time)라 부른다.

그런데 시간 전개된 RNN은 사실상 매우 깊은 신경망이므로, 앞서 본 기울기 소실과 폭주 문제가 그대로, 오히려 더 심각하게 나타난다. 역전파 과정에서 같은 가중치 행렬이 시점 수만큼 반복해서 곱해지기 때문이다. 그 결과 먼 과거의 기울기가 거의 0으로 사라지거나 폭발하기 쉽다.

이로 인해 기본 RNN은 장기 의존성(long-term dependency)을 학습하기 어렵다. 즉 시점이 멀리 떨어진 정보 사이의 관계를 포착하지 못한다. 예를 들어 '나는 프랑스에서 자랐다 ... 그래서 나는 유창한 (프랑스어)를 한다'라는 문장에서 빈칸을 채우려면 멀리 앞의 '프랑스'를 기억해야 하는데, 거리가 멀면 기본 RNN은 그 연결을 놓치기 쉽다. 이 한계가 게이트 구조를 가진 개선 모델의 등장 배경이 되었다.

예시

짧은 의존성('하늘은 _' → '파랗다')은 기본 RNN도 잘 학습하지만, 수십 단어 떨어진 주어-동사 일치처럼 긴 의존성은 기울기 소실로 학습이 어렵다.

13.3 LSTM과 GRU의 게이트

LSTM(Long Short-Term Memory)은 장기 의존성 문제를 해결하기 위해 고안된 특수한 RNN이다. 핵심은 은닉 상태와 별도로 정보를 길게 실어 나르는 셀 상태(cell state) C 를 두고, 여기에 무엇을 더하고 지울지를 세 개의 게이트로 조절하는 것이다. 게이트는 0과 1 사이 값을 내는 시그모이드로 만들어져, 정보가 얼마나 통과할지를 부드럽게 결정하는 밸브처럼 작동한다.

세 게이트의 역할은 다음과 같다. 망각 게이트(forget gate)는 셀 상태에서 어떤 과거 정보를 버릴지 정하고, 입력 게이트(input gate)는 새 정보 중 어떤 것을 셀에 더할지 정하며, 출력 게이트(output gate)는 갱신된 셀 상태에서 무엇을 은닉 상태로 내보낼지 정한다. 이 구조에서 셀 상태는 게이트가 열려 있는 한 정보를 큰 변형 없이 여러 시점에 걸쳐 실어 나를 수 있어, 기울기가 잘 소실되지 않고 먼 과거의 정보가 오래 보존된다.

GRU(Gated Recurrent Unit)는 LSTM을 단순화한 모델로, 별도의 셀 상태 없이 은닉 상태 하나만 쓰고 게이트도 두 개(리셋 게이트와 업데이트 게이트)로 줄였다. 업데이트 게이트가 과거 상태를 얼마나 유지하고 새 정보로 얼마나 대체할지를 한꺼번에 조절한다. GRU는 LSTM보다 파라미터가 적어 계산이 가볍고 학습이 빠르면서도, 많은 과제에서 비슷한 성능을 내어 널리 사용된다.

수식

망각 gate: $f = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$, 입력 gate: $i = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$, 셀 갱신: $C = f \odot C_{t-1} + i \odot C_t$, 출력 gate: $o = \sigma(\dots)$, $h = o \odot \tanh(C)$

13.4 시퀀스-투-시퀀스와 인코더-디코더

기계 번역처럼 입력 시퀀스와 출력 시퀀스의 길이가 서로 다르고 정렬도 일정하지 않은 과제를 다루려면 특별한 구조가 필요하다. 시퀀스-투-시퀀스(sequence-to-sequence)는 이런 문제를 위해 두 개의 RNN을 잇는 인코더-디코더(encoder-decoder) 구조를 사용한다. 인코더가 입력 시퀀스를 읽어 고정 크기의 벡터로 요약하고, 디코더가 그 벡터를 바탕으로 출력 시퀀스를 한 요소씩 생성한다.

구체적으로 인코더 RNN은 입력 문장을 끝까지 읽어 마지막 은닉 상태에 문장 전체의 의미를 압축하는데, 이를 문맥 벡터(context vector)라 한다. 디코더 RNN은 이 문맥 벡터를 초기 상태로 삼아, 시작 토큰부터 출발해 이전에 생성한 단어를 다음 입력으로 넣으며 종료 토큰이 나올

때까지 출력을 이어 간다. 이 틀은 번역, 요약, 챗봇, 음성 인식 등 입출력이 모두 시퀀스인 다양한 과제에 두루 적용된다.

그러나 이 기본 구조에는 병목이 있다. 아무리 긴 입력 문장이라도 그 모든 정보를 하나의 고정 크기 문맥 벡터에 우겨 넣어야 하므로, 문장이 길어지면 앞부분 정보가 손실되어 성능이 떨어진다. 즉 하나의 벡터가 문장 전체를 대표하기에는 용량이 부족하다. 이 병목을 푸는 열쇠가 바로 다음 절에서 다룰 어텐션이다.

의사코드

```
# 시퀀스-투-시퀀스 의사코드
context = encoder(input_sequence) # 입력을 문맥 벡터로 요약
state = context
token = <START>
output = []
while token != <END>:
    token, state = decoder(token, state) # 한 단어씩 생성
    output.append(token)
```

13.5 어텐션의 등장과 트랜스포머의 다리

어텐션(attention)은 디코더가 각 출력 단어를 생성할 때, 하나의 고정 벡터에만 의존하지 않고 입력 시퀀스의 모든 시점을 다시 들여다보며 그 순간에 가장 관련 있는 부분에 집중하도록 하는 메커니즘이다. 예를 들어 번역에서 특정 목표 단어를 낼 때, 원문에서 그에 대응하는 단어에 큰 가중치를 두어 참고한다. 이렇게 입력 전체를 관련도에 따라 가중 합산한 문맥을 매 시점 새로 만들어 쓰므로, 고정 벡터의 병목이 사라진다.

어텐션은 세 요소로 이해할 수 있다. 지금 무엇을 찾는지 나타내는 질의(query), 참조 대상의 색인 역할을 하는 키(key), 실제로 가져올 내용인 값(value)이다. 질의와 각 키의 유사도로 가중치를 구하고, 그 가중치로 값들을 합산해 필요한 정보를 뽑아낸다. 이 질의-키-값 틀은 이후 다양한 어텐션 방식의 공통 언어가 되었다.

어텐션의 위력이 분명해지자, 2017년 '어텐션만 있으면 된다(Attention Is All You Need)'라는 연구가 순환 구조를 아예 걷어내고 오직 어텐션만으로 시퀀스를 처리하는 트랜스포머(Transformer)를 제안했다. 트랜스포머는 시퀀스를 순차적으로 처리하지 않고 모든 시점을 동시에 보며 서로의 관계를 계산하는 자기 어텐션(self-attention)을 사용해, 병렬 계산이 쉽고 장거리 의존성도 잘 포착한다. 이 구조는 오늘날 대규모 언어 모델의 토대가 되었으며, RNN에서 어텐션으로 이어진 이 흐름이 다음 부에서 다룰 현대 자연어 처리의 출발점이다.

수식

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V \quad (Q: \text{질의}, K: \text{키}, V: \text{값})$$

이 장의 요약

- 시퀀스 데이터는 순서가 의미를 가지며 길이가 가변적이어서, 은닉 상태로 맥락을 기억하는 RNN이 적합하다.
- RNN은 $h = \tanh(W_x x + W_h h_{-1} + b)$ 로 과거 정보를 현재로 전달하며, 모든 시점이 같은 가중치를 공유해 임의 길이 입력을 처리한다.
- 시간 전개된 RNN은 매우 깊은 망과 같아, 같은 가중치가 반복 곱해지며 기울기 소실·폭주가 심해 장기 의존성 학습이 어렵다.
- LSTM은 셀 상태와 망각·입력·출력 세 게이트로 정보를 오래 보존해 장기 의존성 문제를 해결한다.
- GRU는 LSTM을 단순화해 게이트를 둘로 줄이고 셀 상태를 없애, 더 가벼우면서 비슷한 성능을 낸다.
- 인코더-디코더 시퀀스-투-시퀀스 구조는 입력을 문맥 벡터로 요약해 다른 길이의 출력 시퀀스를 생성하며, 번역·요약·챗봇에 쓰인다.
- 고정 문맥 벡터는 긴 입력에서 정보 병목을 일으키며, 어텐션이 입력 전체를 관련도에 따라 참조해 이 병목을 해소한다.
- 어텐션은 질의-키-값 틀로 관련 정보를 가중 합산하며, 순환을 제거한 트랜스포머의 자기 어텐션으로 이어져 현대 언어 모델의 토대가 되었다.

핵심 용어

시퀀스 데이터(Sequence Data)	요소의 순서가 의미를 갖는 가변 길이 데이터로 문장·음성·시계열 등.
순환 신경망(Recurrent Neural Network, RNN)	은닉 상태를 시점마다 이어받아 순서와 맥락을 기억하는 신경망.
은닉 상태(Hidden State)	지금까지 본 시퀀스 정보를 요약해 다음 시점으로 전달하는 벡터.
시간 전개(Unrolling)	순환 구조를 시간 축을 따라 펼쳐 시퀀스 길이만큼 깊은 망으로 보는 표현.
시간에 걸친 역전파(BPTT)	시간 전개된 RNN에 역전파를 적용해 가중치를 학습하는 방법.
장기 의존성(Long-term Dependency)	멀리 떨어진 시점 사이의 관계로, 기본 RNN이 학습하기 어려운 대상.
LSTM(Long Short-Term Memory)	셀 상태와 세 게이트로 정보를 오래 보존해 장기 의존성을 학습하는 RNN.
셀 상태(Cell State)	LSTM에서 정보를 큰 변형 없이 여러 시점에 걸쳐 실어 나르는 경로.
게이트(Gate)	시그모이드로 정보 통과량을 0~1로 조절하는 LSTM·GRU의 밸브 구조.
GRU(Gated Recurrent Unit)	셀 상태 없이 게이트를 둘로 줄여 단순화한 경량 순환 유닛.
인코더-디코더(Encoder-Decoder)	입력을 문맥 벡터로 요약하고 출력 시퀀스를 생성하는 두 RNN 구조.
문맥 벡터(Context Vector)	인코더가 입력 시퀀스 전체를 압축한 고정 크기 표현.
어텐션(Attention)	출력 생성 시 입력 전체를 관련도에 따라 가중 참조하는 메커니즘.

트랜스포머(Transformer)

순환을 제거하고 자기 어텐션만으로 시퀀스를 처리하는 병렬적 아키텍처.

제4부

현대 인공지능: 트랜스포머와 LLM·RAG

제14장

트랜스포머와 대규모 언어모델(LLM)

2017년 발표된 트랜스포머(Transformer) 구조는 순환 신경망이 지배하던 자연어 처리의 판도를 완전히 바꾸어 놓았다. 순차적으로만 처리하던 문장을 병렬로 다루면서도 단어들 사이의 장거리 의존 관계를 효과적으로 포착하는 셀프 어텐션(self-attention) 메커니즘 덕분이다. 오늘날 우리가 사용하는 대규모 언어모델(LLM, Large Language Model)은 대부분 이 트랜스포머를 뼈대로 삼아 방대한 말뭉치로 사전학습된 것이다. 이 장에서는 셀프 어텐션의 직관에서 출발하여 멀티헤드 어텐션, 위치 인코딩, 인코더·디코더 구조, 토큰화와 임베딩, 사전학습과 파인튜닝, 인컨텍스트 학습, 스케일링 법칙까지 현대 LLM을 이루는 핵심 개념을 차례로 살펴본다.

14.1 셀프 어텐션의 직관

문장을 이해한다는 것은 각 단어를 고립적으로 보는 것이 아니라 주변 단어와의 관계 속에서 해석하는 일이다. 예를 들어 '그는 은행에서 돈을 찾았다'와 '그는 강 은행에 앉았다'에서 '은행'의 의미는 함께 등장한 다른 단어에 의해 달라진다. 셀프 어텐션은 바로 이 직관을 수식으로 구현한 장치로, 한 단어를 표현할 때 문장 안의 모든 단어를 얼마나 참고할지 가중치로 결정한다. 즉 각 단어가 자기 자신을 포함한 문장 전체를 '둘러보며' 문맥에 맞는 표현을 새로 합성한다.

구체적으로 각 토큰의 임베딩 벡터로부터 세 가지 벡터를 만든다. 질의(Query, Q)는 '내가 무엇을 찾고 있는가'를, 키(Key, K)는 '나는 어떤 정보를 담고 있는가'를, 값(Value, V)은 '실제로 전달할 내용'을 나타낸다. 한 토큰의 Q와 다른 모든 토큰의 K를 내적하면 두 토큰이 얼마나 관련 있는지를 재는 점수(유사도)가 나온다. 이 점수를 정규화한 가중치로 각 토큰의 V를 가중합하면, 문맥이 반영된 새로운 표현이 완성된다.

이 방식의 강점은 두 가지다. 첫째, 문장의 길이와 무관하게 임의의 두 단어가 한 번의 연산으로 직접 연결되므로 멀리 떨어진 단어 사이의 의존 관계도 손실 없이 포착한다. 둘째, 모든 토큰 쌍의 계산이 서로 독립적이어서 행렬 곱으로 한꺼번에 병렬 처리할 수 있고, 이는 GPU를 이용한 대규모 학습을 가능하게 한 결정적 이유가 되었다.

예시

'The animal didn't cross the street because it was too tired'라는 문장에서 'it'이 무엇을 가리키는지는 모호하다. 셀프 어텐션은 'it'의 Query가 'animal'의 Key와 높은 유사도를 갖도록 학습되어, 'it'의 표현에 'animal'의 정보를 많이 섞어 넣는다. 이렇게 대명사 해소 같은 문맥 의존적 이해가 자연스럽게 이루어진다.

14.2 스케일드 닷프로덕트 어텐션

셀프 어텐션의 표준 형태는 스케일드 닷프로덕트 어텐션(scaled dot-product attention)이다. 입력 토큰들의 Query-Key-Value를 각각 행으로 쌓은 행렬 Q, K, V를 사용하며, QK 로 모든 토큰 쌍의 유사도 점수를 한 번에 구한다. 이 점수 행렬을 키 차원의 제곱근 $\sqrt{d}$ 로 나눈 뒤 행 방향 softmax를 적용해 합이 1인 어텐션 가중치를 얻고, 그 가중치로 V를 가중합한다.

$\sqrt{d}$ 로 나누는 스케일링에는 분명한 이유가 있다. 차원 d 가 커지면 내적 값의 분산도 커져 softmax의 입력이 매우 큰 양수와 음수로 벌어진다. 그러면 softmax 출력이 한 토큰에 거의 1로 몰리는 극단적 분포가 되고, 그 지점에서 기울기가 0에 가까워져 학습이 정체된다. $\sqrt{d}$ 로 나누어 점수의 크기를 적절히 유지하면 이런 기울기 소실을 완화할 수 있다.

여기에 마스킹(masking)을 결합하면 용도가 확장된다. 디코더의 자기회귀 생성에서는 아직 생성하지 않은 미래 토큰을 참고하면 안 되므로, 미래 위치의 점수를 softmax 이전에 음의 무한대로 만들어 가중치를 0으로 강제한다. 이를 인과적 마스킹(causal masking)이라 부른다. 반대로 패딩 토큰을 무시하기 위한 패딩 마스크도 같은 원리로 적용한다.

수식

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d}) \cdot V$$

의사코드

```
함수 scaled_dot_product_attention(Q, K, V, mask=None):
    # Q, K, V 크기: (토큰수 n) x (차원 d_k 또는 d_v)
    scores = matmul(Q, transpose(K))    # (n x n) 유사도 점수
    scores = scores / sqrt(d_k)         # 스케일링
    만약 mask가 주어지면:
        scores = scores + mask          # 금지 위치에 -무한대를 더함
    weights = softmax(scores, axis=마지막) # 행마다 합이 1
    output = matmul(weights, V)         # 값의 가중합
    반환 output, weights
```

14.3 멀티헤드 어텐션과 위치 인코딩

하나의 어텐션만으로는 단어들 사이의 다양한 관계(문법적 일치, 지시 관계, 의미적 유사성 등)를 동시에 포착하기 어렵다. 멀티헤드 어텐션(multi-head attention)은 $Q \cdot K \cdot V$ 를 여러 개의 저차원 공간으로 각각 다르게 투영해 여러 '헤드'가 병렬로 어텐션을 수행하게 한 뒤, 각 헤드의 출력을 이어붙이고 다시 선형 변환해 합친다. 각 헤드가 서로 다른 표현 부분공간에서 서로 다른 관계에 주목하므로, 모델은 더 풍부한 문맥 정보를 담을 수 있다.

한편 셀프 어텐션은 집합 연산이라 토큰의 순서 정보를 스스로 갖지 못한다. '개가 사람을 물었다'와 '사람이 개를 물었다'를 구분하려면 위치 정보를 명시적으로 주입해야 한다. 이를 위해 각 위치에 대응하는 위치 인코딩(positional encoding)을 토큰 임베딩에 더한다. 원논문은 서로 다른 주기의 사인·코사인 함수로 위치를 부호화했는데, 이는 상대적 거리를 일관되게 표현하고 학습 때 보지 못한 긴 문장에도 외삽할 수 있는 장점이 있다.

위치 정보를 다루는 방식은 이후 다양하게 발전했다. 위치 임베딩을 학습 가능한 파라미터로 두는 방법(learned positional embedding), 그리고 최근 많은 LLM이 채택한 회전 위치 임베딩(RoPE, Rotary Positional Embedding)이나 상대 위치 편향(relative position bias) 방식이 대표적이다. 이들은 긴 문맥 길이로의 확장성과 상대 위치 표현 능력에서 이점을 보인다.

예시

번역 모델을 뜯어 보면 어떤 헤드는 동사와 그 목적어를 연결하고, 다른 헤드는 관사와 명사의 성·수 일치를 확인하며, 또 다른 헤드는 문장 부호 주변에 주목하는 식으로 역할이 분화되어 있음이 시각화 연구에서 관찰된다.

14.4 인코더·디코더 구조와 토큰화·임베딩

원조 트랜스포머는 인코더와 디코더로 구성된다. 인코더는 입력 문장 전체를 양방향으로 참고해 문맥 표현을 만들고, 디코더는 인코더 표현과 지금까지 생성한 출력을 함께 보며 다음 토큰을 하나씩 만든다. 각 층은 멀티헤드 어텐션과 위치별 피드포워드 신경망으로 이루어지며, 잔차 연결(residual connection)과 층 정규화(layer normalization)로 깊은 신경망의 안정적 학습을 돕는다. 디코더에는 인코더 출력을 참조하는 교차 어텐션(cross-attention) 층이 추가된다.

이 구조는 세 계열로 특화되었다. 인코더만 쓰는 모델(BERT 계열)은 양방향 문맥 이해가 필요한 분류·추출 과제에, 디코더만 쓰는 모델(GPT 계열)은 자기회귀 텍스트 생성에, 인코더-디코더 모델(T5, BART 계열)은 번역·요약 같은 시퀀스 변환에 강점을 보인다. 오늘날 대화형 LLM의 주류는 디코더 전용 구조다.

모델에 텍스트를 넣기 전에는 토큰화(tokenization)를 거친다. 오늘날에는 단어와 문자의 중간 단위인 서브워드(subword) 토큰화가 표준으로, BPE(Byte Pair Encoding), WordPiece, SentencePiece 등이 쓰인다. 서브워드 방식은 자주 쓰는 단어는 통째로, 드문 단어는 조각으로 쪼개어 어휘 크기를 제어하면서도 미등록 단어(OOV) 문제를 줄인다. 각 토큰은 임베딩 행렬을 통해 밀집 벡터로 변환되어 트랜스포머 층의 입력이 된다.

예시

'tokenization'이라는 단어는 BPE에서 'token'과 '##ization'처럼 두 조각으로 나뉠 수 있다. 이렇게 하면 'token', 'tokenizer', 'tokenizing' 같은 파생어들이 공통 조각을 공유하여 어휘 사전을 작게 유지하면서도 표현력을 확보한다.

14.5 사전학습: 다음 토큰 예측과 마스킹

현대 LLM의 능력은 대부분 사전학습(pre-training)에서 나온다. 핵심은 인간이 일일이 정답을 붙이지 않아도 되는 자기지도 학습(self-supervised learning)이라는 점이다. 방대한 텍스트 자체가 정답을 품고 있어, 문장의 일부를 가리고 그것을 맞히도록 학습시키면 모델은 문법·상식·세계 지식을 스스로 흡수한다.

GPT 계열이 쓰는 목표는 다음 토큰 예측(next-token prediction), 즉 인과적 언어 모델링이다. 지금까지의 토큰들이 주어졌을 때 바로 다음 토큰의 확률 분포를 예측하고, 실제 토큰과의 교차 엔트로피를 최소화한다. 문장을 왼쪽에서 오른쪽으로 한 토큰씩 이어가는 이 방식은 그대로 텍스트 생성 능력으로 이어진다. BERT 계열은 마스킹 언어 모델링(MLM, Masked Language Modeling)을 쓰는데, 입력의 일부 토큰을 [MASK]로 가리고 양방향 문맥으로 원래 토큰을 복원하게 한다.

사전학습을 마친 모델은 특정 과제에 맞게 파인튜닝(fine-tuning)한다. 소량의 라벨 데이터로 전체 파라미터를 추가 학습하는 전통적 방식 외에, 최근에는 LoRA 같은 파라미터 효율적

파인튜닝(PEFT)이 널리 쓰인다. 특히 대화형 모델은 지시 튜닝(instruction tuning)과 인간 피드백 기반 강화학습(RLHF) 또는 DPO 같은 선호 정렬(alignment) 과정을 거쳐 사용자의 지시를 잘 따르고 유해한 출력을 줄이도록 다듬어진다.

수식

$$L = - \sum \log P(x_i | x_1, \dots, x_{i-1}; \theta) \text{ (다음 토큰 예측의 손실)}$$

14.6 인컨텍스트 학습과 프롬프팅

충분히 큰 언어모델은 파라미터를 갱신하지 않고도 프롬프트 안에 주어진 예시만으로 새로운 과제를 수행하는 능력을 보인다. 이를 인컨텍스트 학습(in-context learning)이라 한다. 프롬프트에 예시를 몇 개 제시하는 것을 퓨샷(few-shot), 하나만 주는 것을 원샷(one-shot), 예시 없이 지시만 하는 것을 제로샷(zero-shot) 프롬프팅이라 부른다. 모델은 예시의 형식과 패턴을 파악해 이어질 답의 형식을 추론한다.

프롬프트의 구성 방식은 성능에 큰 영향을 준다. 문제를 단계적으로 풀도록 유도하는 생각의 사슬(chain-of-thought) 프롬프팅은 복잡한 추론 과제에서 정확도를 높이고, 역할 부여·출력 형식 지정·제약 명시 같은 기법도 널리 쓰인다. 다만 인컨텍스트 학습은 진짜 학습이라기보다 사전학습으로 이미 습득한 능력을 프롬프트가 '끌어내는' 것에 가깝다는 점을 이해하는 것이 중요하다.

인컨텍스트 학습은 모델을 재학습하지 않고도 빠르게 과제를 바꿀 수 있어 실용적 가치가 크다. 다만 프롬프트에 담을 수 있는 정보는 문맥 창(context window) 길이에 제한되고, 모델이 학습 시점 이후의 최신 지식이나 사적·전문적 지식을 알지 못한다는 한계가 있다. 이 한계를 외부 지식으로 보완하려는 시도가 바로 다음 장들에서 다룰 검색 증강 생성(RAG)이다.

예시

감성 분류를 시킬 때 '리뷰: 최고예요 → 긍정 / 리뷰: 별로였어요 → 부정 / 리뷰: 다시 오고 싶어요 →' 처럼 예시 두 개와 빈 답을 제시하면, 모델은 별도 학습 없이 '긍정'을 이어 쓴다. 이것이 퓨샷 인컨텍스트 학습의 전형적인 모습이다.

14.7 스케일링 법칙과 대표 모델 계열

스케일링 법칙(scaling laws)은 모델 파라미터 수, 학습 데이터 양, 계산량을 늘릴수록 언어모델의 손실이 예측 가능한 멍함수(power-law) 형태로 완만히 감소한다는 경험적 관찰이다. 이 규칙성은 '더 크게, 더 많이'가 성능 향상의 신뢰할 만한 길임을 보여 주었고, 대규모 투자로 이어지는 이론적 근거가 되었다. 나아가 규모가 임계점을 넘으면 작은 모델에는 없던 능력이 비교적 갑작스럽게 나타나는 창발적 능력(emergent abilities)도 보고되었다.

다만 무작정 파라미터만 키우는 것이 최적은 아니다. Chinchilla 연구는 주어진 계산 예산에서 모델 크기와 데이터 양을 균형 있게 늘려야 하며, 많은 대형 모델이 데이터에 비해 과도하게 컸음을 지적했다. 즉 컴퓨트-최적(compute-optimal) 학습에는 파라미터와 토큰 수의 적절한 비율이 존재한다. 이 통찰은 이후 모델 설계에 큰 영향을 주었다.

대표 모델 계열은 아이디어 수준에서 정리할 수 있다. BERT는 양방향 인코더로 문맥 이해 과제의 수준을 끌어올렸고, GPT 계열은 디코더 전용 자기회귀 생성으로 규모를 키워 범용 대화·생성 능력을 열었다. 이후 T5(텍스트-투-텍스트 통합), LLaMA(효율적 오픈 모델), 그리고 다양한 상용·오픈소스 LLM이 등장했다. 세부 구현은 계열마다 다르지만, 트랜스포머·대규모 자기지도 사전학습·정렬이라는 공통 골격을 공유한다는 점이 핵심이다.

수식

$L(N) \propto (N / N_0)^\alpha$ (파라미터 수 N 이 커질수록 손실 L 이 멱함수로 감소)

이 장의 요약

- 셀프 어텐션은 각 토큰을 Query-Key-Value로 변환해 문장 전체를 참고함으로써 문맥 의존적 표현을 만든다.
- 스케일드 닷프로덕트 어텐션은 QK를 $\sqrt{d}$ 로 나눈 뒤 softmax로 가중치를 구하고 V를 가중합하며, $\sqrt{d}$ 스케일링은 기울기 소실을 완화한다.
- 멀티헤드 어텐션은 여러 부분공간에서 서로 다른 관계를 병렬로 포착하고, 위치 인코딩은 순서 정보를 주입한다.
- 트랜스포머는 인코더 전용(BERT), 디코더 전용(GPT), 인코더-디코더(T5) 세 계열로 특화된다.
- 서브워드 토큰화(BPE·WordPiece)와 임베딩을 통해 텍스트가 모델 입력 벡터로 변환된다.
- 사전학습은 다음 토큰 예측(인과적 LM)이나 마스킹 언어 모델링으로 자기지도 학습되며, 이후 파인튜닝·지시 튜닝·정렬을 거친다.
- 인컨텍스트 학습은 파라미터 갱신 없이 프롬프트의 예시만으로 과제를 수행하는 능력으로, 제로/원/퓨샷 프롬프팅이 있다.
- 스케일링 법칙은 규모 확장에 따른 성능 향상을 멱함수로 기술하며, Chinchilla는 파라미터와 데이터의 균형(컴퓨터-최적)을 강조했다.

핵심 용어

트랜스포머(Transformer)	셀프 어텐션을 핵심으로 하는 신경망 구조로, 순환 없이 병렬로 시퀀스를 처리한다.
셀프 어텐션(Self-Attention)	한 시퀀스 내에서 각 토큰이 다른 모든 토큰을 참고해 문맥 표현을 만드는 메커니즘이다.
Query-Key-Value	어텐션에서 질의·키·값 벡터로, Q와 K의 유사도로 가중치를 정하고 V를 가중합한다.
스케일드 닷프로덕트 어텐션(Scaled Dot-Product Attention)	QK를 $\sqrt{d}$ 로 나눈 뒤 softmax를 적용해 V를 가중합하는 표준 어텐션 계산이다.
멀티헤드 어텐션(Multi-Head Attention)	여러 부분공간에서 병렬로 어텐션을 수행해 다양한 관계를 동시에 포착하는 기법이다.
위치 인코딩(Positional Encoding)	순서 정보가 없는 어텐션에 토큰의 위치 정보를 주입하는 방법으로, 사인·코사인 또는 RoPE 등이 있다.

인코더·디코더(Encoder-Decoder)	입력을 문맥 표현으로 부호화하는 인코더와 출력을 생성하는 디코더로 이루어진 구조다.
토큰화(Tokenization)	텍스트를 모델 입력 단위로 분할하는 과정으로, BPE·WordPiece 같은 서브워드 방식이 표준이다.
서브워드(Subword)	단어와 문자의 중간 단위로, 어휘 크기를 제어하며 미등록 단어 문제를 완화한다.
임베딩(Embedding)	토큰이나 텍스트를 의미를 담은 밀집 실수 벡터로 표현한 것이다.
사전학습(Pre-training)	방대한 말뭉치에서 자기지도 방식으로 언어의 일반적 표현을 학습하는 단계다.
다음 토큰 예측(Next-Token Prediction)	이전 토큰들로부터 다음 토큰의 확률을 예측하는 인과적 언어 모델링 목표다.
마스킹 언어 모델링(Masked Language Modeling)	입력의 일부 토큰을 가리고 양방향 문맥으로 복원하게 하는 BERT 계열의 학습 목표다.
파인튜닝(Fine-tuning)	사전학습된 모델을 특정 과제나 도메인 데이터로 추가 학습해 적응시키는 과정이다.
인컨텍스트 학습(In-Context Learning)	파라미터를 갱신하지 않고 프롬프트의 예시만으로 새 과제를 수행하는 능력이다.
스케일링 법칙(Scaling Laws)	모델·데이터·계산량을 늘릴수록 손실이 멱함수로 감소한다는 경험적 규칙이다.

제15장

임베딩과 벡터 검색

언어를 컴퓨터가 다루려면 단어와 문장을 숫자로 바꾸어야 한다. 그러나 단순히 번호를 매기는 것으로는 '왕'과 '여왕'이 의미상 가깝다는 사실을 담을 수 없다. 임베딩(embedding)은 의미가 비슷한 대상이 벡터 공간에서 서로 가까이 놓이도록 학습된 밀집 표현으로, 현대 자연어 처리와 검색의 토대를 이룬다. 이 장에서는 표현 학습으로서의 임베딩, 단어 임베딩에서 문장 임베딩으로의 발전, 코사인 유사도, 벡터 데이터베이스와 근사 최근접 이웃 탐색, 그리고 검색 품질에 영향을 주는 임베딩 모델과 청킹·색인 전략을 다룬다. 이 내용은 다음 장에서 배울 RAG의 핵심 기반이 된다.

15.1 표현 학습으로서의 임베딩

임베딩의 핵심 아이디어는 이산적 기호(단어, 문서 등)를 연속적인 저차원 벡터 공간에 배치하되, 그 공간에서의 거리와 방향이 의미적 관계를 반영하도록 만드는 것이다. 원-핫 인코딩처럼 각 단어에 서로 직교하는 좌표를 주면 모든 단어 쌍이 동일하게 '무관'해지지만, 임베딩은 함께 쓰이는 맥락이 비슷한 단어들을 가깝게 학습해 의미의 유사성을 기하학적 근접성으로 표현한다.

이러한 표현 학습(representation learning)이 강력한 이유는 '분포 가설(distributional hypothesis)'에 있다. 즉 '비슷한 문맥에 나타나는 단어는 비슷한 의미를 가진다'는 언어학적 통찰이다. 모델은 방대한 텍스트에서 어떤 단어들이 함께 등장하는지를 학습함으로써, 명시적 정의 없이도 의미의 지도를 그려낸다. 잘 학습된 임베딩 공간에서는 '파리 프랑스 + 이탈리아 로마' 같은 유추가 벡터 연산으로 성립하기도 한다.

임베딩은 단어에 국한되지 않는다. 문장·문서·이미지·사용자·상품 등 거의 모든 대상을 벡터로 표현할 수 있으며, 이렇게 얻은 벡터는 분류, 군집화, 추천, 검색의 공통 입력이 된다. 특히 검색에서는 질의와 문서를 같은 임베딩 공간에 두고 벡터 간 근접성으로 관련성을 판단하는 의미 검색(semantic search)이 가능해진다.

임베딩의 차원 수는 표현력과 비용의 절충을 나타낸다. 차원이 높을수록 더 미세한 의미 구분을 담을 수 있지만 저장 공간과 검색 계산량이 늘어난다. 또한 임베딩 공간은 사람이 직접 해석하기 어려운 잠재 공간(latent space)이어서 각 축이 무엇을 뜻하는지 명시적으로 알기는 어렵다. 그럼에도 벡터 사이의 거리와 방향이 의미 관계를 안정적으로 반영하기에, 임베딩은 검색·추천·군집화 전반을 잇는 공통 언어로 기능한다.

예시

키워드 검색은 '자동차 고장'으로 검색하면 정확히 그 단어가 든 문서만 찾지만, 임베딩 기반 의미 검색은 '차가 시동이 안 걸려요'라는 문장도 의미가 가깝다고 판단해 함께 찾아낸다. 표현이 의미를 담기에 표면적 단어가 달라도 연결되는 것이다.

15.2 단어 임베딩에서 문장 임베딩으로

초기 임베딩의 대표는 word2vec이다. word2vec은 주변 단어로 중심 단어를 맞히거나(CBOW) 중심 단어로 주변 단어를 맞히는(Skip-gram) 얇은 신경망을 학습하여, 부산물로 얻는 가중치 벡터를 단어 임베딩으로 쓴다. GloVe는 전역적 동시 출현 통계를 활용하고, fastText는 단어를 문자 n-그램으로 쪼개 미등록 단어와 형태 변화에 강하다. 이들은 모두 단어마다 하나의 고정된 벡터를 부여하는 정적 임베딩이다.

정적 임베딩의 한계는 다의어를 다루지 못한다는 점이다. '배'가 과일인지 신체인지 선박인지에 관계없이 같은 벡터를 갖는다. 트랜스포머 기반의 문맥 임베딩(contextual embedding)은 이 문제를 해결한다. 같은 단어라도 문장 속 문맥에 따라 다른 벡터를 부여하므로, 의미의 미묘한 차이를 반영할 수 있다.

검색과 유사도 판단에는 문장·문서 단위의 임베딩이 필요하다. 문장 임베딩은 문맥 모델의 토큰 벡터들을 평균 풀링하거나 요약 토큰(예: [CLS])을 쓰는 방식으로 얻는데, 그대로 쓰기보다 문장 쌍의 의미 유사도를 잘 반영하도록 특별히 학습한 모델이 성능이 좋다. Sentence-BERT처럼 유사한 문장은 가깝게, 다른 문장은 멀게 만드는 대조 학습(contrastive learning) 기반 임베딩 모델이 널리 쓰인다.

예시

word2vec에서 벡터('king') 벡터('man') + 벡터('woman')를 계산하면 그 결과가 벡터('queen')와 가장 가깝게 나오는 유명한 유추 현상이 관찰된다. 이는 임베딩 공간의 특정 방향이 '성별' 같은 의미 축을 담고 있음을 시사한다.

15.3 의미 유사도와 코사인 유사도

두 임베딩이 얼마나 비슷한지를 재는 표준 척도는 코사인 유사도(cosine similarity)다. 이는 두 벡터가 이루는 각도의 코사인 값으로, 벡터의 크기(길이)와 무관하게 방향의 일치도만을 측정한다. 값은 -1에서 1 사이이며, 1에 가까울수록 방향이 같아 의미가 유사하고, 0이면 무관, -1이면 정반대를 뜻한다. 문서의 길이나 임베딩의 스케일 차이에 강건하다는 점에서 텍스트 유사도에 특히 적합하다.

코사인 유사도 외에도 내적(dot product)과 유클리드 거리(L2 distance)가 쓰인다. 벡터를 미리 단위 길이로 정규화(normalize)해 두면 내적과 코사인 유사도가 동일해지고, 코사인 유사도와 유클리드 거리도 단조 관계를 이루어 순위가 같아진다. 그래서 많은 벡터 검색 시스템은 임베딩을 정규화한 뒤 빠른 내적 연산으로 유사도를 계산한다.

다만 코사인 유사도가 곧 '정답 관련성'을 보장하지는 않는다. 임베딩 모델이 어떤 목표로 학습되었는지, 도메인이 맞는지에 따라 같은 점수라도 의미가 달라질 수 있다. 따라서 유사도 척도는 도구일 뿐이며, 실제 검색 품질은 임베딩 모델의 품질과 평가로 뒷받침되어야 한다.

수식

$$\cos(a, b) = (a \cdot b) / (\|a\| \|b\|) = \sum a_i b_i / (\sqrt{\sum a_i^2} \cdot \sqrt{\sum b_i^2})$$

예시

$a = (1, 0, 1)$, $b = (0, 1, 1)$ 일 때 $a \cdot b = 1$, $|a| = |b| = \sqrt{2}$ 이므로 코사인 유사도는 $1/2 = 0.5$ 다.
두 벡터가 한 성분을 공유하므로 어느 정도 유사하지만 완전히 같지는 않음을 수치가 보여 준다.

15.4 벡터 데이터베이스와 색인

임베딩 검색을 실제로 운영하려면 수백만~수십억 개의 벡터를 저장하고, 질의 벡터와 가장 가까운 것들을 빠르게 찾아야 한다. 벡터 데이터베이스(vector database)는 이를 위한 저장·색인·검색 인프라로, 메타데이터 필터링, 갱신, 확장성, 지속성 같은 운영 기능을 함께 제공한다. FAISS(라이브러리)나 pgvector, 그리고 여러 전용 벡터 DB가 대표적이다.

모든 벡터를 질의와 일일이 비교하는 완전 탐색(brute-force)은 정확하지만 규모가 커지면 지연 시간이 감당하기 어렵다. 그래서 실무에서는 약간의 정확도를 포기하는 대신 훨씬 빠르게 근사 결과를 찾는 색인 구조를 쓴다. 이때 핵심 지표는 재현율(recall, 실제 최근접 이웃을 얼마나 놓치지 않는가)과 지연 시간(latency)·처리량(throughput) 사이의 균형이다.

색인 설계에는 거리 척도 선택, 벡터 정규화 여부, 메모리 사용량, 갱신 빈도 같은 실무적 고려가 얹혀 있다. 또한 벡터 검색만으로는 부족한 경우가 많아, 카테고리·날짜·권한 같은 구조적 조건으로 후보를 좁히는 메타데이터 필터링을 결합한다. 이러한 하이브리드 운영이 실제 서비스 품질을 좌우한다.

운영 관점에서는 저장 이상의 요구가 따른다. 문서가 추가·수정·삭제될 때 색인을 어떻게 갱신할지, 임베딩 모델을 교체하면 전체 벡터를 어떻게 재생성(재색인)할지, 트래픽이 늘 때 어떻게 수평 확장할지, 그리고 장애 시 데이터를 어떻게 지속·복구할지가 모두 설계 대상이 된다. 벡터 데이터베이스는 이러한 생애 주기 관리를 뒷받침해야 실제 서비스에서 신뢰할 수 있는 검색 계층이 된다.

15.5 근사 최근접 이웃(ANN): HNSW와 IVF

근사 최근접 이웃(ANN, Approximate Nearest Neighbor) 탐색은 정확한 최근접 이웃을 보장하는 대신 대부분 맞히면서 속도를 크게 높이는 기법의 총칭이다. 대표적으로 그래프 기반의 HNSW와 클러스터 기반의 IVF가 있으며, 여기에 벡터를 압축하는 양자화(quantization)를 결합하기도 한다.

HNSW(Hierarchical Navigable Small World)는 벡터들을 여러 층의 그래프로 연결한 구조다. 상위 층은 노드가 성기게 연결되어 먼 거리를 빠르게 건너뛰고, 하위 층은 촘촘히 연결되어 정밀하게 탐색한다. 질의는 최상위 층에서 시작해 점점 가까운 이웃으로 내려가며 후보를 좁힌다. 높은 재현율과 빠른 검색 속도로 널리 쓰이지만, 그래프를 메모리에 유지해야 해 메모리 사용량이 크다는 특성이 있다.

IVF(Inverted File Index)는 먼저 벡터 공간을 다수의 군집으로 나누어 각 벡터를 가장 가까운 군집 중심에 배정한다. 질의가 오면 가까운 몇 개의 군집만 골라 그 안의 벡터만 비교하므로 탐색 범위가 크게 줄어든다. 탐색할 군집 수(nprobe)를 늘리면 재현율이 오르지만 느려지는 절충이 있다. 여기에 곱 양자화(PQ, Product Quantization)를 더하면 벡터를 압축해 메모리를 절약하면서 대규모 검색이 가능해진다. 실무에서는 데이터 규모·메모리 예산·요구 재현율에 맞추어 이들

방식을 선택하고 파라미터를 조정한다.

예시

1000만 개 문서 임베딩에서 완전 탐색은 질의마다 1000만 번 비교가 필요하지만, IVF로 공간을 1024개 군집으로 나눈 뒤 $nprobe=16$ 으로 두면 평균 16/1024 수준의 벡터만 비교해도 되어 검색이 수십 배 빨라진다. 대신 정답 이웃을 이따금 놓칠 수 있어 재현율을 함께 관찰한다.

15.6 임베딩 모델·청킹이 검색 품질에 미치는 영향

검색 품질의 상한은 임베딩 모델이 결정한다. 도메인 적합성(일반 웹 vs. 의료·법률·코드), 지원 언어, 임베딩 차원, 처리할 수 있는 최대 입력 길이, 그리고 비대칭 검색(짧은 질의와 긴 문서를 잘 매칭하도록 학습되었는지) 여부가 중요한 선택 기준이다. 같은 데이터라도 모델을 바꾸면 검색 결과가 크게 달라질 수 있으므로, 실제 데이터에 대한 평가로 모델을 고르는 것이 바람직하다.

긴 문서는 임베딩 전에 청킹(chunking)으로 잘게 나눈다. 청크가 너무 크면 한 벡터에 여러 주제가 섞여 질의와의 정밀한 매칭이 어렵고, 너무 작으면 문맥이 끊겨 의미가 온전히 담기지 않는다. 그래서 문장·문단 경계를 존중하며 적절한 크기로 나누고, 청크 사이에 약간의 겹침(overlap)을 두어 경계에서 문맥이 잘리는 것을 방지하는 전략이 흔히 쓰인다.

색인 단계의 세부 선택도 품질에 영향을 준다. 어떤 텍스트를 임베딩할지(제목·요약을 덧붙일지), 메타데이터를 어떻게 붙일지, 갱신된 문서를 어떻게 재색인할지 등이 검색 결과의 정확성과 최신성을 좌우한다. 이처럼 임베딩 모델 선택, 청킹 전략, 색인 설계는 서로 맞물려 검색 시스템 전체의 성능을 결정하며, 이는 다음 장의 RAG 파이프라인에서 그대로 이어지는 핵심 설계 요소다.

이 장의 요약

- 임베딩은 이산적 기호를 의미가 근접성으로 반영되는 밀집 벡터 공간에 배치하는 표현 학습의 산물이다.
- 분포 가설에 따라 비슷한 문맥에 나타나는 단어는 비슷한 벡터를 갖게 학습된다.
- word2vec·GloVe·fastText는 정적 단어 임베딩이고, 트랜스포머 기반 문맥 임베딩은 문맥에 따라 벡터가 달라진다.
- 검색에는 문장·문서 임베딩이 필요하며, Sentence-BERT류의 대조 학습 모델이 유사도 반영에 강하다.
- 코사인 유사도는 벡터 크기와 무관하게 방향 일치도로 의미 유사도를 재며, 정규화하면 내적과 동일해진다.
- 벡터 데이터베이스는 대규모 임베딩의 저장·색인·검색과 메타데이터 필터링을 제공한다.
- 근사 최근접 이웃(ANN)은 재현율과 속도를 절충하며, HNSW는 그래프 기반, IVF는 군집 기반 색인이다.
- 임베딩 모델의 도메인·언어 적합성과 청킹·색인 전략이 검색 품질을 함께 결정한다.

핵심 용어

임베딩(Embedding)	단어·문장·문서를 의미 관계가 반영되도록 학습한 밀집 실수 벡터 표현이다.
표현 학습(Representation Learning)	데이터를 과제에 유용한 벡터 표현으로 자동 변환하도록 학습하는 접근이다.
분포 가설(Distributional Hypothesis)	비슷한 문맥에 등장하는 단어는 비슷한 의미를 가진다는 언어학적 전제다.
word2vec	주변 단어 예측(CBOW/Skip-gram)으로 학습하는 대표적 정적 단어 임베딩 기법이다.
정적 임베딩(Static Embedding)	단어마다 문맥과 무관하게 하나의 고정 벡터를 부여하는 임베딩이다.
문맥 임베딩(Contextual Embedding)	같은 단어라도 문장 속 문맥에 따라 다른 벡터를 부여하는 트랜스포머 기반 임베딩이다.
문장 임베딩(Sentence Embedding)	문장 전체의 의미를 하나의 벡터로 표현한 것으로, 의미 검색과 유사도 판단에 쓰인다.
코사인 유사도(Cosine Similarity)	두 벡터가 이루는 각도의 코사인으로 크기와 무관하게 방향 일치도를 재는 유사도 척도다.
정규화(Normalization)	벡터를 단위 길이로 만드는 처리로, 정규화 후에는 내적이 코사인 유사도와 같아진다.
의미 검색(Semantic Search)	표면적 단어가 아니라 임베딩의 근접성으로 관련 문서를 찾는 검색 방식이다.
벡터 데이터베이스(Vector Database)	임베딩의 저장·색인·근접 검색과 메타데이터 필터링을 제공하는 데이터 인프라다.
근사 최근접 이웃(ANN)	정확도를 약간 희생하는 대신 최근접 이웃을 매우 빠르게 근사 탐색하는 기법의 총칭이다.
HNSW	다층 그래프로 벡터를 연결해 상위 층에서 하위 층으로 좁혀 탐색하는 그래프 기반 ANN 색인이다.
IVF(Inverted File Index)	벡터 공간을 군집으로 나눠 가까운 군집만 탐색하는 클러스터 기반 ANN 색인이다.
곱 양자화(Product Quantization)	벡터를 부분 공간별로 압축 부호화해 메모리를 절약하는 양자화 기법이다.
청킹(Chunking)	긴 문서를 임베딩·검색에 적합한 크기의 조각으로 나누는 전처리 과정이다.

검색 증강 생성(RAG)과 생성형 AI의 활용·윤리

대규모 언어모델은 방대한 지식을 담고 있지만, 학습 시점 이후의 최신 정보나 조직 내부의 사적 자료는 알지 못하며, 모르는 것도 그럴듯하게 지어내는 환각(hallucination) 문제를 갖는다. 검색 증강 생성(RAG, Retrieval-Augmented Generation)은 질문에 관련된 문서를 외부에서 검색해 프롬프트에 함께 제공함으로써, 모델이 근거에 기반해 답하도록 만드는 방법이다. 이 장에서는 RAG의 동기와 전체 파이프라인, 청킹·검색·리랭킹 전략, 프롬프트 엔지니어링 기초, 환각 완화, 검색·생성 양면의 평가, 그리고 생성형 AI의 한계·편향·윤리·안전과 실무 적용 팁을 다룬다. 이 교재의 실험 프로젝트가 다루는 바로 그 주제다.

16.1 RAG의 동기: 최신성과 환각 완화

LLM의 지식은 학습 데이터에 고정되어 있다. 학습이 끝난 뒤 세상이 바뀌어도 모델은 그 사실을 알지 못하며(지식 컷오프), 회사 내부 문서나 특정 사용자의 데이터처럼 학습에 포함되지 않은 정보도 답할 수 없다. 모델 자체를 다시 학습하거나 파인튜닝해 지식을 넣는 것은 비용이 크고, 최신 정보를 즉시 반영하기도 어렵다.

RAG는 모델의 가중치를 바꾸지 않고, 답변 시점에 관련 문서를 검색해 프롬프트에 끼워 넣는 방식으로 이 문제를 푼다. 모델은 자신의 내부 지식 대신 제공된 문서를 근거로 답하므로, 지식 베이스만 갱신하면 최신 정보를 즉시 반영할 수 있고 출처를 함께 제시해 검증 가능성을 높인다. 이는 환각을 줄이는 강력한 수단이 된다.

다만 RAG가 환각을 완전히 없애지는 못한다. 검색이 엉뚱한 문서를 가져오면 모델은 잘못된 근거로 답하고, 관련 문서가 있어도 모델이 무시하거나 잘못 해석할 수 있다. 즉 RAG의 답변 품질은 검색 품질에 크게 의존하며, 생성 단계 못지않게 검색 단계의 설계가 중요하다는 점을 처음부터 인식해야 한다.

예시

'우리 회사의 2025년 연차 정책은?'이라는 질문에 일반 LLM은 학습에 없는 내용을 그럴듯하게 지어낼 위험이 있다. RAG는 사내 인사 규정 문서에서 해당 조항을 검색해 프롬프트에 넣어, 모델이 실제 규정에 근거해 답하고 출처 문서를 함께 제시하게 한다.

16.2 RAG 파이프라인 개관

RAG는 크게 오프라인 색인 단계와 온라인 질의 단계로 나뉜다. 색인 단계에서는 문서를 적재하고 청킹한 뒤 임베딩해 벡터 데이터베이스에 색인해 둔다. 질의 단계에서는 사용자의 질문을 같은 임베딩 공간으로 변환해 관련 청크를 검색하고, 이를 프롬프트에 결합해 LLM이 답을 생성한다. 검색과 생성이 하나의 흐름으로 이어지는 것이 RAG의 골격이다.

각 단계의 선택이 최종 품질에 누적적으로 영향을 준다. 어떻게 청킹할지, 어떤 임베딩 모델과 색인을 쓸지, 몇 개의 청크를 가져올지(top-k), 어떻게 프롬프트를 구성할지, 어떤 생성 파라미터를 쓸지가 모두 결과를 좌우한다. 그래서 RAG 개발은 파이프라인 각 지점을 측정하고

개선하는 반복적 실험 과정이 된다.

아래 의사코드는 전형적인 RAG 파이프라인의 두 단계를 요약한 것이다. 실제 시스템에서는 여기에 하이브리드 검색, 리랭킹, 질의 재작성, 출처 인용, 캐싱 같은 요소가 더해진다.

의사코드

```
# --- 오프라인: 색인 구축 ---
함수 build_index(문서들):
    청크들 = []
    각 문서 d에 대해:
        조각들 = chunk(d, 크기=500토큰, 겹침=50토큰) # 청킹
        청크들 += 조각들
    벡터들 = embed(청크들) # 임베딩 모델로 벡터화
    index = vector_db.build(벡터들, 메타데이터=청크들)
    반환 index

# --- 온라인: 질의 응답 ---
함수 rag_answer(질문, index, k=5):
    q_vec = embed(질문) # 질문 임베딩
    후보 = index.search(q_vec, top_k=20) # 근사 최근접 이웃 검색
    상위 = rerank(질문, 후보)[:k] # 리랭킹 후 상위 k개 선택
    context = concat(상위, 출처표시=True)
    prompt = build_prompt(질문, context) # 지시 + 근거 문서 결합
    답변 = LLM.generate(prompt)
    반환 답변, 상위 # 답변과 출처를 함께 반환
```

16.3 청킹 전략

청킹은 RAG 품질의 숨은 결정 요인이다. 검색은 청크 단위로 이루어지므로, 하나의 청크는 질의에 답하기에 충분한 문맥을 담으면서도 여러 주제가 뒤섞이지 않을 만큼 응집적이어야 한다. 청크가 지나치게 크면 무관한 내용까지 함께 검색되어 정밀도가 떨어지고, 너무 작으면 답의 근거가 여러 청크로 흩어져 하나만 가져와서는 답할 수 없게 된다.

실무에서 널리 쓰이는 방식은 고정 크기 청킹에 겹침을 더하는 것이다. 예컨대 300~800토큰 크기로 나누되 인접 청크가 10~20% 정도 겹치게 하여 경계에서 문맥이 잘리는 것을 완화한다. 더 나아가 문장·문단·제목 같은 문서 구조를 존중하는 의미 기반 청킹, 마크다운·코드의 구조를 이용하는 재귀적 분할, 표나 코드 블록을 통째로 유지하는 규칙 등이 활용된다.

청킹은 메타데이터·계층 구조와 결합할 때 더 강력해진다. 각 청크에 문서 제목·섹션 제목·작성일·출처를 붙여 두면 검색 필터링과 출처 인용에 활용할 수 있고, 작은 청크로 검색하되 답변 생성에는 그 청크가 속한 더 넓은 문맥을 함께 제공하는 부모-자식(parent-child) 전략으로 정밀도와 문맥 충분성을 동시에 챙길 수 있다. 정답이 정해진 최적 청킹은 없으며, 문서 성격과 질의 유형에 맞추어 실험으로 정해야 한다.

예시

긴 기술 매뉴얼을 500토큰·겹침 50토큰으로 나누면, '오류 코드 E-102' 설명이 청크 경계에 걸치더라도 겹침 덕분에 인접 청크 양쪽에 온전한 설명이 남아 어느 청크가 검색되어도 답할 수 있다.

16.4 검색 방식: 밀집·희소·하이브리드와 리랭킹

검색 방식은 크게 희소 검색과 밀집 검색으로 나뉜다. 희소 검색(sparse retrieval)은 BM25처럼 단어의 출현 빈도를 이용하는 어휘 기반 방법으로, 정확한 키워드·고유명사·전문 용어 매칭에 강하고 해석이 쉽다. 밀집 검색(dense retrieval)은 임베딩의 근접성으로 의미가 비슷한 문서를 찾아, 표현이 달라도 의미가 통하면 매칭한다. 둘은 서로의 약점을 보완한다.

그래서 실무에서는 두 방식을 함께 쓰는 하이브리드 검색(hybrid retrieval)이 흔하다. 각 방식의 점수를 정규화해 가중 결합하거나, 두 결과의 순위를 융합하는 RRF(Reciprocal Rank Fusion) 같은 기법으로 합친다. 이렇게 하면 키워드 정확성과 의미 유연성을 모두 취할 수 있어, 어느 한쪽만 쓸 때보다 안정적인 검색 품질을 얻는다.

검색으로 후보를 넉넉히 가져온 뒤 정밀하게 다시 순위를 매기는 리랭킹(re-ranking)도 중요하다. 1차 검색은 속도를 위해 질의와 문서를 각각 임베딩해 비교하지만, 리랭커는 질의와 문서를 함께 입력받아 상호작용을 보는 교차 인코더(cross-encoder)를 써서 훨씬 정밀하게 관련성을 평가한다. 비용이 크므로 상위 후보 수십 개에만 적용해, 최종적으로 프롬프트에 넣을 소수의 청크를 고른다.

예시

'ISO 27001 통제 항목 A.9.2'처럼 정확한 식별자가 든 질의는 BM25가 잘 잡고, '접근 권한 관리 요구사항'처럼 개념적 질의는 임베딩이 잘 잡는다. 하이브리드 검색은 두 유형의 질의 모두에서 놓치지 않도록 돕는다.

16.5 프롬프트 엔지니어링과 환각 완화

검색된 문서를 어떻게 프롬프트로 조립하는지가 답변 품질을 좌우한다. 좋은 RAG 프롬프트는 모델에게 '제공된 문맥에만 근거해 답하라', '문맥에 없으면 모른다고 답하라', '근거가 된 출처를 인용하라' 같은 지시를 명확히 준다. 지시와 문맥과 질문을 구획으로 구분하고, 출력 형식을 지정하며, 필요하면 예시(퓨샷)를 제공하는 것도 도움이 된다.

환각을 줄이는 기법은 여러 층위에 걸쳐 있다. 검색 층위에서는 관련성 높은 문서를 확실히 가져오도록 하이브리드·리랭킹을 강화하고, 근거가 부족하면 답을 거절하도록 설계한다. 생성 층위에서는 문맥 근거를 요구하는 지시, 낮은 온도(temperature) 설정, 답변에 인용을 강제하는 형식이 도움이 된다. 나아가 생성한 답이 실제로 문맥에 뒷받침되는지 검증하는 사후 점검(faithfulness 확인)을 붙이기도 한다.

질의 자체를 다듬는 기법도 효과적이다. 모호한 질문을 명확히 재작성하거나 여러 하위 질문으로 분해하는 질의 재작성(query rewriting), 가상의 이상적 답변을 먼저 만들어 그것으로 검색하는 HyDE, 필요한 정보를 여러 번에 걸쳐 검색·추론하는 다단계 검색 등이 있다. 다만 기법을 늘릴수록 지연 시간과 비용이 커지므로, 실제 개선 효과를 측정하며 필요한 만큼만 도입하는 균형이 필요하다.

예시

프롬프트에 '아래 [문맥]에 있는 정보만 사용해 답하고, 문맥에 근거가 없으면 "제공된 자료로는 확인할 수 없습니다"라고 답하라. 각 주장 끝에 [출처 번호]를 표기하라.'는 지시를 넣으면, 모델이 근거 없는 추측을 하는 대신 모른다고 답하는 비율이 높아진다.

16.6 RAG의 평가: 검색과 생성

RAG는 검색과 생성 두 요소로 이루어지므로 평가도 두 축으로 나누어 본다. 검색 품질은 정보 검색의 고전적 지표로 측정한다. 정답 문서를 상위 k개 안에 얼마나 포함하는지(recall@k, hit rate), 상위 결과의 관련성 순위가 얼마나 좋은지(MRR, nDCG) 등이 대표적이다. 이 지표들은 검색 단계의 병목을 진단하는 데 유용하다.

생성 품질은 답변이 근거에 충실한지(faithfulness/groundedness, 문맥으로 뒷받침되는지), 질문에 적절히 답하는지(answer relevance), 그리고 검색된 문맥이 실제로 답에 도움이 되었는지(context relevance/precision)로 평가한다. 정답이 명확한 과제에서는 정확도나 정답과의 일치도를 쓰고, 개방형 답변에서는 사람 평가나 잘 설계된 LLM 심판(LLM-as-a-judge)을 활용하기도 한다.

실무에서는 대표 질의와 기대 답변·근거로 이루어진 평가 세트를 구축해 파이프라인 변경마다 회귀 검증을 한다. RAGAS 같은 프레임워크가 위 지표들의 자동 산출을 돕는다. 다만 자동 지표는 완벽하지 않으므로, 지표 개선이 실제 사용자 경험 향상으로 이어지는지 정성적 검토를 병행하는 것이 바람직하다. 평가 없는 RAG 개선은 방향을 알 수 없는 추측에 불과하다.

예시

임베딩 모델 A와 B를 비교할 때, 200개 질의로 recall@5를 재어 A가 0.72, B가 0.81이면 B의 검색이 더 낫다고 판단한다. 이어 같은 검색 결과로 생성한 답의 faithfulness를 측정해 최종 답변 품질까지 함께 확인한다.

16.7 생성형 AI의 한계·편향·윤리와 실무 팁

생성형 AI는 강력하지만 분명한 한계를 안고 있다. 사실과 다른 내용을 자신 있게 말하는 환각, 학습 데이터에 담긴 사회적 편향의 재생산, 최신성 부족, 그리고 논리적 일관성의 취약함이 대표적이다. 또한 저작권 있는 자료의 학습·출력, 개인정보 노출, 악용(허위정보·피싱·유해 콘텐츠 생성) 가능성 같은 윤리·안전 문제도 함께 고려해야 한다.

편향과 공정성은 특히 신중히 다뤄야 한다. 모델은 학습 말뭉치의 통계를 반영하므로 성별·인종·지역에 관한 고정관념을 답변에 실을 수 있다. 이를 줄이려면 데이터·프롬프트·후처리 각 단계의 완화 노력과 함께, 채용·대출·의료처럼 영향이 큰 영역에서는 사람의 검토를 두는 인간 참여(human-in-the-loop) 설계, 결정의 근거를 설명하는 투명성, 그리고 개인정보·보안 규정 준수가 요구된다.

실무 적용의 원칙은 명확하다. 첫째, RAG로 답변을 근거에 묶고 출처를 제시해 검증 가능성을 확보한다. 둘째, 시스템 프롬프트와 입력 검증으로 프롬프트 주입(prompt injection)과 민감정보 유출을 방어한다. 셋째, 파이프라인의 각 단계를 지표로 측정하며 점진적으로 개선하고, 실패 사례를 모아 지속적으로 보완한다. 넷째, 모델의 답을 맹신하지 않고 중요한 결정에는 인간

검토를 결합한다. 생성형 AI는 유능한 조수이지 최종 권위가 아니며, 그 한계를 이해하고 설계에 반영하는 것이 책임 있는 활용의 핵심이다.

예시

고객 지원 챗봇을 RAG로 구축할 때, 답변마다 근거 문서 링크를 노출하고 '자료로 확인 불가'인 질문은 상담원 연결로 넘기며, 개인정보가 포함된 질의는 마스킹 후 처리하도록 설계하면 정확성·안전성·신뢰를 함께 확보할 수 있다.

이 장의 요약

- RAG는 답변 시점에 관련 문서를 검색해 프롬프트에 넣어, 지식 최신성을 확보하고 환각을 완화하며 출처로 검증 가능성을 높인다.
- RAG 파이프라인은 오프라인 색인(적재→청킹→임베딩→색인)과 온라인 질의(검색→프롬프트 구성→생성)로 이루어진다.
- 청킹은 크기·겹침·문서 구조·메타데이터를 조합하며, 부모-자식 전략으로 정밀도와 문맥을 함께 챙길 수 있다.
- 검색은 희소(BM25)·밀집(임베딩)·하이브리드로 나뉘고, 교차 인코더 리랭킹으로 최종 후보의 관련성을 정밀화한다.
- RAG 프롬프트는 문맥 근거 사용·근거 없을 때 거절·출처 인용을 명시해 환각을 줄인다.
- 질의 재작성·HyDE·다단계 검색 등은 검색 품질을 높이지만 지연·비용과 균형을 맞춰야 한다.
- 평가는 검색(recall@k, MRR, nDCG)과 생성(faithfulness, answer/context relevance)을 나누어 측정하며 회귀 검증이 필수다.
- 생성형 AI는 환각·편향·최신성·저작권·프라이버시 한계를 가지므로, 근거 제시·인간 검토·보안 방어를 갖춘 책임 있는 설계가 필요하다.

핵심 용어

검색 증강 생성(RAG)	질문에 관련된 외부 문서를 검색해 프롬프트에 넣어 근거 기반으로 답을 생성하는 방법이다.
환각(Hallucination)	모델이 사실이 아닌 내용을 그럴듯하게 지어내어 답하는 현상이다.
지식 컷오프(Knowledge Cutoff)	모델이 학습한 데이터의 시점 이후 정보를 알지 못하는 한계다.
청킹(Chunking)	긴 문서를 검색·임베딩에 적합한 크기의 응집적 조각으로 나누는 전처리다.
겹침(Overlap)	인접 청크가 일부 내용을 공유하게 해 경계에서 문맥이 끊기는 것을 완화하는 기법이다.
희소 검색(Sparse Retrieval)	BM25처럼 단어 빈도에 기반해 키워드를 매칭하는 어휘 기반 검색이다.
밀집 검색(Dense Retrieval)	임베딩의 근접성으로 의미가 유사한 문서를 찾는 검색 방식이다.
하이브리드 검색(Hybrid Retrieval)	희소·밀집 검색의 점수나 순위를 결합해 키워드와 의미를 모두 활용하는 검색이다.
리랭킹(Re-ranking)	1차 검색 후보를 교차 인코더 등으로 다시 정밀하게 순위 매겨 최종 문맥을 고르는 단계다.

교차 인코더(Cross-Encoder)	질의와 문서를 함께 입력받아 상호작용을 보고 관련성을 정밀 평가하는 모델이다.
프롬프트 엔지니어링(Prompt Engineering)	지시·문맥·형식을 설계해 모델이 원하는 답을 내도록 유도하는 기술이다.
질의 재작성(Query Rewriting)	모호한 질문을 명확화하거나 하위 질문으로 분해해 검색 품질을 높이는 기법이다.
충실성(Faithfulness)	생성된 답이 제공된 문맥에 실제로 근거하는 정도를 나타내는 RAG 평가 지표다.
recall@k	정답 문서를 상위 k개 검색 결과에 포함하는 비율로 검색 재현율을 측정하는 지표다.
LLM 심판(LLM-as-a-judge)	개방형 답변의 품질을 다른 LLM으로 자동 평가하는 방법이다.
프롬프트 주입(Prompt Injection)	악의적 입력으로 시스템 지시를 무력화하거나 원치 않는 동작을 유도하는 공격이다.